

QUANTITATIVE REASONING 1

| >>> with Python

Comprehensive Course Notes • Python Edition

Every example solved by hand and with a Python program

• Google Colab guide • Real outputs and charts • Exercises, MCQs and answer key

```
marks.ipynb
1 import statistics as st
2 marks = [12, 25, 37, 18, 42]
3 print(st.mean(marks))
```

Output

26.8

4

Units incl. Python

203

Solved Examples

252

Python Programs

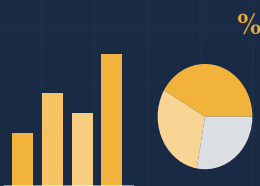
133

MCQs

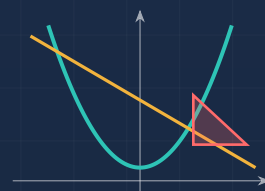
SYLLABUS AT A GLANCE

```
>>> print(2 + 3)
5
>>> x = 12
```

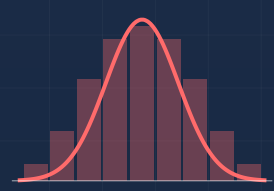
UNIT 0
Python & Colab



UNIT 1
Numerical Literacy



UNIT 2
Mathematical Concepts



UNIT 3
Statistical Concepts

PREPARED BY

Muhammad Idrees

M.Phil (Mathematics)

Lecturer, Department of Mathematics

Government Boys Postgraduate College
Sariab Road, Quetta

idrees@idrees.pk

COURSE AT A GLANCE

CREDITS
03

PRE-REQUISITE
Nil

PLACEMENT
1 – 4 Semesters

TYPE
Mandatory

RUNS ON
Google Colab (free)

LANGUAGE
Python 3

Course Overview

Description

Quantitative Reasoning (I) is an introductory undergraduate course on the fundamentals of quantitative concepts and analysis. It familiarises students with the basic concepts of mathematics and statistics and develops their ability to analyse and interpret quantitative information through theory and practical exercises.

Course Learning Outcomes (CLOs)

By the end of this course, students shall have:

- CLO 1:** Fundamental numerical literacy to work with numbers, understand their meaning and present data accurately;
- CLO 2:** Understanding of fundamental mathematical and statistical concepts;
- CLO 3:** Basic ability to interpret data presented in various formats, including tables, graphs, charts and equations.

How these notes are organised

Every topic of the syllabus follows the same pattern:

Key Concepts	definitions and formulas in simple words
Examples	easy examples, each solved step by step
Exercise	short practice questions (answers at the end)
MCQs	quick self-test (answers at the end)
Summary	the main points to remember
>>> Python Solution	the same example solved by a Python program, with its real output

Using this Python edition

- **Start with Unit 0.** It teaches the Python you need and shows how to run programs free on **Google Colab** in a web browser – no installation is required.
- **Solve by hand first.** Read each example and its step-by-step solution, then study the Python program that follows. The program repeats the same steps, so you can see the mathematics and the code side by side.
- **Type and run every program.** Each program is complete on its own: copy it into a new Colab cell and press Shift + Enter. The output printed in the book is the real output of the program (random simulations use `random.seed` so you get the same numbers).
- **Check your exercises with Python.** Exercises are meant to be solved by hand; then use the programs as templates to check your answers.
- The programs are also supplied as ready-made Colab notebooks (one per unit): open Colab, choose **File** → **Upload notebook**, and select the `.ipynb` file.

Syllabus Map

Unit 0	Python Fundamentals and Google Colab (Python edition): running Python in Colab; variables and arithmetic; printing results; lists, sets and dictionaries; conditions; loops; functions; the math, fractions, statistics, random, itertools, sympy, numpy, matplotlib and scipy libraries.
Unit 1	Numerical Literacy: number system and arithmetic; units, dimensions, area, perimeter, volume; rates, ratios, proportions, percentages; types and sources of data; measurement scales; tabular and graphical presentation of data; QR exercises using number knowledge.
Unit 2	Fundamental Mathematical Concepts: basics of geometry; sets and their operations; relations, functions and graphs; exponents, factoring and simplifying expressions; algebraic and graphical solutions of linear and quadratic equations and inequalities; QR exercises.
Unit 3	Fundamental Statistical Concepts: population and sample; central tendency, dispersion and data interpretation; rules of counting; basic probability; random variables and probability distributions; QR exercises.

Suggested Reading

1. Madison, B. L. & Steen, L. A. – *Quantitative Reasoning: Tools for Today's Informed Citizen*.
2. Madison, B. L. & Bressoud, D. M. – *Quantitative Reasoning for the Information Age*.
3. Ellis, W. – *Fundamentals of Mathematics*.
4. Zaslow, E. – *Quantitative Reasoning: Thinking in Numbers*.
5. Bueno de Mesquita, E. & Fowler, A. – *Thinking Clearly with Data*.
6. Bennett, J. O., Briggs, W. L. & Badalamenti, A. – *Using and Understanding Mathematics*.
7. Rosen, K. H. – *Discrete Mathematics and its Applications*.
8. Chatfield, C. – *Statistics for Technology*.
9. Lock, R. H. et al. – *Statistics: Unlocking the Power of Data*.

Contents

Course Overview	1
Description	1
Course Learning Outcomes (CLOs)	1
How these notes are organised	1
Using this Python edition	1
Syllabus Map	2
Suggested Reading	2
0 Python Fundamentals and Google Colab	5
0.1 Getting Started: Python and Google Colab	5
A first look at Google Colab	6
0.2 Variables, Data Types and Arithmetic	10
0.3 Strings, Printing and Formatting Output	14
0.4 Lists, Tuples, Dictionaries and Sets	17
0.5 Making Decisions: Conditions	20
0.6 Repetition: Loops and Comprehensions	23
0.7 Functions	26
0.8 Python Libraries for Mathematics	29
0.9 Algebra with SymPy	33
0.10 Matplotlib, NumPy and SciPy	35
1 Numerical Literacy	41
1.1 Number System and Basic Arithmetic Operations	41
1.2 Units, Conversions, Dimensions, Area, Perimeter and Volume	50
1.3 Rates, Ratios, Proportions and Percentages	57
1.4 Types and Sources of Data	66
1.5 Measurement Scales	71
1.6 Tabular and Graphical Presentation of Data	75
1.7 Quantitative Reasoning Exercises Using Number Knowledge	85
2 Fundamental Mathematical Concepts	91
2.1 Basics of Geometry (Lines, Angles, Circles, Polygons)	91
2.2 Sets and their Operations	98
2.3 Relations, Functions and their Graphs	104
2.4 Exponents, Factoring and Simplifying Algebraic Expressions	112
2.5 Linear and Quadratic Equations and Inequalities (Algebraic and Graphical)	120

2.6	Quantitative Reasoning Exercises Using Mathematical Concepts	129
3	Fundamental Statistical Concepts	135
3.1	Population and Sample	135
3.2	Measures of Central Tendency, Dispersion and Data Interpretation	141
3.3	Rules of Counting (Multiplicative, Permutation and Combination)	149
3.4	Basic Probability Theory	155
3.5	Introduction to Random Variables and their Probability Distributions	162
3.6	Quantitative Reasoning Exercises Using Statistical Concepts	169
	Answer Key	176

UNIT 0

Python Fundamentals and Google Colab

Why this unit?

Every example in Units 1–3 is solved twice: first **by hand**, step by step, and then with a short **Python program** that repeats the same steps. This unit teaches exactly the Python you need for that – nothing more. You do *not* need to install anything: all programs run free in your web browser on **Google Colab**. Work through the examples in order, type every program yourself, and run it.

Section	You will learn	Used later in
0.1	Google Colab, first program, errors	every unit
0.2–0.3	variables, arithmetic, printing results	Unit 1
0.4–0.7	lists, sets, conditions, loops, functions	Units 1–3
0.8	math, fractions, statistics, random, itertools	Units 1 and 3
0.9	algebra with sympy	Unit 2
0.10	charts with matplotlib, probability with scipy	Units 1–3

0.1 Getting Started: Python and Google Colab

Key Concepts

- **Python** is a free, easy-to-read programming language. It is widely used in mathematics, statistics, data science and artificial intelligence.
- A **program** is a list of instructions. Python carries them out from top to bottom.
- **Google Colab** (Colaboratory) is a free website where you write and run Python in a *notebook*. It needs only a web browser and a Google (Gmail) account; the common mathematics libraries are already installed.
- A **notebook** (file type `.ipynb`) is made of **cells**: *code cells* (Python) and *text cells* (notes). Each code cell shows its output just below it.

A first look at Google Colab



1 notebook name (click to rename) 2 menus – *Runtime* runs or restarts cells 3 add a code or text cell
4 run button (or **Shift + Enter**) 5 the output appears under the cell 6 connection to Google's computer

Your first notebook – step by step

1. Open a web browser (Chrome is best) and go to `colab.research.google.com`.
2. Sign in with your Google (Gmail) account.
3. Click **File** → **New notebook**. A new notebook named `Untitled0.ipynb` opens.
4. Click the name at the top-left and rename it, e.g. `QR1_Unit1`.
5. Type a program in the grey code cell, e.g. `print("Hello")`.
6. Run the cell: click the round **play button** on its left, or press **Shift + Enter**. The first run takes a few seconds while Colab connects.
7. Read the output under the cell. Add new cells with **+ Code** or **+ Text**.
8. Your work is saved automatically in Google Drive, in the folder **Colab Notebooks**. Use **File** → **Download** to save a `.ipynb` or `.py` copy.
9. If something goes wrong, use **Runtime** → **Restart session** and run the cells again (**Runtime** → **Run all** runs every cell from the top).
10. To open the notebooks that come with this book: **File** → **Upload notebook**, then choose the `.ipynb` file.

Shortcut	Action in Colab
Shift + Enter	run the cell and move to the next one
Ctrl + Enter	run the cell and stay on it
Ctrl + /	turn selected lines into comments (or back)
Ctrl + M then B	insert a new code cell below
Ctrl + M then D	delete the current cell
Tab	complete a name, e.g. type pri then Tab

Google Colab Tip

- Colab already has numpy, sympy, matplotlib, scipy and pandas. If a library is ever missing, run `!pip install name` in a cell.
- In Colab, the value of the **last line** of a cell is shown automatically, even without `print`.
- Colab works on a mobile browser too, but a laptop or desktop is much easier for typing code.
- Prefer working offline? Install Python from `python.org` (it comes with the IDLE editor), or install *Anaconda* and use *Jupyter Notebook*, which looks and works like Colab.

Example 0.1.1: Your first program

Write a program that prints a welcome message and the result of $2 + 3$.

Solution.

Step 1: `print(...)` shows whatever is inside the brackets.

Step 2: Text must be inside quotation marks; numbers and calculations are written without quotes.

▷ Why? Python treats `"2+3"` (in quotes) as text, but `2+3` as a sum to calculate.

>>> Python Program

Example 0.1.1

CODE

```
1 # My first program
2 print("Assalam-o-Alaikum! Welcome to Quantitative Reasoning.")
3 print(2 + 3)
4 print("2 + 3")      # in quotes: printed as text, not calculated
```

OUTPUT

```
Assalam-o-Alaikum! Welcome to Quantitative Reasoning.
5
2 + 3
```

Example 0.1.2: Colab as a calculator

Use a code cell as a calculator to find $15 \times 4 + 7$ and $\frac{100}{8}$.

Solution.

Step 1: Python uses `*` for multiplication and `/` for division.

Step 2: The value of the last line of a cell is displayed automatically, so the final line needs no `print`.

>>> Python Program

Example 0.1.2

CODE

```
1 print(15 * 4 + 7)    # print() shows the result of any line
2 100 / 8              # the LAST line of a cell is shown automatically
```

OUTPUT

```
67
12.5
```

Example 0.1.3: Comments

Write a short program with comments explaining each line.

Solution.

Step 1: Anything after # on a line is a **comment**: Python ignores it.

Step 2: Comments explain your thinking to the reader (and to yourself later).

>>> Python Program

Example 0.1.3

CODE

```
1 # Program: area of a rectangle
2 length = 12          # length in metres
3 width = 8            # width in metres
4 print(length * width) # area in square metres
```

OUTPUT

```
96
```

Example 0.1.4: Importing a module

Use the math module to find $\sqrt{144}$ and the value of π .

Solution.

Step 1: Extra tools are kept in **modules**. `import math` loads the mathematics module.

Step 2: Its tools are used as `math.name`, e.g. `math.sqrt(144)`.

>>> Python Program

Example 0.1.4

CODE

```
1 import math          # load the mathematics module
2
3 print(math.sqrt(144)) # square root
4 print(math.pi)       # the value of pi
```

OUTPUT

```
12.0
3.141592653589793
```

Example 0.1.5: Reading an error message

A student typed `prnt("Hello")`. What happens, and how is it fixed?

Solution.

Step 1: Python stops and shows an **error message**. Read the *last line* first: it names the error.

Step 2: `NameError` means Python does not know the name `prnt` – a spelling mistake.

Step 3: Fix: write `print("Hello")`. Python is case-sensitive, so `Print` is also wrong.

▷ *Why? Errors are normal. Every programmer reads error messages many times a day.*

>>> Python Program

Example 0.1.5

CODE

```
1 prnt("Hello")
```

OUTPUT

```
Traceback (most recent call last):  
...  
NameError: name 'prnt' is not defined
```

Exercise 0.1

1. Write a program that prints your name on one line and your college on the next.
2. What does a Colab cell display if its only line is `7 * 6`?
3. Which key combination runs a cell and moves to the next one?
4. Find and fix the error: `Print("Quetta")`.
5. In which Google Drive folder are Colab notebooks saved automatically?

Multiple Choice Questions – 0.1

1. Google Colab runs Python:
(a) only on Windows
(b) in a web browser
(c) only on phones
(d) without internet
2. The file extension of a Colab notebook is:
(a) `.docx`
(b) `.ipynb`
(c) `.xlsx`
(d) `.pdf`
3. A comment in Python starts with:
(a) `//`
(b) `#`
(c) `--`
(d) `%`
4. The error in `prnt(5)` is a:
(a) `NameError`
(b) `ZeroDivisionError`
(c) `TypeError`
(d) no error
5. To load the mathematics module we write:
(a) `load math`
(b) `import math`
(c) `use math`
(d) `math()`

Summary of 0.1

- Colab: browser + Google account; type code in a cell; run with Shift + Enter; output appears below.
- `print()` shows results; `#` starts a comment; `import` loads a module.
- Read error messages from the last line; check spelling, capitals and brackets.

0.2 Variables, Data Types and Arithmetic

Key Concepts

- A **variable** is a name that stores a value: `length = 12`. The `=` sign means “store”, not “equals”.
- Names may contain letters, digits and `_`, must not start with a digit, and are case-sensitive (Area and area are different). Choose meaningful names: `price`, `total_marks`.
- **Data types:** `int` (whole numbers: 25), `float` (decimals: 3.5), `str` (text: "Quetta"), `bool` (True/False). `type(x)` tells the type.
- **Conversion:** `int("25")`, `float("3.5")`, `str(7)`; rounding: `round(3.14159, 2)` gives 3.14.
- `input("message")` reads text typed by the user (always as `str`).

Operator	Meaning	Example	Result
<code>+</code> <code>-</code>	add, subtract	<code>17 + 5</code> , <code>17 - 5</code>	22, 12
<code>*</code>	multiply	<code>17 * 5</code>	85
<code>/</code>	divide (always a float)	<code>17 / 5</code>	3.4
<code>//</code>	whole-number (floor) division	<code>17 // 5</code>	3
<code>%</code>	remainder (modulus)	<code>17 % 5</code>	2
<code>**</code>	power	<code>2 ** 3</code>	8

Python follows the same order of operations as BODMAS: brackets, then `**`, then `*` `/` `//` `%`, then `+` `-` (left to right).

Example 0.2.1: Variables

Store the length 12 m and width 8 m of a plot in variables and find its area and perimeter.

Solution.

Step 1: Give each quantity a meaningful name and store its value.

Step 2: Write the formulas using the names: `area = length * width`.

▷ Why? If the length changes, we change one line and every result updates.

>>> Python Program

Example 0.2.1

CODE

```
1 length = 12      # metres
2 width = 8        # metres
3 area = length * width
4 perimeter = 2 * (length + width)
5 print("Area =", area, "square metres")
6 print("Perimeter =", perimeter, "metres")
```

OUTPUT

```
Area = 96 square metres
Perimeter = 40 metres
```

Example 0.2.2: Data types

Find the type of 25, 3.5, "Quetta" and True.

Solution.

Step 1: Use `type()` on each value.

Step 2: Whole numbers are `int`, decimals are `float`, text is `str`, and `True/False` are `bool`.

>>> Python Program**Example 0.2.2****CODE**

```
1 print(type(25))
2 print(type(3.5))
3 print(type("Quetta"))
4 print(type(True))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Example 0.2.3: All arithmetic operators

For $a = 17$ and $b = 5$ find $a + b$, $a - b$, $a \times b$, $a \div b$, the quotient, the remainder and a^b .

Solution.

Step 1: Apply each operator from the table above.

Step 2: Note that `/` gives 3.4 while `//` gives the whole-number quotient 3, and `%` gives the remainder 2 (because $17 = 5 \times 3 + 2$).

>>> Python Program**Example 0.2.3****CODE**

```
1 a = 17
2 b = 5
3 print("a + b =", a + b)
4 print("a - b =", a - b)
5 print("a * b =", a * b)
6 print("a / b =", a / b)
7 print("a // b =", a // b)      # quotient
8 print("a % b =", a % b)      # remainder
9 print("a ** b =", a ** b)     # power
```

OUTPUT

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
a // b = 3
a % b = 2
a ** b = 1419857
```

Example 0.2.4: Order of operations

Evaluate $18 \div 3 \times 2 + (5 - 2)^2$ in Python.

Solution.

Step 1: Translate symbols: $\div \rightarrow /$, $\times \rightarrow *$, power $\rightarrow **$.

Step 2: Python applies BODMAS automatically, so the answer is 21 (shown as 21.0 because / gives a float).

Step 3: Brackets change the order: without them, $5 - 2^2$ would give 1, not 9.

>>> Python Program**Example 0.2.4****CODE**

```
1 print(18 / 3 * 2 + (5 - 2) ** 2)    # BODMAS is applied automatically
2 print(18 // 3 * 2 + (5 - 2) ** 2)   # // keeps whole numbers
3 print(5 - 2 ** 2)                   # power first: 5 - 4
4 print((5 - 2) ** 2)                 # brackets first: 3 ** 2
```

OUTPUT

```
21.0
21
1
9
```

Example 0.2.5: Type conversion and rounding

Convert text to numbers, round a decimal, and see what `int()` does to 7.9.

Solution.

Step 1: `int("25")` turns the text "25" into the number 25, so arithmetic works.

Step 2: `round(x, 2)` rounds to 2 decimal places.

Step 3: `int(7.9)` cuts off the decimal part (gives 7); it does not round.

>>> Python Program**Example 0.2.5****CODE**

```
1 print(int("25") + 5)                # text "25" becomes the number 25
2 print(float("3.75") * 2)
3 print(round(3.14159, 2))             # round to 2 decimal places
4 print(int(7.9))                     # int() cuts off the decimal part
5 print(round(7.9))                   # round() rounds to the nearest whole number
```

OUTPUT

```
30
7.5
3.14
7
8
```

Example 0.2.6: Reading input from the user

Write a program that asks for a student's name and marks out of 80, and prints the percentage.

Solution.

Step 1: `input()` shows a message and waits for the user to type; in Colab a box appears under the cell.

Step 2: The typed value is text, so convert marks with `float()` before calculating.

Step 3: Percentage = $\frac{\text{marks}}{80} \times 100$. (Below, the user typed Ali and 68.)

>>> Python Program**Example 0.2.6****CODE**

```
1 name = input("Enter your name: ")
2 marks = float(input("Enter marks out of 80: "))
3 percentage = marks / 80 * 100
4 print(name, "scored", percentage, "percent")
```

OUTPUT

```
Enter your name: Ali
Enter marks out of 80: 68
Ali scored 85.0 percent
```

Exercise 0.2

1. Find the values of `2 ** 10`, `17 // 4` and `17 % 4`.
2. What is the type of `4 / 2`? Why?
3. Write a program to find the area of a circle of radius 5 using $\pi = 3.14159$, rounded to 2 decimals.
4. Which of these are valid variable names: `total_marks`, `2nd_test`, `Price`, `my marks`?
5. What does `int(9.99)` give?

Multiple Choice Questions – 0.2

1. `7 / 2` gives:
(a) 3 (b) 3.5
(c) 1 (d) 3.0
2. `7 // 2` gives:
(a) 3 (b) 3.5
(c) 1 (d) 4
3. `2 ** 3` gives:
(a) 6 (b) 5
(c) 8 (d) 9
4. `type("12")` is:
(a) int (b) float
(c) str (d) bool
5. `10 % 3` gives:
(a) 3 (b) 1
(c) 0 (d) 3.33

Summary of 0.2

- `name = value` stores a value; use clear names.
- Types: `int`, `float`, `str`, `bool`; convert with `int()`, `float()`, `str()`.
- Operators: `+` `-` `*` `/` `//` `%` `**`; Python follows BODMAS.
- `input()` always returns text – convert it before doing arithmetic.

0.3 Strings, Printing and Formatting Output

Key Concepts

- A **string** is text in quotes: `"Quetta"` or `'Quetta'`. Join strings with `+`; repeat with `*`: `"-" * 20`. `len(s)` gives the number of characters.
- **f-strings** put values inside text: `f"Area = {area} m2"`.
- **Format codes** inside the braces control how numbers look:

Code	Meaning	Example (value → shown)
<code>:.2f</code>	2 decimal places	3.14159 → 3.14
<code>:</code> , <code>,</code>	comma separators	1234567 → 1,234,567
<code>:.1%</code>	percentage, 1 decimal	0.756 → 75.6%
<code>:.2e</code>	scientific notation	45000 → 4.50e+04
<code>>8</code> <code><8</code>	right / left align in 8 spaces	for neat tables

Example 0.3.1: Working with strings

Store the word "Quetta"; find its length, write it in capitals, join it with another word and draw a line of dashes.

Solution.

Step 1: `len()` counts characters; `.upper()` makes capitals.

Step 2: `+` joins two strings; `*` repeats a string.

>>> Python Program

Example 0.3.1

CODE

```
1 city = "Quetta"
2 print(len(city))           # number of characters
3 print(city.upper())        # capital letters
4 print(city + " Balochistan") # joining two strings
5 print("-" * 20)            # a line of 20 dashes
```

OUTPUT

```
6
QUETTA
Quetta Balochistan
-----
```

Example 0.3.2: f-strings

A student scored 68 out of 80. Print a sentence giving the marks and the percentage.

Solution.

Step 1: Calculate the percentage in a variable.

Step 2: Put an f before the opening quote and write variables inside { }.

Step 3: Use `:.1f` to show one decimal place.

>>> Python Program**Example 0.3.2****CODE**

```
1 name = "Sara"
2 marks = 68
3 percentage = marks / 80 * 100
4 print(f"{name} scored {marks} out of 80, that is {percentage:.1f}%.")
```

OUTPUT

```
Sara scored 68 out of 80, that is 85.0%.
```

Example 0.3.3: Formatting numbers

Show π to 4 decimal places, 1234567.891 with commas, 0.756 as a percentage and 45000 in scientific notation.

Solution.

Step 1: Choose the right format code from the table and place it after a colon inside the braces.

▷ Why? Formatting changes only how a number is shown, not the value stored.

>>> Python Program**Example 0.3.3****CODE**

```
1 import math
2 print(f"{math.pi:.4f}")           # 4 decimal places
3 print(f"{1234567.891:,.2f}")      # commas and 2 decimals
4 print(f"{0.756:.1%}")             # as a percentage
5 print(f"{45000:.2e}")             # scientific notation
```

OUTPUT

```
3.1416
1,234,567.89
75.6%
4.50e+04
```

Example 0.3.4: Printing a neat table

Print a small table of three items and their prices with the columns lined up.

Solution.

Step 1: Use `:<12` to left-align names in 12 spaces and `:>8` to right-align numbers in 8 spaces.

Step 2: Print a header and a dashed line first.

>>> Python Program

Example 0.3.4

CODE

```

1 print(f"{'Item':<12}{'Price':>8}")
2 print("-" * 20)
3 print(f"{'Notebook':<12}{120:>8}")
4 print(f"{'Pen':<12}{30:>8}")
5 print(f"{'Calculator':<12}{1450:>8}")

```

OUTPUT

Item	Price

Notebook	120
Pen	30
Calculator	1450

Exercise 0.3

1. Print "Quantitative Reasoning" in capital letters and find its length.
2. Write an f-string that prints Total = Rs. 3250 using a variable bill = 3250.
3. Show 2/3 to 3 decimal places.
4. Show 0.35 as a percentage with no decimals.

Multiple Choice Questions – 0.3

1. `len("Python")` is:

(a) 5	(b) 6
(c) 7	(d) 1
2. `"ab" * 3` gives:

(a) "ababab"	(b) "ab3"
(c) error	(d) "aaabbb"
3. `f"{2/3:.2f}"` shows:

(a) 0.6	(b) 0.66
(c) 0.67	(d) 0.667
4. The format code for comma separators is:

(a) :.2f	(b) :,
(c) :%	(d) :e

Summary of 0.3

- Strings are text in quotes; + joins, * repeats, `len()` counts.
- f-strings: `f"...{value:format}..."` – the most useful way to print answers.
- `:.2f` decimals, `:,` commas, `:.1%` percentages, `:.2e` scientific notation.

0.4 Lists, Tuples, Dictionaries and Sets

Key Concepts

Type	Written as	Key property	Used for
list	[12, 25, 37]	ordered, can change	data values, marks
tuple	(3, 4)	ordered, cannot change	points (x, y)
dictionary	{"Food": 24000}	key → value pairs	tables, budgets
set	{1, 2, 3}	no order, no repeats	set theory (Unit 2)

- Positions (**indexes**) start at 0: marks[0] is the first value, marks[-1] the last. **Slicing**: marks[1:3] gives positions 1 and 2.
- Useful functions: len, sum, max, min, sorted; list methods .append(x), .sort(), .count(x).
- Set operations: A | B (union), A & B (intersection), A - B (difference).

Example 0.4.1: Lists

Store the marks 12, 25, 37, 18, 42 in a list. Find the first and last marks, the number of marks, the highest, the lowest, and the marks in order.

Solution.

Step 1: A list is written in square brackets with commas.

Step 2: Index 0 is the first item and index -1 the last.

Step 3: sorted() returns a new ordered list; the original is unchanged.

>>> Python Program

Example 0.4.1

CODE

```
1 marks = [12, 25, 37, 18, 42]
2 print("First mark:", marks[0])
3 print("Last mark :", marks[-1])
4 print("How many  :", len(marks))
5 print("Highest   :", max(marks))
6 print("Lowest    :", min(marks))
7 print("In order  :", sorted(marks))
8 print("Positions 1 to 2:", marks[1:3])
```

OUTPUT

```
First mark: 12
Last mark : 42
How many  : 5
Highest   : 42
Lowest    : 12
In order  : [12, 18, 25, 37, 42]
Positions 1 to 2: [25, 37]
```

Example 0.4.2: Average of a list

Find the mean of the marks above, and add a new mark of 30.

Solution.

Step 1: Mean = $\frac{\text{sum of values}}{\text{number of values}} \Rightarrow \text{sum}(\text{marks}) / \text{len}(\text{marks})$.

Step 2: `.append(30)` adds a value at the end; recompute the mean.

>>> Python Program**Example 0.4.2****CODE**

```
1 marks = [12, 25, 37, 18, 42]
2 mean = sum(marks) / len(marks)
3 print("Mean =", mean)
4 marks.append(30)           # add a new mark
5 print("New list:", marks)
6 print("New mean =", sum(marks) / len(marks))
```

OUTPUT

```
Mean = 26.8
New list: [12, 25, 37, 18, 42, 30]
New mean = 27.333333333333332
```

Example 0.4.3: Tuples

Store the point (3,4) as a tuple and find its distance from the origin.

Solution.

Step 1: `x, y = point` *unpacks* the tuple into two variables.

Step 2: Distance = $\sqrt{x^2 + y^2}$ (Pythagoras); `** 0.5` is the square root.

>>> Python Program**Example 0.4.3****CODE**

```
1 point = (3, 4)
2 x, y = point           # unpack the tuple
3 distance = (x**2 + y**2) ** 0.5  # ** 0.5 is the square root
4 print("x =", x, " y =", y)
5 print("Distance from origin =", distance)
```

OUTPUT

```
x = 3  y = 4
Distance from origin = 5.0
```

Example 0.4.4: Dictionaries

Store a family budget (Food 24000, Rent 18000, Transport 6000) in a dictionary; look up the rent, add Savings 3000 and find the total.

Solution.

Step 1: A dictionary links a **key** (the item) to a **value** (the amount).

Step 2: `budget["Rent"]` looks up a value; `budget["Savings"] = 3000` adds a new pair.

Step 3: `sum(budget.values())` adds all amounts.

>>> Python Program

Example 0.4.4

CODE

```

1 budget = {"Food": 24000, "Rent": 18000, "Transport": 6000}
2 print("Rent:", budget["Rent"])
3 budget["Savings"] = 3000           # add a new item
4 print(budget)
5 print("Total:", sum(budget.values()))

```

OUTPUT

```

Rent: 18000
{'Food': 24000, 'Rent': 18000, 'Transport': 6000, 'Savings': 3000}
Total: 51000

```

Example 0.4.5: Sets

Let $A = \{1, 2, 3, 4, 5\}$ and $B = \{2, 4, 6, 8, 10\}$. Find $A \cup B$, $A \cap B$, $A - B$ and check whether $3 \in A$.

Solution.

Step 1: Python sets behave exactly like mathematical sets.

Step 2: | is union, & is intersection, - is difference, in tests membership.

>>> Python Program

Example 0.4.5

CODE

```

1 A = {1, 2, 3, 4, 5}
2 B = {2, 4, 6, 8, 10}
3 print("A union B      :", A | B)
4 print("A intersection B :", A & B)
5 print("A - B          :", A - B)
6 print("Is 3 in A?     :", 3 in A)

```

OUTPUT

```

A union B      : {1, 2, 3, 4, 5, 6, 8, 10}
A intersection B : {2, 4}
A - B          : {1, 3, 5}
Is 3 in A?     : True

```

Exercise 0.4

1. For data = [5, 9, 2, 7] find data[1], data[-1] and sorted(data).
2. Find the mean of [10, 20, 30, 40] using sum and len.
3. Create a dictionary of three cities and their temperatures and print one value.
4. For $P = \{1, 2, 3\}$ and $Q = \{3, 4\}$, write Python for $P \cup Q$ and $P \cap Q$ and give the results.

Multiple Choice Questions – 0.4

1. For x = [4, 8, 15], x[0] is:

(a) 4	(b) 8
(c) 15	(d) error

2. Which type stores key–value pairs?

(a) list	(b) tuple
(c) dictionary	(d) set
3. {1, 2, 2, 3} has how many elements?

(a) 4	(b) 3
(c) 2	(d) 1
4. The intersection of sets A and B is written:

(a) A B	(b) A & B
(c) A - B	(d) A + B

Summary of 0.4

- Lists [] hold data; indexes start at 0; sum/len gives the mean.
- Tuples () cannot change; dictionaries {key: value} store tables; sets { } remove repeats.
- Set operators | & - match $\cup$, $\cap$ and set difference.

0.5 Making Decisions: Conditions

Key Concepts

- **Comparison operators** give True or False: == (equal), != (not equal), <, >, <=, >=.
- **Logical operators:** and (both true), or (at least one true), not (reverses).
- if / elif / else choose what to do. The lines inside each part are **indented** (4 spaces); Colab indents automatically after a colon.

```
if condition:
    # runs when the condition is True
elif other_condition:
    # runs when the first is False and this is True
else:
    # runs when all conditions are False
```

Example 0.5.1: Pass or fail

A student passes if marks are 50 or more. Decide for marks = 62.

Solution.

Step 1: The condition is marks >= 50.

Step 2: If it is True print “Pass”, otherwise print “Fail”.

▷ Why? Note the colon : at the end of if and else, and the indentation below them.

>>> Python Program

Example 0.5.1

CODE

```
1 marks = 62
2 if marks >= 50:
```

```
3     print("Pass")
4 else:
5     print("Fail")
```

OUTPUT

```
Pass
```

Example 0.5.2: Grades with elif

Give grade A for 80+, B for 65–79, C for 50–64 and F below 50. Find the grade for 72.

Solution.

Step 1: Test the highest band first; Python stops at the first condition that is true.

Step 2: $72 \geq 80$? No. $72 \geq 65$? Yes $\Rightarrow$ grade B.

>>> Python Program**Example 0.5.2****CODE**

```
1 marks = 72
2 if marks >= 80:
3     grade = "A"
4 elif marks >= 65:
5     grade = "B"
6 elif marks >= 50:
7     grade = "C"
8 else:
9     grade = "F"
10 print("Grade:", grade)
```

OUTPUT

```
Grade: B
```

Example 0.5.3: Even or odd

Decide whether 17 is even or odd.

Solution.

Step 1: A number is even if dividing by 2 leaves remainder 0: $n \% 2 == 0$.

Step 2: $17 \% 2 = 1$, so 17 is odd.

>>> Python Program**Example 0.5.3****CODE**

```
1 n = 17
2 if n % 2 == 0:
3     print(n, "is even")
4 else:
5     print(n, "is odd")
```

OUTPUT

```
17 is odd
```

Example 0.5.4: Logical operators

A shop gives a discount if a customer is a student *and* spends at least Rs. 2000, *or* has a loyalty card. Decide for a student who spends Rs. 2500 and has no card.

Solution.

Step 1: Store each fact as True/False.

Step 2: Combine: (is_student and spent >= 2000) or has_card.

Step 3: (True and True) or False $\Rightarrow$ True: discount given.

>>> Python Program**Example 0.5.4****CODE**

```
1 is_student = True
2 spent = 2500
3 has_card = False
4 discount = (is_student and spent >= 2000) or has_card
5 print("Discount given?", discount)
```

OUTPUT

```
Discount given? True
```

Exercise 0.5

1. What do `5 == 5.0`, `3 != 3` and `4 >= 7` give?
2. Write a program that prints "Positive", "Negative" or "Zero" for a number n.
3. What is True and False? True or False? not True?
4. Write a program that decides whether a year (e.g. 2028) is divisible by 4.

Multiple Choice Questions – 0.5

1. The operator to *compare* two values for equality is:

(a) =	(b) ==
(c) :=	(d) ===
2. `(3 > 2)` and `(2 > 5)` is:

(a) True	(b) False
(c) error	(d) None
3. The lines inside an if must be:

(a) in capitals	(b) indented
(c) numbered	(d) in quotes
4. `12 % 2 == 0` tests whether 12 is:

(a) prime	(b) even
(c) odd	(d) positive

Summary of 0.5

- Comparisons give True/False; `==` compares, `=` stores.
- `if/elif/else` with a colon and indentation; the first true branch runs.
- Combine conditions with `and`, `or`, `not`.

0.6 Repetition: Loops and Comprehensions

Key Concepts

- A for loop repeats for each item of a list: `for m in marks:`
- `range(1, 11)` gives 1, 2, ..., 10 (the end number is *not* included); `range(5)` gives 0, 1, 2, 3, 4.
- A while loop repeats *as long as* a condition is true – make sure it eventually becomes false!
- An **accumulator** is a variable that builds up a result: start `total = 0`, then `total += x` in the loop.
- A **list comprehension** builds a list in one line: `[x**2 for x in range(1, 6)]`.

Example 0.6.1: Looping over a list

Print each mark from the list [45, 72, 88, 39] with “Pass” or “Fail”.

Solution.

Step 1: `for m in marks:` takes each mark in turn and runs the indented lines.

Step 2: Inside the loop, use the `if/else` from Section 0.5.

>>> Python Program

Example 0.6.1

CODE

```
1 marks = [45, 72, 88, 39]
2 for m in marks:
3     if m >= 50:
4         print(m, "Pass")
5     else:
6         print(m, "Fail")
```

OUTPUT

```
45 Fail
72 Pass
88 Pass
39 Fail
```

Example 0.6.2: Using range

Print the multiplication table of 7 from 7×1 to 7×10 .

Solution.

Step 1: `range(1, 11)` gives the numbers 1 to 10.

Step 2: For each number `i`, print `7 * i`.

>>> Python Program

Example 0.6.2

CODE

```
1 for i in range(1, 11):
2     print(f"7 x {i:>2} = {7 * i:>3}")
```

OUTPUT

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Example 0.6.3: Accumulating a sum

Find $1 + 2 + 3 + \dots + 100$ with a loop, and check with the formula $\frac{n(n+1)}{2}$.

Solution.

Step 1: Start with `total = 0`.

Step 2: Add each number: `total = total + i` (short form `total += i`).

Step 3: Formula: $\frac{100 \times 101}{2} = 5050$. Both should agree.

>>> Python Program

Example 0.6.3

CODE

```
1 total = 0
2 for i in range(1, 101):
3     total += i          # same as total = total + i
4 print("Sum by loop      :", total)
5 n = 100
6 print("Sum by formula :", n * (n + 1) // 2)
```

OUTPUT

```
Sum by loop      : 5050
Sum by formula   : 5050
```

Example 0.6.4: A while loop

Rs. 10 000 is invested at 10% compound interest per year. After how many years will it first double?

Solution.

Step 1: We do not know how many years in advance, so use `while amount < 20000`.

Step 2: Each pass: multiply the amount by 1.10 and add 1 to the year counter.

Step 3: The loop stops as soon as the amount reaches 20 000.

>>> Python Program

Example 0.6.4

CODE

```
1 amount = 10000
2 years = 0
3 while amount < 20000:
4     amount = amount * 1.10
5     years += 1
```

```
6 print("Years needed:", years)
7 print(f"Amount then : Rs. {amount:,.2f}")
```

OUTPUT

```
Years needed: 8
Amount then : Rs. 21,435.89
```

Example 0.6.5: List comprehensions

Make a list of the squares of 1 to 10, and a list of the even numbers among them.

Solution.

Step 1: `[x**2 for x in range(1, 11)]` – “ x^2 for each x from 1 to 10”.

Step 2: Add a condition at the end to filter: `[s for s in squares if s % 2 == 0]`.

>>> Python Program**Example 0.6.5****CODE**

```
1 squares = [x**2 for x in range(1, 11)]
2 print("Squares:", squares)
3 even_squares = [s for s in squares if s % 2 == 0]
4 print("Even squares:", even_squares)
```

OUTPUT

```
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even squares: [4, 16, 36, 64, 100]
```

Example 0.6.6: Counting frequencies

Count how many times each value occurs in 2,3,1,4,2,3,2,5.

Solution.

Step 1: Start with an empty dictionary `freq = {}`.

Step 2: For each value, add 1 to its count: `freq[x] = freq.get(x, 0) + 1`.

▷ Why? `freq.get(x, 0)` gives the current count, or 0 if x has not been seen yet.

Step 3: This is exactly how a frequency table (Section 1.6) is built.

>>> Python Program**Example 0.6.6****CODE**

```
1 data = [2, 3, 1, 4, 2, 3, 2, 5]
2 freq = {}
3 for x in data:
4     freq[x] = freq.get(x, 0) + 1
5 for value in sorted(freq):
6     print(value, "occurs", freq[value], "times")
```

OUTPUT

```
1 occurs 1 times
2 occurs 3 times
3 occurs 2 times
4 occurs 1 times
5 occurs 1 times
```

Exercise 0.6

1. What numbers does `range(2, 10, 2)` give?
2. Write a loop that prints the cubes of 1 to 5.
3. Use a loop to find $1 + 3 + 5 + \dots + 19$ (sum of odd numbers up to 19).
4. Write a comprehension giving the numbers from 1 to 30 divisible by 3.

Multiple Choice Questions – 0.6

1. `range(4)` gives:
(a) 1,2,3,4 (b) 0,1,2,3
(c) 0,1,2,3,4 (d) 4
2. `len([x for x in range(10) if x > 6])` is:
(a) 3 (b) 4
(c) 6 (d) 10
3. A while loop runs:
(a) exactly 10 times (b) while its condition is true
(c) once (d) never
4. `total += 5` means:
(a) `total = 5` (b) `total = total + 5`
(c) `total == 5` (d) `total = total * 5`

Summary of 0.6

- `for` repeats over a list or a range; `while` repeats until a condition fails.
- Accumulator pattern: start at 0, add inside the loop.
- Comprehensions build lists in one line, optionally with an `if` filter.

0.7 Functions

Key Concepts

- A **function** is a named, reusable piece of code. Define it once, use it many times.
- `def name(parameters):` starts a function; `return` sends the answer back.
- Parameters can have **default values**: `def interest(P, R, T=1):`
- A function can return several values as a tuple: `return mean, low, high`
- A **lambda** is a one-line function: `f = lambda x: 3*x - 2` – ideal for $f(x)$ in Unit 2.

Example 0.7.1: Defining a function

Write a function that returns the area of a circle of radius r , and use it for $r = 7$ and $r = 3$.

Solution.

Step 1: `def circle_area(r):` names the function and its input.

Step 2: Inside, compute and return πr^2 .

Step 3: Call it with different radii.

>>> Python Program

Example 0.7.1

CODE

```
1 import math
2
3 def circle_area(r):
4     return math.pi * r**2
5
6 print(f"r = 7: area = {circle_area(7):.2f}")
7 print(f"r = 3: area = {circle_area(3):.2f}")
```

OUTPUT

```
r = 7: area = 153.94
r = 3: area = 28.27
```

Example 0.7.2: Default values

Write a function for simple interest $I = \frac{PRT}{100}$ where $T = 1$ year unless given.

Solution.

Step 1: $T=1$ in the definition makes 1 the default.

Step 2: `simple_interest(50000, 8)` uses $T = 1$; `simple_interest(50000, 8, 3)` uses $T = 3$.

>>> Python Program

Example 0.7.2

CODE

```
1 def simple_interest(P, R, T=1):
2     return P * R * T / 100
3
4 print(simple_interest(50000, 8))      # T = 1 (default)
5 print(simple_interest(50000, 8, 3))  # T = 3 years
```

OUTPUT

```
4000.0
12000.0
```

Example 0.7.3: Returning several values

Write a function that returns the mean, smallest and largest value of a list.

Solution.

Step 1: Compute each value, then return mean, low, high.

Step 2: Unpack the answer into three variables when calling.

>>> Python Program

Example 0.7.3

CODE

```
1 def summary(values):
2     mean = sum(values) / len(values)
3     return mean, min(values), max(values)
4
```

```

5 m, low, high = summary([3, 5, 7, 7, 8])
6 print("Mean:", m, " Smallest:", low, " Largest:", high)

```

OUTPUT

```
Mean: 6.0 Smallest: 3 Largest: 8
```

Example 0.7.4: Lambda functions

Define $f(x) = 3x - 2$ and find $f(0)$, $f(2)$ and $f(-1)$.

Solution.

Step 1: `f = lambda x: 3*x - 2` reads almost like mathematics.

Step 2: Call it like a function: `f(2)`.

>>> Python Program**Example 0.7.4****CODE**

```

1 f = lambda x: 3*x - 2
2 print("f(0) =", f(0))
3 print("f(2) =", f(2))
4 print("f(-1) =", f(-1))
5 print([f(x) for x in range(5)])    # f applied to 0, 1, 2, 3, 4

```

OUTPUT

```

f(0) = -2
f(2) = 4
f(-1) = -5
[-2, 1, 4, 7, 10]

```

Example 0.7.5: Functions working together

Write a function `percent_change(old, new)` and use it for $400 \rightarrow 500$ and $2500 \rightarrow 2000$.

Solution.

Step 1: Percentage change = $\frac{\text{new} - \text{old}}{\text{old}} \times 100$.

Step 2: A positive answer means an increase; a negative answer means a decrease.

>>> Python Program**Example 0.7.5****CODE**

```

1 def percent_change(old, new):
2     return (new - old) / old * 100
3
4 print(f"400 -> 500 : {percent_change(400, 500):.1f}%")
5 print(f"2500 -> 2000: {percent_change(2500, 2000):.1f}%")

```

OUTPUT

```

400 -> 500 : 25.0%
2500 -> 2000: -20.0%

```

Exercise 0.7

1. Write a function `rect_perimeter(l, w)` and find the perimeter of a 12×8 rectangle.
2. Write a lambda for $g(x) = x^2 + 1$ and find $g(3)$.
3. What does a function return if it has no `return` statement?
4. Write a function `is_even(n)` that returns `True` or `False`.

Multiple Choice Questions – 0.7

1. A function is defined with the keyword:

(a) <code>function</code>	(b) <code>def</code>
(c) <code>fun</code>	(d) <code>define</code>
2. The value a function sends back is given by:

(a) <code>print</code>	(b) <code>return</code>
(c) <code>send</code>	(d) <code>output</code>
3. If `f = lambda x: x * x`, then `f(4)` is:

(a) 8	(b) 16
(c) 4	(d) 44
4. In `def f(a, b=2)`, the value 2 is a:

(a) default value	(b) return value
(c) loop	(d) comment

Summary of 0.7

- `def name(inputs): ... return answer` – write once, use often.
- Default values make inputs optional; tuples return several answers.
- `lambda` writes short mathematical functions like $f(x) = 3x - 2$.

0.8 Python Libraries for Mathematics**Key Concepts**

Module	Useful tools
<code>math</code>	<code>sqrt</code> , <code>pi</code> , <code>gcd</code> , <code>lcm</code> , <code>factorial</code> , <code>comb</code> , <code>perm</code> , <code>ceil</code> , <code>floor</code>
<code>fractions</code>	<code>Fraction(3, 4)</code> – exact fractions, automatically simplified
<code>statistics</code>	<code>mean</code> , <code>median</code> , <code>mode</code> , <code>pstdev</code> , <code>stdev</code> , <code>pvariance</code> , <code>variance</code> , <code>quantiles</code>
<code>random</code>	<code>randint</code> , <code>choice</code> , <code>random</code> , <code>seed</code> – simulations
<code>itertools</code>	<code>permutations</code> , <code>combinations</code> , <code>product</code> – listing arrangements

These modules come with Python itself, so they work everywhere. Import them at the top of a cell, e.g. `import math` or `from fractions import Fraction`.

Example 0.8.1: The math module

Use math to find $\sqrt{50}$, $\text{HCF}(12, 18)$, $\text{LCM}(12, 18)$, $5!$, 5C_2 and 5P_2 .

Solution.

Step 1: `math.gcd` gives the HCF ("greatest common divisor") and `math.lcm` the LCM.

Step 2: `math.comb(n, r)` is nC_r and `math.perm(n, r)` is nP_r – used in Unit 3.

>>> Python Program**Example 0.8.1****CODE**

```
1 import math
2 print("sqrt(50) =", round(math.sqrt(50), 4))
3 print("HCF      =", math.gcd(12, 18))
4 print("LCM      =", math.lcm(12, 18))
5 print("5!       =", math.factorial(5))
6 print("5C2      =", math.comb(5, 2))
7 print("5P2      =", math.perm(5, 2))
```

OUTPUT

```
sqrt(50) = 7.0711
HCF      = 6
LCM      = 36
5!       = 120
5C2      = 10
5P2      = 20
```

Example 0.8.2: Exact fractions

Find $\frac{3}{4} + \frac{5}{6}$ exactly, and write 0.75 and 0.375 as fractions.

Solution.

Step 1: `Fraction(3, 4)` stores $\frac{3}{4}$ exactly (no decimal rounding).

Step 2: `Fraction("0.75")` converts a decimal written as text into a fraction.

▷ Why? Ordinary decimals can be slightly inaccurate in a computer; *Fraction* is always exact.

>>> Python Program**Example 0.8.2****CODE**

```
1 from fractions import Fraction
2 s = Fraction(3, 4) + Fraction(5, 6)
3 print("3/4 + 5/6 =", s)
4 print("as decimal:", float(s))
5 print("0.75  =", Fraction("0.75"))
6 print("0.375 =", Fraction("0.375"))
```

OUTPUT

```
3/4 + 5/6 = 19/12
as decimal: 1.5833333333333333
0.75  = 3/4
0.375 = 3/8
```

Example 0.8.3: The statistics module

For the data 3, 5, 7, 7, 8 find the mean, median, mode, population SD and sample SD.

Solution.

Step 1: `statistics.mean`, `statistics.median` and `statistics.mode` give the averages.

Step 2: `pstdev` divides by n (population); `stdev` divides by $n - 1$ (sample) – see Section 3.2.

>>> Python Program**Example 0.8.3****CODE**

```
1 import statistics as st
2 data = [3, 5, 7, 7, 8]
3 print("Mean      :", st.mean(data))
4 print("Median    :", st.median(data))
5 print("Mode      :", st.mode(data))
6 print("Pop. SD   :", round(st.pstdev(data), 4))
7 print("Sample SD :", st.stdev(data))
```

OUTPUT

```
Mean      : 6
Median    : 7
Mode      : 7
Pop. SD   : 1.7889
Sample SD : 2.0
```

Example 0.8.4: Random numbers and simulation

Simulate rolling a die 10 times.

Solution.

Step 1: `random.randint(1, 6)` gives a random whole number from 1 to 6.

Step 2: `random.seed(1)` fixes the starting point, so the “random” results can be repeated exactly.

▷ Why? Without a seed, you get different numbers each time you run the cell.

>>> Python Program**Example 0.8.4****CODE**

```
1 import random
2 random.seed(1)           # makes the results repeatable
3 rolls = [random.randint(1, 6) for _ in range(10)]
4 print("Rolls:", rolls)
5 print("Number of sixes:", rolls.count(6))
```

OUTPUT

```
Rolls: [2, 5, 1, 3, 1, 4, 4, 4, 6, 4]
Number of sixes: 1
```

Example 0.8.5: Listing arrangements

List all arrangements of 2 letters from A, B, C, and all selections of 2 letters.

Solution.

Step 1: `permutations("ABC", 2)`: order matters (AB and BA are different).

Step 2: `combinations("ABC", 2)`: order does not matter.

Step 3: Counts: $6 = {}^3P_2$ and $3 = {}^3C_2$.

>>> Python Program

Example 0.8.5

CODE

```
1 from itertools import permutations, combinations
2 arr = ["".join(p) for p in permutations("ABC", 2)]
3 sel = ["".join(c) for c in combinations("ABC", 2)]
4 print("Arrangements:", arr, "->", len(arr))
5 print("Selections  :", sel, "->", len(sel))
```

OUTPUT

```
Arrangements: ['AB', 'AC', 'BA', 'BC', 'CA', 'CB'] -> 6
Selections  : ['AB', 'AC', 'BC'] -> 3
```

Exercise 0.8

1. Use `math` to find $\sqrt{225}$, `HCF(24, 36)` and `7!`.
2. Use `Fraction` to find $\frac{2}{3} - \frac{1}{4}$.
3. Use `statistics` to find the median of 10, 3, 7, 12.
4. How many items does `list(combinations("ABCD", 2))` contain?

Multiple Choice Questions – 0.8

1. `math.gcd(12, 18)` is:

(a) 6	(b) 36
(c) 3	(d) 216
2. `Fraction(2, 4)` is shown as:

(a) 2/4	(b) 1/2
(c) 0.5	(d) 0.50
3. The sample standard deviation is given by:

(a) <code>pstdev</code>	(b) <code>stdev</code>
(c) <code>mean</code>	(d) <code>mode</code>
4. `math.comb(5, 2)` is:

(a) 10	(b) 20
(c) 25	(d) 120

Summary of 0.8

- `math`: roots, HCF/LCM, factorials, nC_r , nP_r . `fractions`: exact arithmetic.
- `statistics`: mean, median, mode, variance, SD, quartiles.
- `random`: simulations (use `seed` to repeat results). `itertools`: list arrangements.

0.9 Algebra with SymPy

Key Concepts

SymPy is a library for *symbolic* mathematics: it works with letters such as x exactly, like you do on paper.

Task	SymPy
create a symbol	<code>x = symbols("x")</code>
expand / factorise	<code>expand((x+3)*(x-5)), factor(x**2 - 25)</code>
simplify	<code>simplify((x**2 - 9)/(x + 3))</code>
solve an equation	<code>solve(Eq(3*x + 5, 20), x)</code>
solve a system	<code>solve([eq1, eq2], [x, y])</code>
substitute a value	<code>expr.subs(x, 2)</code>
exact numbers	<code>Rational(1, 3), sqrt(20) → 2*sqrt(5)</code> ; decimal value: <code>N(expr)</code>
inequalities	<code>solve_univariate_inequality(x**2 - 9 < 0, x)</code>

SymPy writes powers as `**`: `x**2` means x^2 , and `2*x` means $2x$ (the `*` cannot be left out).

Example 0.9.1: Expanding, factorising and simplifying

Expand $(x + 3)(x - 5)$, factorise $x^2 - 25$ and $2x^2 + 7x + 3$, and simplify $\frac{x^2 - 9}{x + 3}$.

Solution.

Step 1: Create the symbol x , then use `expand`, `factor` and `simplify`.

Step 2: Compare with the hand methods of Section 2.4 – the answers agree.

>>> Python Program

Example 0.9.1

CODE

```
1 from sympy import symbols, expand, factor, simplify
2 x = symbols("x")
3 print(expand((x + 3)*(x - 5)))
4 print(factor(x**2 - 25))
5 print(factor(2*x**2 + 7*x + 3))
6 print(simplify((x**2 - 9)/(x + 3)))
```

OUTPUT

```
x**2 - 2*x - 15
(x - 5)*(x + 5)
(x + 3)*(2*x + 1)
x - 3
```

Example 0.9.2: Solving equations

Solve $3x + 5 = 20$, $x^2 - 5x + 6 = 0$ and the system $x + y = 10$, $x - y = 2$.

Solution.

Step 1: Write each equation with `Eq(left, right)`.

Step 2: solve returns a list of solutions, or a dictionary for a system.

>>> Python Program

Example 0.9.2

CODE

```
1 from sympy import symbols, Eq, solve
2 x, y = symbols("x y")
3 print(solve(Eq(3*x + 5, 20), x))
4 print(solve(Eq(x**2 - 5*x + 6, 0), x))
5 print(solve([Eq(x + y, 10), Eq(x - y, 2)], [x, y]))
```

OUTPUT

```
[5]
[2, 3]
{x: 6, y: 4}
```

Example 0.9.3: Exact values and substitution

For $x^2 + 2x - 4 = 0$ find the exact roots and their decimal values; also evaluate $x^2 - 3x + 2$ at $x = -2$.

Solution.

Step 1: SymPy keeps square roots exact: $-1 \pm \sqrt{5}$.

Step 2: `N()` (or `float()`) gives the decimal value.

Step 3: `.subs(x, -2)` puts -2 in place of x .

>>> Python Program

Example 0.9.3

CODE

```
1 from sympy import symbols, solve, N
2 x = symbols("x")
3 roots = solve(x**2 + 2*x - 4, x)
4 print("Exact :", roots)
5 print("Decimal:", [round(float(N(r)), 4) for r in roots])
6 expr = x**2 - 3*x + 2
7 print("Value at x = -2:", expr.subs(x, -2))
```

OUTPUT

```
Exact : [-1 + sqrt(5), -sqrt(5) - 1]
Decimal: [1.2361, -3.2361]
Value at x = -2: 12
```

Example 0.9.4: Solving inequalities

Solve $2x - 3 < 7$ and $x^2 - 9 < 0$.

Solution.

Step 1: `solve_univariate_inequality` returns the answer as a condition on x .

Step 2: `&` in the answer means “and”: $-3 < x$ and $x < 3$, i.e. $-3 < x < 3$.

>>> Python Program

Example 0.9.4

CODE

```
1 from sympy import symbols, solve_univariate_inequality
2 x = symbols("x", real=True)
3 print(solve_univariate_inequality(2*x - 3 < 7, x))
4 print(solve_univariate_inequality(x**2 - 9 < 0, x))
```

OUTPUT

```
x < 5
(-3 < x) & (x < 3)
```

Exercise 0.9

1. Use SymPy to factorise $x^2 + 7x + 12$.
2. Solve $5x - 7 = 18$ with SymPy.
3. Solve $x^2 - 4x + 1 = 0$ exactly.
4. Expand $(2x - 1)^2$.

Multiple Choice Questions – 0.9

1. In SymPy, x^2 is written:

(a) x^2	(b) $x**2$
(c) $x*2$	(d) $x2$
2. `factor(x**2 - 25)` gives:

(a) $(x - 5)**2$	(b) $(x - 5)*(x + 5)$
(c) $x - 25$	(d) $x**2 - 25$
3. To substitute $x = 2$ in `expr` we use:

(a) <code>expr(2)</code>	(b) <code>expr.subs(x, 2)</code>
(c) <code>expr = 2</code>	(d) <code>solve(expr, 2)</code>
4. `solve(Eq(2*x, 8), x)` returns:

(a) <code>[4]</code>	(b) <code>4.0</code>
(c) <code>[16]</code>	(d) <code>x = 4</code>

Summary of 0.9

- `symbols` creates letters; `expand`, `factor`, `simplify` work like hand algebra.
- `solve(Eq(...), x)` solves equations and systems; answers are exact.
- `N()` gives decimals; `.subs()` evaluates at a value.

0.10 Matplotlib, NumPy and SciPy

Key Concepts

- **NumPy** (`import numpy as np`) works on whole arrays of numbers at once: `np.array([1, 2, 3]) * 10` gives `[10 20 30]`. `np.linspace(-3, 3, 100)` makes 100 evenly spaced x -values for smooth graphs.

- **Matplotlib** (`import matplotlib.pyplot as plt`) draws charts:

<code>plt.plot(x, y)</code>	line graph	<code>plt.bar(names, values)</code>	bar chart
<code>plt.pie(values, labels=...)</code>	pie chart	<code>plt.hist(data, bins=...)</code>	histogram
<code>plt.scatter(x, y)</code>	points	<code>plt.title, plt.xlabel, plt.grid</code>	labels, grid

- End with `plt.show()`; in Colab the chart appears under the cell (right-click to save it).
- **SciPy** (`from scipy import stats`) has ready-made probability distributions: `stats.binom.pmf(r, n, p)` and `stats.norm.cdf(x, mu, sigma)` (Section 3.5).

Example 0.10.1: NumPy arrays

Convert the lengths 2, 5, 7.5 km to metres in one step and find their mean.

Solution.

Step 1: Put the values in `np.array`.

Step 2: Multiplying the array by 1000 multiplies *every* value – no loop needed.

>>> Python Program

Example 0.10.1

CODE

```
1 import numpy as np
2 km = np.array([2, 5, 7.5])
3 metres = km * 1000          # every value is multiplied
4 print("Metres:", metres)
5 print("Mean  :", metres.mean())
```

OUTPUT

```
Metres: [2000. 5000. 7500.]
Mean   : 4833.333333333333
```

Example 0.10.2: A line graph

Draw the graph of $y = 2x + 1$ for $-3 \leq x \leq 3$.

Solution.

Step 1: Make many x -values with `np.linspace`; compute y for all of them at once.

Step 2: `plt.plot(x, y)`, add a title, axis labels and a grid, then `plt.show()`.

>>> Python Program

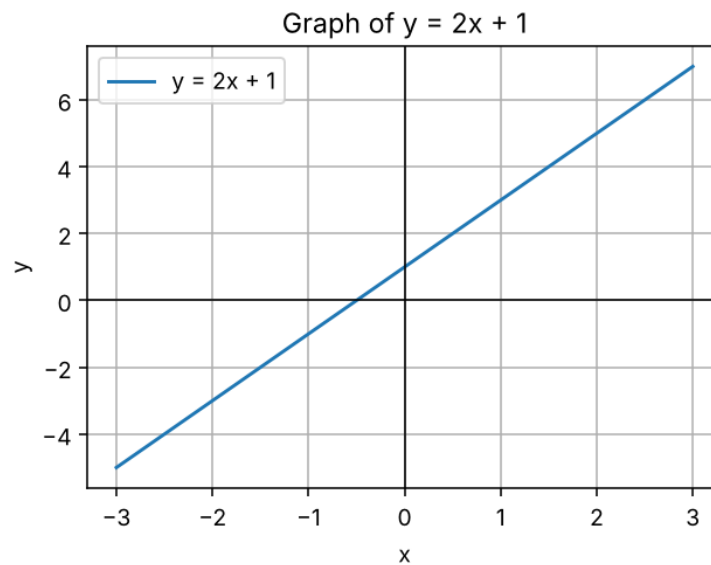
Example 0.10.2

CODE

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x = np.linspace(-3, 3, 100)
4 y = 2*x + 1
5 plt.figure(figsize=(5, 3.5))
6 plt.plot(x, y, label="y = 2x + 1")
7 plt.axhline(0, color="black", linewidth=0.8)
8 plt.axvline(0, color="black", linewidth=0.8)
9 plt.title("Graph of y = 2x + 1")
10 plt.xlabel("x"); plt.ylabel("y")
11 plt.grid(True); plt.legend()
```

```
12 plt.show()
```

OUTPUT (FIGURE)



Example 0.10.3: Bar chart and pie chart

Draw a bar chart and a pie chart of favourite sports: Cricket 50, Football 30, Hockey 20, Others 20.

Solution.

Step 1: Store the categories and counts in two lists.

Step 2: `plt.subplots(1, 2)` places two charts side by side; `autopct` prints percentages on the pie.

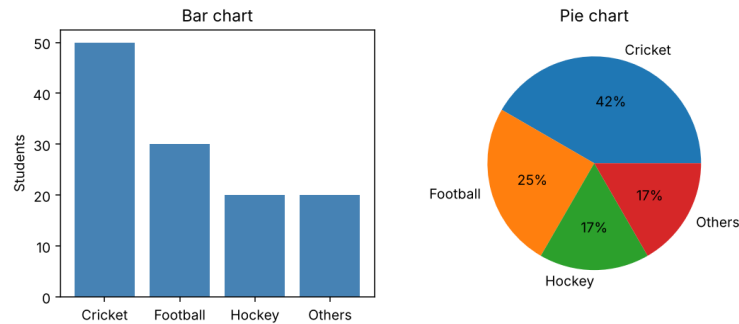
>>> Python Program

Example 0.10.3

CODE

```
1 import matplotlib.pyplot as plt
2 sports = ["Cricket", "Football", "Hockey", "Others"]
3 students = [50, 30, 20, 20]
4 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.5))
5 ax1.bar(sports, students, color="steelblue")
6 ax1.set_title("Bar chart"); ax1.set_ylabel("Students")
7 ax2.pie(students, labels=sports, autopct="%1.0f%%")
8 ax2.set_title("Pie chart")
9 plt.show()
```

OUTPUT (FIGURE)



Example 0.10.4: Histogram

Draw a histogram of the 20 marks of Section 1.6 with classes 10–19, 20–29, 30–39, 40–49.

Solution.

Step 1: Give the class boundaries as `bins=[9.5, 19.5, 29.5, 39.5, 49.5]`.

Step 2: `edgecolor` draws the bar outlines so the bars are easy to see.

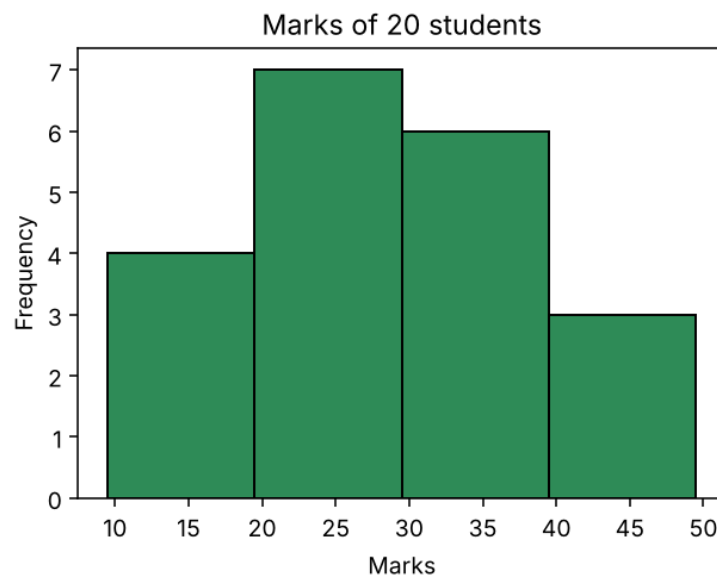
>>> Python Program

Example 0.10.4

CODE

```
1 import matplotlib.pyplot as plt
2 marks = [12, 25, 37, 18, 42, 29, 33, 15, 27, 39,
3          21, 45, 31, 24, 36, 19, 28, 41, 34, 26]
4 plt.figure(figsize=(5, 3.5))
5 plt.hist(marks, bins=[9.5, 19.5, 29.5, 39.5, 49.5],
6          color="seagreen", edgecolor="black")
7 plt.title("Marks of 20 students")
8 plt.xlabel("Marks"); plt.ylabel("Frequency")
9 plt.show()
```

OUTPUT (FIGURE)



Example 0.10.5: Probability with SciPy

Find the probability of exactly 2 heads in 4 tosses of a coin, and the probability that a normal value with $\mu = 60$, $\sigma = 8$ is below 76.

Solution.

Step 1: `stats.binom.pmf(2, 4, 0.5)` gives $P(X = 2)$ for $n = 4$, $p = 0.5$.

Step 2: `stats.norm.cdf(76, 60, 8)` gives $P(X < 76)$ – the area under the normal curve to the left of 76.

>>> Python Program**Example 0.10.5****CODE**

```
1 from scipy import stats
2 p2 = stats.binom.pmf(2, 4, 0.5)      # P(X = 2), n = 4, p = 0.5
3 below = stats.norm.cdf(76, 60, 8)   # P(X < 76), mu = 60, sigma = 8
4 print(f"P(exactly 2 heads in 4 tosses) = {p2:.4f}")
5 print(f"P(X < 76) = {below:.4f}")
```

OUTPUT

```
P(exactly 2 heads in 4 tosses) = 0.3750
P(X < 76) = 0.9772
```

Exercise 0.10

1. Use NumPy to find the squares of `[1, 2, 3, 4]` in one step.
2. Draw the graph of $y = x^2$ for $-3 \leq x \leq 3$.
3. Draw a bar chart of a week's temperatures (choose your own values).
4. Use SciPy to find $P(X = 3)$ for a binomial distribution with $n = 3$, $p = 0.5$.

Multiple Choice Questions – 0.10

1. The command that displays a Matplotlib chart is:

(a) <code>plt.draw_now()</code>	(b) <code>plt.show()</code>
(c) <code>print(plt)</code>	(d) <code>plt.run()</code>
2. A pie chart is drawn with:

(a) <code>plt.bar</code>	(b) <code>plt.pie</code>
(c) <code>plt.hist</code>	(d) <code>plt.plot</code>
3. `np.array([1, 2]) * 3` gives:

(a) <code>[1 2 1 2 1 2]</code>	(b) <code>[3 6]</code>
(c) <code>[4 5]</code>	(d) error
4. `stats.binom.pmf(r, n, p)` gives:

(a) $P(X = r)$	(b) $P(X \leq r)$
(c) the mean	(d) the SD

Summary of 0.10

- NumPy: fast calculations on whole arrays; `linspace` for smooth graphs.
- Matplotlib: `plot`, `bar`, `pie`, `hist`; always label and `show()`.
- SciPy: `binom.pmf` and `norm.cdf` for probabilities.

- You now know all the Python needed for this book – in every unit, solve by hand first, then check with Python.

UNIT 1

Numerical Literacy

1.1 Number System and Basic Arithmetic Operations

Key Concepts

Types of numbers

- **Natural numbers** $\mathbb{N} = \{1, 2, 3, \dots\}$ – counting numbers.
- **Whole numbers** $\mathbb{W} = \{0, 1, 2, 3, \dots\}$ – natural numbers together with 0.
- **Integers** $\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$ – whole numbers and their negatives.
- **Rational numbers** $\mathbb{Q}$ – numbers that can be written as $\frac{p}{q}$ where p, q are integers and $q \neq 0$, e.g. $\frac{3}{4}$, -5 , 0.25 , $0.\bar{3}$. Their decimals either *terminate* or *repeat*.
- **Irrational numbers** – numbers that *cannot* be written as $\frac{p}{q}$; their decimals never end and never repeat, e.g. $\sqrt{2}$, $\sqrt{5}$, π .
- **Real numbers** $\mathbb{R}$ – all rational and irrational numbers together.

$$\mathbb{N} \subset \mathbb{W} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$$

Order of operations (BODMAS / PEMDAS): Brackets $\rightarrow$ Orders (powers, roots) $\rightarrow$ Division and Multiplication (left to right) $\rightarrow$ Addition and Subtraction (left to right).

HCF (Highest Common Factor) = largest number dividing both numbers. **LCM** (Least Common Multiple) = smallest number that both numbers divide. For two numbers: $\text{HCF} \times \text{LCM} = a \times b$.

Scientific notation: a number written as $a \times 10^n$ where $1 \leq a < 10$ and n is an integer.

Example 1.1.1: Classifying numbers

Classify each number: -3 , 0 , 7 , $\frac{2}{5}$, $\sqrt{2}$, π , $0.333\dots$

Solution.

Step 1: Ask first: *is it a counting number?* Only 7 is, so only $7 \in \mathbb{N}$.

▷ *Why? Natural numbers are $1, 2, 3, \dots$ – the numbers we use to count objects.*

Step 2: Ask: *is it a whole number?* 0 and 7 are whole numbers.

▷ *Why? Whole numbers are the natural numbers together with 0.*

Step 3: Ask: *is it an integer?* -3 , 0 , 7 are integers.

▷ *Why? Integers include the negatives of whole numbers, but no fractions.*

Step 4: Ask: *can it be written as $\frac{p}{q}$?* $-3 = \frac{-3}{1}$, $0 = \frac{0}{1}$, $7 = \frac{7}{1}$, $\frac{2}{5}$ and $0.333\dots = \frac{1}{3}$ can, so they are rational.

▷ *Why? A decimal that repeats (or ends) can always be turned into a fraction.*

Step 5: $\sqrt{2} = 1.41421\dots$ and $\pi = 3.14159\dots$ never end and never repeat, so they are irrational.

Step 6: Every number above is real.

Number	N	W	Z	Q	Irrational	R
-3			✓	✓		✓
0		✓	✓	✓		✓
7	✓	✓	✓	✓		✓
$\frac{2}{5}$				✓		✓
$\sqrt{2}$					✓	✓
π					✓	✓
0.333...				✓		✓

>>> Python Solution

Example 1.1.1

CODE

```

1 from fractions import Fraction
2
3 numbers = {"-3": -3, "0": 0, "7": 7, "2/5": Fraction(2, 5),
4           "sqrt(2)": 2 ** 0.5, "pi": 3.141592653589793, "0.333...": Fraction(1,
5           ↪ 3)}
6
7 irrational = {"sqrt(2)", "pi"}          # never end, never repeat
8
9 for name, x in numbers.items():
10     kinds = []
11     if name in irrational:
12         kinds.append("irrational")
13     else:
14         if x == int(x) and x >= 1: kinds.append("natural")
15         if x == int(x) and x >= 0: kinds.append("whole")
16         if x == int(x): kinds.append("integer")
17         kinds.append("rational")
18     kinds.append("real")
19     print(f"{name:>9}: {'', '.join(kinds)}")

```

OUTPUT

```

-3: integer, rational, real
0: whole, integer, rational, real
7: natural, whole, integer, rational, real
2/5: rational, real
sqrt(2): irrational, real
pi: irrational, real
0.333...: rational, real

```

Example 1.1.2: Operations with integers

Find (a) $(-7) + 12$ (b) $(-4) \times (-6)$ (c) $(-20) \div 5$ (d) The temperature in Quetta is -3°C at night and rises by 8°C by noon. Find the noon temperature.

Solution.

Step 1: (a) Signs are different, so subtract the sizes: $12 - 7 = 5$, and keep the sign of the larger size (12 is positive): $(-7) + 12 = 5$.

Step 2: (b) Negative $\times$ negative = positive: $(-4) \times (-6) = +24$.

▷ Why? Same signs give a positive product or quotient; different signs give a negative one.

Step 3: (c) Negative $\div$ positive = negative: $(-20) \div 5 = -4$.

Step 4: (d) A rise means addition: $-3 + 8 = 5$. The noon temperature is 5°C .

>>> Python Solution

Example 1.1.2

CODE

```
1 print("(a) (-7) + 12 =", (-7) + 12)
2 print("(b) (-4) * (-6) =", (-4) * (-6))
3 print("(c) (-20) / 5 =", (-20) // 5)
4 night = -3
5 noon = night + 8           # a rise means addition
6 print("(d) Noon temperature =", noon, "degrees C")
```

OUTPUT

```
(a) (-7) + 12 = 5
(b) (-4) * (-6) = 24
(c) (-20) / 5 = -4
(d) Noon temperature = 5 degrees C
```

Example 1.1.3: Order of operations

Simplify $18 \div 3 \times 2 + (5 - 2)^2$.

Solution.

Step 1: Brackets first: $(5 - 2) = 3$, so the expression becomes $18 \div 3 \times 2 + 3^2$.

▷ Why? B in BODMAS – brackets always come first.

Step 2: Orders (powers): $3^2 = 9$, giving $18 \div 3 \times 2 + 9$.

Step 3: Division and multiplication have equal rank, so work from **left to right**: $18 \div 3 = 6$, then $6 \times 2 = 12$.

▷ Why? Doing 3×2 first would give $18 \div 6 = 3$ – a wrong answer.

Step 4: Finally add: $12 + 9 = 21$.

Answer: 21

>>> Python Solution

Example 1.1.3

CODE

```
1 step1 = 5 - 2           # brackets
2 step2 = step1 ** 2      # orders (powers)
3 step3 = 18 / 3 * 2      # division then multiplication, left to right
4 answer = step3 + step2  # addition
5 print("Brackets:", step1, "Power:", step2, "Div/Mult:", step3)
6 print("Answer:", answer)
7 print("Check in one line:", 18 / 3 * 2 + (5 - 2) ** 2)
```

OUTPUT

```
Brackets: 3 Power: 9 Div/Mult: 12.0
Answer: 21.0
Check in one line: 21.0
```

Example 1.1.4: Nested brackets

Simplify $50 - [12 + (8 - 3) \times 2] \div 2$.

Solution.

Step 1: Start with the innermost bracket: $(8 - 3) = 5$.

Step 2: Inside the square bracket, multiply before adding: $5 \times 2 = 10$, then $12 + 10 = 22$.

▷ *Why? BODMAS also applies inside a bracket.*

Step 3: The expression is now $50 - 22 \div 2$. Divide before subtracting: $22 \div 2 = 11$.

Step 4: Subtract: $50 - 11 = 39$.

Answer: 39

>>> Python Solution**Example 1.1.4****CODE**

```
1 inner = 8 - 3                # innermost bracket
2 square = 12 + inner * 2      # inside [ ]: multiply, then add
3 answer = 50 - square / 2     # divide before subtracting
4 print("Inner bracket:", inner)
5 print("Square bracket:", square)
6 print("Answer:", answer)
```

OUTPUT

```
Inner bracket: 5
Square bracket: 22
Answer: 39.0
```

Example 1.1.5: Adding fractions

Find $\frac{3}{4} + \frac{5}{6}$.

Solution.

Step 1: The denominators are different (4 and 6), so we need a common denominator. LCM of 4 and 6 is 12.

▷ *Why? We can only add "parts" when the parts are the same size (twelfths).*

Step 2: Convert: $\frac{3}{4} = \frac{3 \times 3}{4 \times 3} = \frac{9}{12}$ and $\frac{5}{6} = \frac{5 \times 2}{6 \times 2} = \frac{10}{12}$.

Step 3: Add the numerators, keep the denominator: $\frac{9 + 10}{12} = \frac{19}{12}$.

Step 4: As a mixed number: $19 \div 12 = 1$ remainder 7, so $\frac{19}{12} = 1\frac{7}{12}$.

>>> Python Solution**Example 1.1.5****CODE**

```
1 from fractions import Fraction
2 import math
3
4 lcm = math.lcm(4, 6)
5 print("LCM of denominators:", lcm)
```

```

6 total = Fraction(3, 4) + Fraction(5, 6)
7 print("3/4 + 5/6 =", total)
8 whole, rem = divmod(total.numerator, total.denominator)
9 print(f"As a mixed number: {whole} {rem}/{total.denominator}")

```

OUTPUT

```

LCM of denominators: 12
3/4 + 5/6 = 19/12
As a mixed number: 1 7/12

```

Example 1.1.6: Multiplying and dividing fractions

Find (a) $\frac{2}{3} \times \frac{9}{10}$ (b) $\frac{3}{4} \div \frac{9}{8}$.

Solution.

Step 1: (a) Multiply numerators and denominators: $\frac{2 \times 9}{3 \times 10} = \frac{18}{30}$.

Step 2: Simplify by dividing top and bottom by HCF 6: $\frac{18}{30} = \frac{3}{5}$.

Step 3: (b) To divide, multiply by the **reciprocal** (flip the second fraction): $\frac{3}{4} \times \frac{8}{9}$.

▷ Why? Dividing by $\frac{9}{8}$ asks "how many $\frac{9}{8}$ fit in $\frac{3}{4}$?" – the same as multiplying by $\frac{8}{9}$.

Step 4: $\frac{3 \times 8}{4 \times 9} = \frac{24}{36} = \frac{2}{3}$.

>>> Python Solution**Example 1.1.6****CODE**

```

1 from fractions import Fraction
2 product = Fraction(2, 3) * Fraction(9, 10)
3 print("(a) 2/3 x 9/10 =", product)
4 reciprocal = 1 / Fraction(9, 8)
5 print("    reciprocal of 9/8 =", reciprocal)
6 print("(b) 3/4 / 9/8 =", Fraction(3, 4) / Fraction(9, 8))

```

OUTPUT

```

(a) 2/3 x 9/10 = 3/5
    reciprocal of 9/8 = 8/9
(b) 3/4 / 9/8 = 2/3

```

Example 1.1.7: Mixed numbers

Find $2\frac{1}{2} + 1\frac{2}{3}$.

Solution.

Step 1: Change to improper fractions: $2\frac{1}{2} = \frac{2 \times 2 + 1}{2} = \frac{5}{2}$ and $1\frac{2}{3} = \frac{1 \times 3 + 2}{3} = \frac{5}{3}$.

Step 2: LCM of 2 and 3 is 6: $\frac{5}{2} = \frac{15}{6}$, $\frac{5}{3} = \frac{10}{6}$.

Step 3: Add: $\frac{15 + 10}{6} = \frac{25}{6} = 4\frac{1}{6}$.

>>> Python Solution

Example 1.1.7

CODE

```

1 from fractions import Fraction
2 a = 2 + Fraction(1, 2)          # 2 1/2 = 5/2
3 b = 1 + Fraction(2, 3)          # 1 2/3 = 5/3
4 total = a + b
5 print("Improper fractions:", a, "and", b)
6 print("Sum:", total)
7 whole, rem = divmod(total.numerator, total.denominator)
8 print(f"Mixed number: {whole} {rem}/{total.denominator}")

```

OUTPUT

```

Improper fractions: 5/2 and 5/3
Sum: 25/6
Mixed number: 4 1/6

```

Example 1.1.8: HCF and LCM

Find the HCF and LCM of 12 and 18.

Solution.

Step 1: Write each number as a product of primes: $12 = 2 \times 2 \times 3 = 2^2 \times 3$, $18 = 2 \times 3 \times 3 = 2 \times 3^2$.

Step 2: HCF = product of the *lowest* powers of the *common* primes $= 2^1 \times 3^1 = 6$.

▷ Why? The HCF can only contain what both numbers share.

Step 3: LCM = product of the *highest* powers of *all* primes $= 2^2 \times 3^2 = 36$.

▷ Why? The LCM must contain everything needed to build each number.

Step 4: Check: $\text{HCF} \times \text{LCM} = 6 \times 36 = 216 = 12 \times 18$. ✓

>>> Python Solution

Example 1.1.8

CODE

```

1 import math
2
3 def prime_factors(n):
4     factors, d = [], 2
5     while n > 1:
6         while n % d == 0:
7             factors.append(d)
8             n //= d
9         d += 1
10    return factors
11
12 print("12 =", prime_factors(12))
13 print("18 =", prime_factors(18))
14 hcf, lcm = math.gcd(12, 18), math.lcm(12, 18)
15 print("HCF =", hcf, " LCM =", lcm)
16 print("Check HCF x LCM = 12 x 18:", hcf * lcm == 12 * 18)

```

OUTPUT

```

12 = [2, 2, 3]
18 = [2, 3, 3]

```

```
HCF = 6   LCM = 36
Check HCF x LCM = 12 x 18: True
```

Example 1.1.9: LCM in daily life

Two bells ring every 12 minutes and every 18 minutes. They ring together at 9:00 a.m. When will they next ring together?

Solution.

Step 1: Bell 1 rings at 12, 24, 36, 48, ... minutes; bell 2 at 18, 36, 54, ... minutes.

Step 2: The first common time is the LCM of 12 and 18, which is 36 (previous example).

▷ Why? "Next time together" means the smallest common multiple.

Step 3: 9:00 + 36 minutes = 9:36 a.m.

>>> Python Solution

Example 1.1.9

CODE

```
1 import math
2 bell1 = [12 * k for k in range(1, 6)]
3 bell2 = [18 * k for k in range(1, 6)]
4 print("Bell 1 rings at:", bell1)
5 print("Bell 2 rings at:", bell2)
6 minutes = math.lcm(12, 18)
7 print(f"Together again after {minutes} minutes, at 9:{minutes:02d} a.m.")
```

OUTPUT

```
Bell 1 rings at: [12, 24, 36, 48, 60]
Bell 2 rings at: [18, 36, 54, 72, 90]
Together again after 36 minutes, at 9:36 a.m.
```

Example 1.1.10: HCF in daily life

A floor is 240 cm by 180 cm. Find the largest square tile that fits exactly, and the number of tiles needed.

Solution.

Step 1: The side of the tile must divide both 240 and 180 exactly, and be as large as possible: we need the HCF.

Step 2: $240 = 2^4 \times 3 \times 5$, $180 = 2^2 \times 3^2 \times 5$. $\text{HCF} = 2^2 \times 3 \times 5 = 60$ cm.

Step 3: Tiles along the length: $240 \div 60 = 4$; along the width: $180 \div 60 = 3$.

Step 4: Number of tiles = $4 \times 3 = 12$.

>>> Python Solution

Example 1.1.10

CODE

```
1 import math
2 side = math.gcd(240, 180)
3 along_length = 240 // side
4 along_width = 180 // side
5 print("Largest tile side:", side, "cm")
6 print("Tiles needed:", along_length, "x", along_width, "=", along_length * along_width)
```

```
↪ along_width)
```

OUTPUT

```
Largest tile side: 60 cm
Tiles needed: 4 x 3 = 12
```

Example 1.1.11: Repeating decimals to fractions

Write (a) $0.\overline{3} = 0.333\dots$ (b) $0.\overline{45} = 0.4545\dots$ as fractions.

Solution.

Step 1: (a) Let $x = 0.333\dots$. One digit repeats, so multiply by 10: $10x = 3.333\dots$

Step 2: Subtract to remove the endless tail: $10x - x = 3.333\dots - 0.333\dots \Rightarrow 9x = 3 \Rightarrow x = \frac{3}{9} = \frac{1}{3}$.

▷ *Why? The repeating parts are identical, so they cancel exactly.*

Step 3: (b) Let $x = 0.4545\dots$. Two digits repeat, so multiply by 100: $100x = 45.4545\dots$

Step 4: Subtract: $99x = 45 \Rightarrow x = \frac{45}{99} = \frac{5}{11}$ (dividing by 9).

>>> Python Solution**Example 1.1.11****CODE**

```
1 from fractions import Fraction
2 # x = 0.333...: 10x - x = 3 -> 9x = 3
3 print("0.333... =", Fraction(3, 9))
4 # x = 0.4545...: 100x - x = 45 -> 99x = 45
5 print("0.4545... =", Fraction(45, 99))
6 print("Check:", float(Fraction(5, 11)))
```

OUTPUT

```
0.333... = 1/3
0.4545... = 5/11
Check: 0.45454545454545453
```

Example 1.1.12: Terminating or repeating?

Without dividing fully, decide whether $\frac{7}{8}$ and $\frac{2}{3}$ give terminating or repeating decimals.

Solution.

Step 1: A fraction in lowest terms terminates only if its denominator has *no prime factors other than 2 and 5*.

▷ *Why? Our number system is based on $10 = 2 \times 5$.*

Step 2: $8 = 2^3$: only 2's, so $\frac{7}{8}$ terminates: $\frac{7}{8} = 0.875$.

Step 3: 3 is not 2 or 5, so $\frac{2}{3}$ repeats: $\frac{2}{3} = 0.666\dots = 0.\overline{6}$.

>>> Python Solution**Example 1.1.12****CODE**

```
1 from fractions import Fraction
```

```

2
3 def terminates(fr):
4     d = fr.denominator          # Fraction is already in lowest terms
5     for p in (2, 5):
6         while d % p == 0:
7             d //= p
8     return d == 1
9
10 for fr in [Fraction(7, 8), Fraction(2, 3)]:
11     kind = "terminating" if terminates(fr) else "repeating"
12     print(f"{fr} = {float(fr)} ... {kind}")

```

OUTPUT

```

7/8 = 0.875 ... terminating
2/3 = 0.6666666666666666 ... repeating

```

Example 1.1.13: Scientific notation

Write (a) 45 000 (b) 0.00072 in scientific notation; then find (c) $(3 \times 10^4) \times (2 \times 10^3)$ (d) $8 \times 10^6 \div 4 \times 10^2$.

Solution.

Step 1: (a) Place the decimal point after the first non-zero digit: 4.5. It moved 4 places left, so $45\,000 = 4.5 \times 10^4$.

Step 2: (b) 7.2: the point moved 4 places right, so $0.00072 = 7.2 \times 10^{-4}$.

▷ Why? Moving left $\Rightarrow$ positive power (big number); moving right $\Rightarrow$ negative power (small number).

Step 3: (c) Multiply the numbers and add the powers: $(3 \times 2) \times 10^{4+3} = 6 \times 10^7$.

Step 4: (d) Divide the numbers and subtract the powers: $(8 \div 4) \times 10^{6-2} = 2 \times 10^4$.

>>> Python Solution**Example 1.1.13****CODE**

```

1 print(f"(a) 45000 = {45000:.1e}")
2 print(f"(b) 0.00072 = {0.00072:.1e}")
3 c = (3e4) * (2e3)
4 d = (8e6) / (4e2)
5 print(f"(c) {c:.0e}")
6 print(f"(d) {d:.0e}")

```

OUTPUT

```

(a) 45000 = 4.5e+04
(b) 0.00072 = 7.2e-04
(c) 6e+07
(d) 2e+04

```

Exercise 1.1 (solve by hand, then check with Python)

1. Classify the numbers -5 , $\frac{3}{7}$, $\sqrt{9}$, $\sqrt{5}$, 0 as natural, integer, rational or irrational.
2. Simplify: $20 - 4 \times 3 + 8 \div 2$.

3. Find $\frac{2}{3} - \frac{1}{4}$.
4. Find the HCF and LCM of 24 and 36.
5. Write $0.\overline{6}$ as a fraction in lowest terms.
6. Write 3 200 000 in scientific notation.

Multiple Choice Questions – 1.1

1. Which of the following is irrational?

(a) 0.5	(b) $\sqrt{16}$
(c) $\sqrt{7}$	(d) $\frac{22}{7}$
2. The value of $8 + 2 \times 5$ is:

(a) 50	(b) 18
(c) 15	(d) 20
3. The LCM of 4 and 6 is:

(a) 2	(b) 12
(c) 24	(d) 10
4. 0.00056 in scientific notation is:

(a) 5.6×10^{-4}	(b) 5.6×10^4
(c) 56×10^{-5}	(d) 0.56×10^{-3}
5. Zero belongs to which set but NOT to $\mathbb{N}$?

(a) $\mathbb{W}$	(b) irrationals
(c) none	(d) positive integers

Summary of 1.1

- $\mathbb{N} \subset \mathbb{W} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$; irrationals have non-ending, non-repeating decimals.
- Always follow BODMAS; do $\times$ and $\div$ (and $+$ and $-$) from left to right.
- To add/subtract fractions, first make the denominators equal using the LCM.
- $\text{HCF} \times \text{LCM} = \text{product of the two numbers}$.
- Scientific notation $a \times 10^n$ with $1 \leq a < 10$ makes very large/small numbers easy to read.

1.2 Units, Conversions, Dimensions, Area, Perimeter and Volume

Key Concepts

Units tell us *what* is being measured. Common metric conversions:

Length	1 km = 1000 m, 1 m = 100 cm, 1 cm = 10 mm, 1 inch = 2.54 cm
Mass	1 kg = 1000 g, 1 tonne = 1000 kg
Capacity	1 L = 1000 mL, 1 mL = 1 cm ³ , 1 m ³ = 1000 L
Time	1 h = 60 min = 3600 s
Area	1 m ² = 100 × 100 = 10 000 cm ²
Volume	1 m ³ = 100 × 100 × 100 = 1 000 000 cm ³

Dimensions: length is 1-dimensional (unit: m), area is 2-dimensional (unit: m²), volume is

3-dimensional (unit: m^3).

Perimeter = distance around a flat shape. **Area** = space inside a flat shape. **Volume** = space inside a solid.

Important Formulas

Shape	Perimeter / Surface	Area / Volume
Rectangle ($l \times w$)	$P = 2(l + w)$	$A = lw$
Square (side a)	$P = 4a$	$A = a^2$
Triangle (base b , height h)	$P = \text{sum of sides}$	$A = \frac{1}{2}bh$
Circle (radius r)	$C = 2\pi r$	$A = \pi r^2$
Cuboid	–	$V = lwh$
Cube (edge a)	–	$V = a^3$
Cylinder	–	$V = \pi r^2 h$
Sphere	–	$V = \frac{4}{3}\pi r^3$

Example 1.2.1: Simple unit conversions

Convert (a) 5 km to metres (b) 2.5 kg to grams (c) 4500 g to kg (d) 750 mL to litres (e) 2 h 15 min to minutes.

Solution.

Step 1: Big unit → small unit: multiply. (a) $5 \times 1000 = 5000$ m. (b) $2.5 \times 1000 = 2500$ g.

▷ Why? A kilometre is bigger than a metre, so we need more metres to describe the same length.

Step 2: Small unit → big unit: divide. (c) $4500 \div 1000 = 4.5$ kg. (d) $750 \div 1000 = 0.75$ L.

Step 3: (e) $2 \text{ h} = 2 \times 60 = 120$ min; total = $120 + 15 = 135$ min.

>>> Python Solution

Example 1.2.1

CODE

```
1 print("(a) 5 km =", 5 * 1000, "m")          # big -> small: multiply
2 print("(b) 2.5 kg =", 2.5 * 1000, "g")
3 print("(c) 4500 g =", 4500 / 1000, "kg")      # small -> big: divide
4 print("(d) 750 mL =", 750 / 1000, "L")
5 print("(e) 2 h 15 min =", 2 * 60 + 15, "min")
```

OUTPUT

```
(a) 5 km = 5000 m
(b) 2.5 kg = 2500.0 g
(c) 4500 g = 4.5 kg
(d) 750 mL = 0.75 L
(e) 2 h 15 min = 135 min
```

Example 1.2.2: Area and volume units

Convert (a) 3 m^2 to cm^2 (b) $25\,000 \text{ cm}^2$ to m^2 (c) 2 m^3 to cm^3 .

Solution.

Step 1: A square of side 1 m is $100 \text{ cm} \times 100 \text{ cm}$, so $1 \text{ m}^2 = 10\,000 \text{ cm}^2$.

▷ *Why? Area has two dimensions, so the length factor 100 is used twice.*

Step 2: (a) $3 \times 10\,000 = 30\,000 \text{ cm}^2$. (b) $25\,000 \div 10\,000 = 2.5 \text{ m}^2$.

Step 3: A cube of side 1 m is $100 \times 100 \times 100 \text{ cm}^3$, so $1 \text{ m}^3 = 1\,000\,000 \text{ cm}^3$.

Step 4: (c) $2 \times 1\,000\,000 = 2\,000\,000 \text{ cm}^3$.

Remember: a very common mistake is to convert m^2 to cm^2 by multiplying by 100 only.

>>> Python Solution**Example 1.2.2****CODE**

```
1 cm2_per_m2 = 100 ** 2      # area: square the factor
2 cm3_per_m3 = 100 ** 3      # volume: cube the factor
3 print("1 m^2 =", cm2_per_m2, "cm^2; 1 m^3 =", cm3_per_m3, "cm^3")
4 print("(a) 3 m^2      =", 3 * cm2_per_m2, "cm^2")
5 print("(b) 25000 cm^2 =", 25000 / cm2_per_m2, "m^2")
6 print("(c) 2 m^3      =", 2 * cm3_per_m3, "cm^3")
```

OUTPUT

```
1 m^2 = 10000 cm^2; 1 m^3 = 1000000 cm^3
(a) 3 m^2      = 30000 cm^2
(b) 25000 cm^2 = 2.5 m^2
(c) 2 m^3      = 2000000 cm^3
```

Example 1.2.3: Converting a speed

Convert (a) 72 km/h into m/s (b) 15 m/s into km/h .

Solution.

Step 1: (a) Convert the top unit: $72 \text{ km} = 72 \times 1000 = 72\,000 \text{ m}$.

Step 2: Convert the bottom unit: $1 \text{ h} = 3600 \text{ s}$.

Step 3: Speed = $\frac{72\,000 \text{ m}}{3600 \text{ s}} = 20 \text{ m/s}$.

▷ *Why? Since $\frac{1000}{3600} = \frac{5}{18}$, the shortcut is: $\text{km/h} \times \frac{5}{18} = \text{m/s}$.*

Step 4: (b) Reverse the shortcut: $\text{m/s} \times \frac{18}{5} = \text{km/h}$, so $15 \times \frac{18}{5} = 54 \text{ km/h}$.

>>> Python Solution**Example 1.2.3****CODE**

```
1 metres = 72 * 1000
2 seconds = 3600
3 print("(a) 72 km/h =", metres / seconds, "m/s")
4 print("    shortcut:", 72 * 5 / 18, "m/s")
5 print("(b) 15 m/s  =", 15 * 18 / 5, "km/h")
```

OUTPUT

```
(a) 72 km/h = 20.0 m/s
```

```
shortcut: 20.0 m/s
(b) 15 m/s = 54.0 km/h
```

Example 1.2.4: Rectangle

A plot is 12 m long and 8 m wide. Find its perimeter and area.

Solution.

Step 1: Perimeter is the distance round the edge: two lengths and two widths.

Step 2: $P = 2(l + w) = 2(12 + 8) = 2(20) = 40$ m.

Step 3: Area counts the 1 m $\times$ 1 m squares inside: $A = l \times w = 12 \times 8 = 96$ m².

▷ Why? Perimeter is a length (m); area is in square units (m²).

>>> Python Solution

Example 1.2.4

CODE

```
1 l, w = 12, 8
2 print("Perimeter =", 2 * (l + w), "m")
3 print("Area      =", l * w, "m^2")
```

OUTPUT

```
Perimeter = 40 m
Area      = 96 m^2
```

Example 1.2.5: Finding a missing side

A rectangular room has area 84 m² and length 12 m. Find its width and perimeter.

Solution.

Step 1: Use $A = l \times w$ and put in what we know: $84 = 12 \times w$.

Step 2: Divide both sides by 12: $w = 84 \div 12 = 7$ m.

Step 3: Perimeter = $2(12 + 7) = 2(19) = 38$ m.

>>> Python Solution

Example 1.2.5

CODE

```
1 area, length = 84, 12
2 width = area / length
3 print("Width      =", width, "m")
4 print("Perimeter =", 2 * (length + width), "m")
```

OUTPUT

```
Width      = 7.0 m
Perimeter = 38.0 m
```

Example 1.2.6: Triangle

A right triangle has sides 6 cm, 8 cm and 10 cm (the 90° angle is between the 6 cm and 8 cm sides). Find its perimeter and area.

Solution.

Step 1: Perimeter = $6 + 8 + 10 = 24$ cm.

Step 2: In a right triangle, the two sides that meet at 90° act as base and height: $b = 8$, $h = 6$.

Step 3: $A = \frac{1}{2}bh = \frac{1}{2} \times 8 \times 6 = 24 \text{ cm}^2$.

▷ Why? A triangle is half of the rectangle 8×6 .

>>> Python Solution

Example 1.2.6

CODE

```
1 a, b, c = 6, 8, 10
2 print("Perimeter =", a + b + c, "cm")
3 print("Area      =", 0.5 * b * a, "cm^2")      # base 8, height 6
```

OUTPUT

```
Perimeter = 24 cm
Area      = 24.0 cm^2
```

Example 1.2.7: Circle

Find the circumference and area of a circle of radius 7 cm. (Take $\pi = \frac{22}{7}$.)

Solution.

Step 1: Circumference: $C = 2\pi r = 2 \times \frac{22}{7} \times 7$. The 7's cancel: $C = 2 \times 22 = 44 \text{ cm}$.

Step 2: Area: $A = \pi r^2 = \frac{22}{7} \times 7 \times 7 = 22 \times 7 = 154 \text{ cm}^2$.

▷ Why? $\pi = \frac{22}{7}$ is convenient when the radius is a multiple of 7; otherwise use $\pi \approx 3.14$.

>>> Python Solution

Example 1.2.7

CODE

```
1 from fractions import Fraction
2 pi = Fraction(22, 7)
3 r = 7
4 print("Circumference =", 2 * pi * r, "cm")
5 print("Area          =", pi * r ** 2, "cm^2")
```

OUTPUT

```
Circumference = 44 cm
Area          = 154 cm^2
```

Example 1.2.8: Composite shape

A lawn is a rectangle $10 \text{ m} \times 6 \text{ m}$ with a semicircle attached on one 6 m side. Find its area ($\pi \approx 3.14$).

Solution.

Step 1: Split the shape into simple parts: a rectangle and a semicircle.

Step 2: Rectangle: $10 \times 6 = 60 \text{ m}^2$.

Step 3: The semicircle's diameter is 6 m, so $r = 3 \text{ m}$. Area = $\frac{1}{2}\pi r^2 = \frac{1}{2} \times 3.14 \times 9 = 14.13 \text{ m}^2$.

Step 4: Total area = $60 + 14.13 = 74.13 \text{ m}^2$.

>>> Python Solution

Example 1.2.8

CODE

```

1 pi = 3.14
2 rect = 10 * 6
3 r = 6 / 2                                # semicircle diameter = 6
4 semi = 0.5 * pi * r ** 2
5 print("Rectangle :", rect, "m^2")
6 print("Semicircle:", round(semi, 2), "m^2")
7 print("Total      :", round(rect + semi, 2), "m^2")

```

OUTPUT

```

Rectangle : 60 m^2
Semicircle: 14.13 m^2
Total      : 74.13 m^2

```

Example 1.2.9: Volume of a water tank

A tank is 2 m long, 1.5 m wide and 1 m high. How many litres of water can it hold?

Solution.

Step 1: Volume of a cuboid = $lwh = 2 \times 1.5 \times 1 = 3 \text{ m}^3$.

Step 2: $1 \text{ m}^3 = 1000 \text{ L}$, so capacity = $3 \times 1000 = 3000 \text{ litres}$.

▷ Why? Volume tells the space inside; litres are the usual unit for liquids.

>>> Python Solution

Example 1.2.9

CODE

```

1 volume = 2 * 1.5 * 1                    # cubic metres
2 litres = volume * 1000
3 print("Volume =", volume, "m^3")
4 print("Capacity =", litres, "litres")

```

OUTPUT

```

Volume  = 3.0 m^3
Capacity = 3000.0 litres

```

Example 1.2.10: Cylinder

Find the volume of a cylindrical can with radius 3 cm and height 10 cm.

Solution.

Step 1: A cylinder is a stack of circles, so $V = (\text{area of base}) \times \text{height} = \pi r^2 h$.

Step 2: $V = \pi \times 3^2 \times 10 = 90\pi \text{ cm}^3$.

Step 3: $90\pi \approx 90 \times 3.1416 \approx 282.74 \text{ cm}^3$ (about 283 mL).

>>> Python Solution

Example 1.2.10

CODE

```

1 import math
2 r, h = 3, 10
3 V = math.pi * r ** 2 * h
4 print(f"V = 90*pi = {V:.2f} cm^3")

```

OUTPUT

```
V = 90*pi = 282.74 cm^3
```

Example 1.2.11: Cost of painting a room

A room is 5 m long, 4 m wide and 3 m high. Paint costs Rs.150 per m^2 . Find the cost of painting the four walls.

Solution.

Step 1: The four walls form two pairs: two walls 5×3 and two walls 4×3 .

Step 2: Wall area = $2(5 \times 3) + 2(4 \times 3) = 30 + 24 = 54 \text{ m}^2$. (Formula: $2(l + w)h = 2(9)(3) = 54$.)

Step 3: Cost = $54 \times 150 = \text{Rs. } 8100$.

>>> Python Solution

Example 1.2.11

CODE

```
1 l, w, h = 5, 4, 3
2 wall_area = 2 * (l + w) * h
3 cost = wall_area * 150
4 print("Wall area =", wall_area, "m^2")
5 print("Cost      = Rs.", cost)
```

OUTPUT

```
Wall area = 54 m^2
Cost      = Rs. 8100
```

Example 1.2.12: Reasoning with dimensions

Which of $2\pi r$, πr^2 , $\frac{4}{3}\pi r^3$ could be a volume?

Solution.

Step 1: Numbers such as 2, π and $\frac{4}{3}$ have no units. Only the powers of r (a length) matter.

Step 2: $2\pi r$ has r^1 : a length. πr^2 has r^2 : an area. $\frac{4}{3}\pi r^3$ has r^3 : a volume.

▷ Why? Checking dimensions is a quick way to catch a wrong formula.

>>> Python Solution

Example 1.2.12

CODE

```
1 from sympy import symbols, pi, Rational, degree
2 r = symbols("r")
3 formulas = {"2*pi*r": 2*pi*r, "pi*r^2": pi*r**2, "4/3*pi*r^3": Rational(4,
    ↪ 3)*pi*r**3}
4 names = {1: "length", 2: "area", 3: "volume"}
5 for text, f in formulas.items():
6     print(f"{text:<12} has r^{degree(f, r)} -> {names[degree(f, r)]}")
```

OUTPUT

```
2*pi*r      has r^1 -> length
pi*r^2      has r^2 -> area
4/3*pi*r^3  has r^3 -> volume
```

Exercise 1.2 (solve by hand, then check with Python)

1. Convert 3.5 m into centimetres.
2. Convert 90 km/h into m/s.
3. Find the perimeter and area of a square of side 9 cm.
4. Find the area of a triangle with base 10 cm and height 6 cm.
5. A circle has diameter 14 cm. Find its circumference and area ($\pi = \frac{22}{7}$).
6. Find the volume of a cube of edge 4 cm.
7. Convert 10 inches into centimetres.

Multiple Choice Questions – 1.2

1. 1 m^2 is equal to:

(a) 100 cm^2	(b) 1000 cm^2
(c) $10\,000 \text{ cm}^2$	(d) $1\,000\,000 \text{ cm}^2$
2. Volume is measured in:

(a) m	(b) m^2
(c) m^3	(d) kg
3. The area of a circle of radius 1 unit is:

(a) π	(b) 2π
(c) π^2	(d) 1
4. 36 km/h equals:

(a) 6 m/s	(b) 10 m/s
(c) 36 m/s	(d) 3.6 m/s
5. One litre is equal to:

(a) 100 cm^3	(b) 1000 cm^3
(c) 1 m^3	(d) 10 cm^3

Summary of 1.2

- Convert units by multiplying with the right conversion factor; keep units written in every step.
- Square the factor for area, cube the factor for volume.
- $\text{km/h} \times \frac{5}{18} = \text{m/s}$; $1 \text{ m}^3 = 1000 \text{ L}$.
- Perimeter (length unit), area (square unit), volume (cubic unit) – always state the unit.

1.3 Rates, Ratios, Proportions and Percentages**Key Concepts**

- **Ratio** $a : b$ compares two quantities of the *same* kind. It is simplified like a fraction.
- **Rate** compares two quantities of *different* kinds, e.g. km/h, Rs/kg, km/L. A **unit rate** is the rate for one unit (e.g. price per gram).
- **Proportion** is an equality of two ratios: $a : b = c : d \iff ad = bc$ (cross multiplication).

- **Direct proportion:** one quantity increases, the other increases at the same rate ($y = kx$).
- **Inverse proportion:** one quantity increases, the other decreases ($xy = k$).
- **Percentage** means “out of 100”: $x\% = \frac{x}{100}$.

Important Formulas

$$x\% \text{ of } N = \frac{x}{100} \times N, \quad \text{Percentage} = \frac{\text{part}}{\text{whole}} \times 100, \quad \% \text{ change} = \frac{\text{new} - \text{old}}{\text{old}} \times 100$$

Example 1.3.1: Simplifying ratios

Simplify (a) 24 : 36 (b) 50 cm : 2 m (c) $\frac{1}{2} : \frac{1}{3}$.

Solution.

Step 1: (a) HCF of 24 and 36 is 12. Divide both terms: $24 : 36 = 2 : 3$.

▷ Why? Like a fraction, a ratio does not change if both terms are divided by the same number.

Step 2: (b) Make the units the same first: 2 m = 200 cm. So $50 : 200 = 1 : 4$.

Step 3: (c) Multiply both terms by the LCM of the denominators, 6: $\frac{1}{2} \times 6 : \frac{1}{3} \times 6 = 3 : 2$.

>>> Python Solution

Example 1.3.1

CODE

```
1 import math
2
3 def simplify_ratio(a, b):
4     h = math.gcd(a, b)
5     return a // h, b // h
6
7 print("(a) 24:36 ->", simplify_ratio(24, 36))
8 print("(b) 50 cm : 2 m = 50:200 ->", simplify_ratio(50, 2 * 100))
9 print("(c) 1/2 : 1/3, times 6 ->", (6 // 2, 6 // 3))
```

OUTPUT

```
(a) 24:36 -> (2, 3)
(b) 50 cm : 2 m = 50:200 -> (1, 4)
(c) 1/2 : 1/3, times 6 -> (3, 2)
```

Example 1.3.2: Dividing in a ratio

Divide Rs. 5000 between Ali and Sara in the ratio 2 : 3.

Solution.

Step 1: The ratio 2 : 3 means the money is cut into $2 + 3 = 5$ equal parts.

Step 2: Value of one part = $5000 \div 5 = 1000$.

Step 3: Ali gets $2 \times 1000 = \text{Rs. } 2000$; Sara gets $3 \times 1000 = \text{Rs. } 3000$.

Step 4: Check: $2000 + 3000 = 5000$. ✓

>>> Python Solution

Example 1.3.2

CODE

```

1 total = 5000
2 ratio = [2, 3]
3 part = total / sum(ratio)
4 shares = [r * part for r in ratio]
5 print("One part =", part)
6 print("Ali =", shares[0], " Sara =", shares[1], " Check:", sum(shares))

```

OUTPUT

```

One part = 1000.0
Ali = 2000.0  Sara = 3000.0  Check: 5000.0

```

Example 1.3.3: Three-part ratio

Rs. 9000 is shared among three brothers in the ratio 2 : 3 : 4. Find each share.

Solution.

Step 1: Total parts = $2 + 3 + 4 = 9$; one part = $9000 \div 9 = 1000$.

Step 2: Shares: $2 \times 1000 = \text{Rs. } 2000$, $3 \times 1000 = \text{Rs. } 3000$, $4 \times 1000 = \text{Rs. } 4000$.

Step 3: Check: $2000 + 3000 + 4000 = 9000$. ✓

>>> Python Solution

Example 1.3.3

CODE

```

1 total = 9000
2 ratio = [2, 3, 4]
3 part = total / sum(ratio)
4 shares = [r * part for r in ratio]
5 print("Shares:", shares, " Check:", sum(shares))

```

OUTPUT

```

Shares: [2000.0, 3000.0, 4000.0]  Check: 9000.0

```

Example 1.3.4: Solving a proportion

Find x if $3 : 5 = x : 20$.

Solution.

Step 1: Write as fractions: $\frac{3}{5} = \frac{x}{20}$.

Step 2: Cross multiply: $5 \times x = 3 \times 20 \Rightarrow 5x = 60$.

▷ Why? If $\frac{a}{b} = \frac{c}{d}$ then $ad = bc$.

Step 3: Divide by 5: $x = 12$. Check: $\frac{12}{20} = \frac{3}{5}$. ✓

>>> Python Solution

Example 1.3.4

CODE

```

1 from sympy import symbols, Eq, solve, Rational
2 x = symbols("x")
3 print("x =", solve(Eq(Rational(3, 5), x / 20), x))
4 # by cross multiplication: 5x = 3 * 20

```

```
5 print("x =", 3 * 20 / 5)
```

OUTPUT

```
x = [12]
x = 12.0
```

Example 1.3.5: Map scale

A map has scale 1 : 50 000. Two towns are 4 cm apart on the map. Find the real distance in km.

Solution.

Step 1: 1 : 50 000 means 1 cm on the map is 50 000 cm on the ground.

Step 2: Real distance = $4 \times 50\,000 = 200\,000$ cm.

Step 3: Convert: $200\,000$ cm = 2000 m = 2 km.

>>> Python Solution**Example 1.3.5****CODE**

```
1 scale = 50000
2 map_cm = 4
3 real_cm = map_cm * scale
4 print("Real distance =", real_cm, "cm =", real_cm / 100, "m =", real_cm / 100000,
    ↪ "km")
```

OUTPUT

```
Real distance = 200000 cm = 2000.0 m = 2.0 km
```

Example 1.3.6: Unit rate – which is the better buy?

Pack A: 500 g for Rs. 450. Pack B: 1 kg for Rs. 850. Which is cheaper per gram?

Solution.

Step 1: The packs are different sizes, so compare the **unit rate** (price for one gram).

Step 2: Pack A: $450 \div 500 = \text{Rs. } 0.90$ per gram.

Step 3: Pack B: $1 \text{ kg} = 1000 \text{ g}$; $850 \div 1000 = \text{Rs. } 0.85$ per gram.

Step 4: Pack B costs less per gram, so **Pack B** is the better buy.

>>> Python Solution**Example 1.3.6****CODE**

```
1 packs = {"A": (450, 500), "B": (850, 1000)}      # (price, grams)
2 for name, (price, grams) in packs.items():
3     print(f"Pack {name}: Rs. {price / grams:.2f} per gram")
4 best = min(packs, key=lambda k: packs[k][0] / packs[k][1])
5 print("Better buy: Pack", best)
```

OUTPUT

```
Pack A: Rs. 0.90 per gram
Pack B: Rs. 0.85 per gram
Better buy: Pack B
```

Example 1.3.7: Speed, distance and time

A bus travels 150 km at an average speed of 60 km/h. How long does the journey take?

Solution.

Step 1: Speed is a rate: $\text{speed} = \frac{\text{distance}}{\text{time}}$, so $\text{time} = \frac{\text{distance}}{\text{speed}}$.

Step 2: $\text{Time} = \frac{150}{60} = 2.5$ hours.

Step 3: 0.5 h = 30 min, so the journey takes 2 h 30 min.

>>> Python Solution**Example 1.3.7****CODE**

```
1 distance, speed = 150, 60
2 time = distance / speed
3 hours = int(time)
4 minutes = round((time - hours) * 60)
5 print("Time =", time, "hours =", hours, "h", minutes, "min")
```

OUTPUT

```
Time = 2.5 hours = 2 h 30 min
```

Example 1.3.8: Direct proportion

5 pens cost Rs. 150. Find the cost of 8 pens.

Solution.

Step 1: More pens $\Rightarrow$ more cost, at the same rate: this is **direct proportion**.

Step 2: Unitary method – find the cost of *one* pen: $150 \div 5 = \text{Rs. } 30$.

Step 3: Cost of 8 pens = $8 \times 30 = \text{Rs. } 240$.

>>> Python Solution**Example 1.3.8****CODE**

```
1 cost_one = 150 / 5 # unitary method
2 print("One pen = Rs.", cost_one)
3 print("8 pens = Rs.", 8 * cost_one)
```

OUTPUT

```
One pen = Rs. 30.0
8 pens = Rs. 240.0
```

Example 1.3.9: Inverse proportion

6 workers build a wall in 10 days. How long will 15 workers take (working at the same rate)?

Solution.

Step 1: More workers $\Rightarrow$ *fewer* days: this is **inverse proportion**, so the product stays fixed.

Step 2: Total work = $6 \times 10 = 60$ worker-days.

▷ Why? *One worker alone would need 60 days.*

Step 3: Days for 15 workers = $60 \div 15 = 4$ days.

>>> Python Solution

Example 1.3.9

CODE

```
1 work = 6 * 10 # worker-days stay constant
2 print("Total work =", work, "worker-days")
3 print("15 workers need", work / 15, "days")
```

OUTPUT

```
Total work = 60 worker-days
15 workers need 4.0 days
```

Example 1.3.10: Fractions, decimals and percentages

Convert (a) $\frac{3}{8}$ to a decimal and a percentage (b) 0.06 to a percentage (c) 125% to a decimal and a fraction.

Solution.

Step 1: (a) $3 \div 8 = 0.375$; multiply by 100: $0.375 \times 100 = 37.5\%$.

Step 2: (b) $0.06 \times 100 = 6\%$.

▷ Why? Percent means “per hundred”, so decimal $\times 100$ gives the percentage.

Step 3: (c) $125\% = \frac{125}{100} = 1.25 = \frac{5}{4}$.

>>> Python Solution

Example 1.3.10

CODE

```
1 from fractions import Fraction
2 print(f"(a) 3/8 = {3/8} = {3/8:.1%}")
3 print(f"(b) 0.06 = {0.06:.0%}")
4 print("(c) 125% =", 125 / 100, "=", Fraction(125, 100))
```

OUTPUT

```
(a) 3/8 = 0.375 = 37.5%
(b) 0.06 = 6%
(c) 125% = 1.25 = 5/4
```

Example 1.3.11: Finding percentages

(a) Find 15% of 800. (b) A student gets 45 out of 60. What percentage is this?

Solution.

Step 1: (a) “of” means multiply: $\frac{15}{100} \times 800 = 15 \times 8 = 120$.

Step 2: (b) Write the part over the whole: $\frac{45}{60} = 0.75$.

Step 3: Multiply by 100: $0.75 \times 100 = 75\%$.

>>> Python Solution

Example 1.3.11

CODE

```
1 print("(a) 15% of 800 =", 15 / 100 * 800)
2 print(f"(b) 45 out of 60 = {45 / 60 * 100}%")
```

OUTPUT

- (a) 15% of 800 = 120.0
 (b) 45 out of 60 = 75.0%

Example 1.3.12: Percentage increase and decrease

(a) A price rises from Rs. 400 to Rs. 500. (b) A price falls from Rs. 2500 to Rs. 2000. Find the percentage change in each case.

Solution.

Step 1: (a) Change = $500 - 400 = 100$ (an increase).

Step 2: Divide by the **original** value: $\frac{100}{400} \times 100 = 25\%$ increase.

▷ Why? Percentage change always compares with where we started.

Step 3: (b) Change = $2500 - 2000 = 500$ (a decrease); $\frac{500}{2500} \times 100 = 20\%$ decrease.

>>> Python Solution

Example 1.3.12

CODE

```
1 def percent_change(old, new):
2     return (new - old) / old * 100
3
4 print(f"(a) 400 -> 500 : {percent_change(400, 500):+.0f}%")
5 print(f"(b) 2500 -> 2000: {percent_change(2500, 2000):+.0f}%")
```

OUTPUT

```
(a) 400 -> 500 : +25%
(b) 2500 -> 2000: -20%
```

Example 1.3.13: Discount

A shirt costs Rs. 2500 and has a 20% discount. Find the sale price.

Solution.

Step 1: Discount = $\frac{20}{100} \times 2500 = \text{Rs. } 500$.

Step 2: Sale price = $2500 - 500 = \text{Rs. } 2000$.

Step 3: Quick method: after 20% off you pay 80%: $0.80 \times 2500 = \text{Rs. } 2000$.

>>> Python Solution

Example 1.3.13

CODE

```
1 price = 2500
2 discount = 20 / 100 * price
3 print("Discount = Rs.", discount)
4 print("Sale price = Rs.", price - discount)
5 print("Quick way = Rs.", 0.80 * price)
```

OUTPUT

```
Discount = Rs. 500.0
Sale price = Rs. 2000.0
Quick way = Rs. 2000.0
```

Example 1.3.14: Reverse percentage

After a 20% increase, a salary is Rs. 36 000. What was it before the increase?

Solution.

Step 1: The new salary is $100\% + 20\% = 120\%$ of the old one: $1.2 \times \text{old} = 36\,000$.

Step 2: Old salary $= 36\,000 \div 1.2 = \text{Rs. } 30\,000$.

▷ *Why? Taking 20% of 36 000 and subtracting would be wrong – the 20% was of the old salary.*

Step 3: Check: $30\,000 + 20\% \text{ of } 30\,000 = 30\,000 + 6000 = 36\,000$. ✓

>>> Python Solution**Example 1.3.14****CODE**

```
1 new_salary = 36000
2 old_salary = new_salary / 1.20
3 print("Old salary = Rs.", old_salary)
4 print("Check:", old_salary * 1.20)
```

OUTPUT

```
Old salary = Rs. 30000.0
Check: 36000.0
```

Example 1.3.15: Profit percentage

A shopkeeper buys a fan for Rs. 8000 and sells it for Rs. 10 000. Find the profit percentage.

Solution.

Step 1: Profit = selling price – cost price = $10\,000 - 8000 = \text{Rs. } 2000$.

Step 2: Profit % = $\frac{\text{profit}}{\text{cost price}} \times 100 = \frac{2000}{8000} \times 100 = 25\%$.

>>> Python Solution**Example 1.3.15****CODE**

```
1 cost, sale = 8000, 10000
2 profit = sale - cost
3 print("Profit = Rs.", profit)
4 print(f"Profit % = {profit / cost * 100:.0f}%")
```

OUTPUT

```
Profit = Rs. 2000
Profit % = 25%
```

Example 1.3.16: A common trap: successive percentages

A price of Rs. 100 is increased by 10% and then decreased by 10%. Is it back to Rs. 100?

Solution.

Step 1: After the 10% increase: $100 + 10 = 110$.

Step 2: The 10% decrease is taken of the *new* price: 10% of 110 is 11, so $110 - 11 = 99$.

Step 3: Final price Rs. 99, i.e. a net 1% **decrease**.

▷ *Why? The two percentages are of different bases (100 and 110), so they do not cancel.*

>>> Python Solution

Example 1.3.16

CODE

```

1 price = 100
2 after_rise = price * 1.10
3 after_fall = after_rise * 0.90
4 print(f"After +10%: {after_rise:.0f}")
5 print(f"After -10%: {after_fall:.0f}")
6 print(f"Net change: {(after_fall - price) / price:.0%}")

```

OUTPUT

```

After +10%: 110
After -10%: 99
Net change: -1%

```

Exercise 1.3 (solve by hand, then check with Python)

1. Simplify the ratio 45 : 60.
2. Divide Rs. 1200 in the ratio 1 : 2 : 3.
3. Find x if $x : 9 = 4 : 12$.
4. A car travels 240 km on 16 litres of petrol. Find the fuel rate in km/L.
5. 4 workers finish a job in 12 days. How many days will 8 workers take?
6. 18 is what percentage of 72?
7. A salary of Rs. 40 000 increases by 12%. Find the new salary.
8. A shirt marked Rs. 1500 is sold for Rs. 1200. Find the percentage discount.

Multiple Choice Questions – 1.3

1. The ratio 50 : 75 in simplest form is:

(a) 1 : 2	(b) 2 : 3
(c) 3 : 4	(d) 5 : 7
2. 25% of 240 is:

(a) 48	(b) 60
(c) 80	(d) 96
3. A price rises 20% and then falls 20%. The net change is:

(a) no change	(b) 4% decrease
(c) 4% increase	(d) 2% decrease
4. Speed (km/h) is an example of a:

(a) ratio	(b) rate
(c) percentage	(d) proportion
5. If x and y are inversely proportional, then:

(a) $x + y$ is constant	(b) x/y is constant
(c) xy is constant	(d) $x - y$ is constant

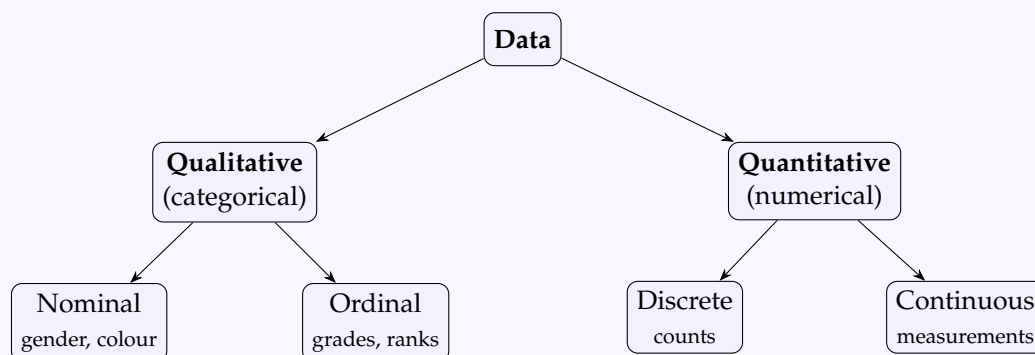
Summary of 1.3

- Ratio: same units; rate: different units; unit rates help compare prices and speeds.
- Proportion $a : b = c : d$ is solved by cross multiplication $ad = bc$.
- Direct: divide then multiply (unitary method). Inverse: product stays constant.
- % change = $\frac{\text{change}}{\text{original}} \times 100$; successive percentages do not simply add.

1.4 Types and Sources of Data

Key Concepts

Data are facts or figures collected to study something. A characteristic that changes from person to person (or item to item) is called a **variable**.



- **Qualitative data** describe a quality or category (e.g. blood group, city, religion).
- **Quantitative data** are numbers:
Discrete – obtained by **counting**, take separate values (number of students, cars).
Continuous – obtained by **measuring**, can take any value in a range (height, weight, time).
- **Primary data** – collected *first-hand* by the investigator (survey, interview, questionnaire, experiment, observation).
- **Secondary data** – already collected by someone else (Pakistan Bureau of Statistics, census reports, Economic Survey of Pakistan, State Bank reports, books, websites).
- **Cross-sectional data** are collected at one point in time; **time-series data** are collected over time (e.g. yearly inflation).

Example 1.4.1: Classifying variables

Classify: (i) gender, (ii) number of brothers, (iii) height, (iv) blood group, (v) temperature, (vi) cars in a parking lot.

Solution.

Step 1: Ask: *is the answer a number?* Gender and blood group are categories $\Rightarrow$ **qualitative**.

Step 2: For numerical variables ask: *counted or measured?*

Step 3: Counted (only separate values 0, 1, 2, ...): number of brothers, cars $\Rightarrow$ **discrete**.

Step 4: Measured (any value in a range, e.g. 165.3 cm): height, temperature $\Rightarrow$ **continuous**.

Variable	Type	Reason
Gender	Qualitative	a category, not a number
Number of brothers	Quantitative, discrete	counted: 0, 1, 2, ...
Height	Quantitative, continuous	measured; can be 165.3 cm
Blood group	Qualitative	categories A, B, AB, O
Temperature	Quantitative, continuous	measured
Cars in a parking lot	Quantitative, discrete	counted

>>> Python Solution

Example 1.4.1

CODE

```

1 variables = {
2     "gender":      ("words", None),
3     "brothers":    ("number", "counted"),
4     "height":      ("number", "measured"),
5     "blood group": ("words", None),
6     "temperature": ("number", "measured"),
7     "cars in a lot": ("number", "counted"),
8 }
9 for name, (answer, how) in variables.items():
10     if answer == "words":
11         kind = "qualitative"
12     elif how == "counted":
13         kind = "quantitative, discrete"
14     else:
15         kind = "quantitative, continuous"
16     print(f"{name:<14} -> {kind}")

```

OUTPUT

```

gender          -> qualitative
brothers        -> quantitative, discrete
height          -> quantitative, continuous
blood group     -> qualitative
temperature     -> quantitative, continuous
cars in a lot   -> quantitative, discrete

```

Example 1.4.2: Trickier cases

Classify: (a) shoe size, (b) time to solve a puzzle, (c) goals scored in a match, (d) rainfall in a month.

Solution.

Step 1: (a) Shoe sizes come in fixed steps ($7, 7\frac{1}{2}, 8, \dots$) with nothing in between $\Rightarrow$ **discrete**.

▷ Why? Being a number with a decimal does not make data continuous; what matters is whether every in-between value is possible.

Step 2: (b) Time can be 2.5 min, 2.53 min, ... $\Rightarrow$ **continuous**.

Step 3: (c) Goals are counted $\Rightarrow$ **discrete**.

Step 4: (d) Rainfall is measured in mm $\Rightarrow$ **continuous**.

>>> Python Solution

Example 1.4.2

CODE

```

1 # Discrete: only separate values are possible. Continuous: any value in a range.
2 in_between_possible = {"shoe size": False, "time to solve puzzle": True,
3                        "goals in a match": False, "monthly rainfall": True}
4 for name, between in in_between_possible.items():
5     print(f"{name:<22} -> {'continuous' if between else 'discrete'}")

```

OUTPUT

```

shoe size          -> discrete
time to solve puzzle -> continuous
goals in a match   -> discrete
monthly rainfall   -> continuous

```

Example 1.4.3: Primary or secondary?

- (a) A student asks 50 classmates how many hours they study daily.
 (b) A researcher uses population figures from the Census report of Pakistan.

Solution.

Step 1: Ask: *who collected the data, and for what purpose?*

Step 2: (a) The student collected it personally for this study ⇒ **primary**.

Step 3: (b) The Pakistan Bureau of Statistics collected it earlier; the researcher re-uses it ⇒ **secondary**.

▷ *Why? Primary data fit your question exactly but cost time; secondary data are quick but may not match your needs.*

>>> Python Solution

Example 1.4.3

CODE

```

1 studies = [
2     ("student asks 50 classmates", "self"),
3     ("researcher uses census figures", "Pakistan Bureau of Statistics"),
4 ]
5 for what, collector in studies:
6     kind = "primary" if collector == "self" else "secondary"
7     print(f"{what:<32} -> {kind} (collected by {collector})")

```

OUTPUT

```

student asks 50 classmates      -> primary (collected by self)
researcher uses census figures  -> secondary (collected by Pakistan Bureau of
Statistics)

```

Example 1.4.4: Judging a source

Which source is more reliable for Pakistan's inflation rate: a social-media post or the State Bank of Pakistan website?

Solution.

Step 1: Ask three questions of any source: *Who* collected it? *How*? *When*?

Step 2: The State Bank is an official body that publishes its method and dates.

Step 3: A social-media post usually gives no method or date and may be copied wrongly.

Step 4: So the **State Bank website** is more reliable.

>>> Python Solution

Example 1.4.4

CODE

```
1 sources = {
2     "State Bank of Pakistan website": {"who": True, "how": True, "when": True},
3     "social-media post": {"who": False, "how": False, "when": False},
4 }
5 for name, checks in sources.items():
6     score = sum(checks.values())
7     print(f"{name:<31} passes {score}/3 checks")
8 print("More reliable:", max(sources, key=lambda s: sum(sources[s].values())))
```

OUTPUT

```
State Bank of Pakistan website passes 3/3 checks
social-media post                passes 0/3 checks
More reliable: State Bank of Pakistan website
```

Example 1.4.5: Planning a data collection

A teacher wants to study the study habits of her BS students. Suggest a method and classify three variables she could record.

Solution.

Step 1: Method: a short **questionnaire** given in class – this produces *primary* data.

Step 2: Variable “gender” $\Rightarrow$ qualitative (nominal).

Step 3: Variable “number of subjects studied daily” $\Rightarrow$ quantitative, discrete.

Step 4: Variable “hours of study per day” $\Rightarrow$ quantitative, continuous.

▷ Why? Knowing the type of each variable early tells us which tables and graphs to use later.

>>> Python Solution

Example 1.4.5

CODE

```
1 survey = {
2     "gender": "qualitative (nominal)",
3     "subjects studied daily": "quantitative, discrete",
4     "hours of study per day": "quantitative, continuous",
5 }
6 print("Method: questionnaire -> primary data")
7 for var, kind in survey.items():
8     print(f" {var:<24} {kind}")
```

OUTPUT

```
Method: questionnaire -> primary data
gender                qualitative (nominal)
subjects studied daily quantitative, discrete
hours of study per day quantitative, continuous
```

Example 1.4.6: Cross-sectional or time-series?

(a) Marks of 40 students in the October 2026 test. (b) The monthly price of petrol from January to December.

Solution.

Step 1: (a) Many individuals, *one* point in time $\Rightarrow$ **cross-sectional**.

Step 2: (b) One quantity followed *over time* $\Rightarrow$ **time-series** (best shown on a line graph).

>>> Python Solution**Example 1.4.6****CODE**

```
1 datasets = {"marks of 40 students, Oct 2026": "one time", "monthly petrol price,
    ↪ Jan-Dec": "over time"}
2 for name, when in datasets.items():
3     kind = "cross-sectional" if when == "one time" else "time-series"
4     print(f"{name:<32} -> {kind}")
```

OUTPUT

```
marks of 40 students, Oct 2026    -> cross-sectional
monthly petrol price, Jan-Dec    -> time-series
```

Exercise 1.4 (solve by hand, then check with Python)

- Classify each as qualitative, discrete or continuous: (a) eye colour, (b) marks in a test out of 100, (c) weight of a newborn baby, (d) number of mobile phones in a house, (e) province of residence, (f) time to run 100 m.
- State whether the data are primary or secondary: (a) data from the Economic Survey of Pakistan; (b) a shopkeeper records daily sales himself; (c) a teacher measures heights of her students.
- Give one example each of cross-sectional data and time-series data.

Multiple Choice Questions – 1.4

- The number of students in a class is:

(a) qualitative	(b) continuous
(c) discrete	(d) nominal
- Weight of a person is:

(a) discrete	(b) continuous
(c) qualitative	(d) ordinal
- Data taken from a census report are:

(a) primary	(b) secondary
(c) qualitative	(d) experimental
- Which is qualitative data?

(a) age	(b) income
(c) religion	(d) height
- Yearly wheat production of Pakistan from 2015 to 2025 is:

(a) time-series data	(b) cross-sectional data
(c) nominal data	(d) primary data

Summary of 1.4

- Qualitative = categories; quantitative = numbers (discrete = counted, continuous = measured).
- Primary data are collected by you; secondary data come from others (reports, websites).
- Check the source, method and date of any data before trusting it.

1.5 Measurement Scales

Key Concepts

A **measurement scale** tells us what we can do with the values of a variable.

Scale	Order?	Equal gaps?	True zero?	Examples
Nominal	No	No	No	gender, blood group, shirt numbers, CNIC numbers
Ordinal	Yes	No	No	grades A/B/C, ranks 1st/2nd/3rd, satisfaction level
Interval	Yes	Yes	No	temperature in °C, calendar years, IQ score
Ratio	Yes	Yes	Yes	height, weight, age, income, distance

Memory tip – NOIR: Nominal → Ordinal → Interval → Ratio (each level has all properties of the level before it, plus one more).

Example 1.5.1: Identify the scale

Identify the scale: (a) cricket shirt numbers; (b) position in a race; (c) temperature in °C; (d) monthly income.

Solution.

Step 1: (a) A shirt number is just a label; shirt 10 is not “more” than shirt 5 ⇒ **nominal**.

Step 2: (b) 1st is ahead of 2nd (order), but the time gap between 1st and 2nd need not equal the gap between 2nd and 3rd ⇒ **ordinal**.

Step 3: (c) Equal gaps (10° to 20° is the same rise as 20° to 30°), but 0°C does not mean “no temperature” ⇒ **interval**.

Step 4: (d) Rs. 0 means no income, and Rs. 60 000 is twice Rs. 30 000 ⇒ **ratio**.

>>> Python Solution

Example 1.5.1

CODE

```

1 def scale(order, equal_gaps, true_zero):
2     if not order:         return "nominal"
3     if not equal_gaps:    return "ordinal"
4     if not true_zero:     return "interval"
5     return "ratio"
6
7 print("shirt numbers :", scale(False, False, False))
8 print("race position :", scale(True, False, False))

```

```

9 print("temperature C :", scale(True, True, False))
10 print("monthly income :", scale(True, True, True))

```

OUTPUT

```

shirt numbers : nominal
race position : ordinal
temperature C : interval
monthly income : ratio

```

Example 1.5.2: A three-question test

Use the questions (1) *Is there an order?* (2) *Are the gaps equal?* (3) *Is there a true zero?* to classify: education level (Matric/Inter/BS), year of birth, exam marks, mobile numbers.

Solution.

Step 1: Education level: order *yes*, equal gaps *no* $\Rightarrow$ **ordinal**.

Step 2: Year of birth: order *yes*, equal gaps *yes*, but year 0 is not “no time” $\Rightarrow$ **interval**.

Step 3: Exam marks: order, equal gaps, and 0 marks means none correct $\Rightarrow$ **ratio**.

Step 4: Mobile numbers: no meaningful order $\Rightarrow$ **nominal**.

▷ *Why? Stop at the first “no”: that tells you the level.*

>>> Python Solution**Example 1.5.2****CODE**

```

1 def scale(order, equal_gaps, true_zero):
2     if not order: return "nominal"
3     if not equal_gaps: return "ordinal"
4     if not true_zero: return "interval"
5     return "ratio"
6
7 items = {"education level": (True, False, False), "year of birth": (True, True,
8     ↪ False),
9     "exam marks": (True, True, True), "mobile numbers": (False, False, False)}
10 for name, answers in items.items():
11     print(f"{name:<16} -> {scale(*answers)}")

```

OUTPUT

```

education level -> ordinal
year of birth   -> interval
exam marks      -> ratio
mobile numbers  -> nominal

```

Example 1.5.3: Why is 20°C not “twice as hot” as 10°C?**Solution.**

Step 1: Celsius has no true zero (0°C is just the freezing point of water).

Step 2: Convert to Fahrenheit using $F = \frac{9}{5}C + 32$: 10°C = 50°F and 20°C = 68°F; 68 is not twice 50.

Step 3: The “twice” disappears when the units change, so ratios are meaningless on an interval scale; only differences are meaningful.

By contrast, 60 kg *is* twice 30 kg in any unit, because weight is on a ratio scale.

>>> Python Solution

Example 1.5.3

CODE

```
1 to_f = lambda c: 9 / 5 * c + 32
2 print("10 C =", to_f(10), "F; 20 C =", to_f(20), "F")
3 print("Ratio in Celsius   :", 20 / 10)
4 print("Ratio in Fahrenheit:", round(to_f(20) / to_f(10), 2))
5 print("Ratio of 60 kg to 30 kg:", 60 / 30, "(same in any unit)")
```

OUTPUT

```
10 C = 50.0 F; 20 C = 68.0 F
Ratio in Celsius   : 2.0
Ratio in Fahrenheit: 1.36
Ratio of 60 kg to 30 kg: 2.0 (same in any unit)
```

Example 1.5.4: Which average is allowed?

Which average is sensible for (a) favourite colour, (b) class rank, (c) temperature, (d) weight?

Solution.

Step 1: (a) Nominal: only counting is meaningful $\Rightarrow$ use the **mode**.

Step 2: (b) Ordinal: we can sort, so the **median** (and mode) make sense.

Step 3: (c) Interval: equal gaps allow adding, so the **mean** is fine (but not ratios).

Step 4: (d) Ratio: all averages *and* ratios such as “twice as heavy” are allowed.

>>> Python Solution

Example 1.5.4

CODE

```
1 allowed = {"nominal": "mode", "ordinal": "median (and mode)",
2           "interval": "mean, median, mode (no ratios)", "ratio": "all averages
   ↪ and ratios"}
3 variables = {"favourite colour": "nominal", "class rank": "ordinal",
4             "temperature": "interval", "weight": "ratio"}
5 for v, s in variables.items():
6     print(f"{v:<16} ({s:<8}) -> {allowed[s]}")
```

OUTPUT

```
favourite colour (nominal ) -> mode
class rank       (ordinal ) -> median (and mode)
temperature      (interval) -> mean, median, mode (no ratios)
weight           (ratio   ) -> all averages and ratios
```

Example 1.5.5: Rating scales

A survey uses Strongly disagree (1), Disagree (2), Neutral (3), Agree (4), Strongly agree (5). Which scale is this?

Solution.

Step 1: The codes have an order: 5 shows more agreement than 4.

Step 2: But is the gap from “Neutral” to “Agree” the same as from “Agree” to “Strongly

agree"? We cannot be sure.

Step 3: So the scale is **ordinal**; the median is the safest average.

▷ Why? The numbers 1–5 are only codes; writing them as numbers does not make the gaps equal.

>>> Python Solution

Example 1.5.5

CODE

```
1 import statistics as st
2 codes = {1: "Strongly disagree", 2: "Disagree", 3: "Neutral", 4: "Agree", 5:
    ↪ "Strongly agree"}
3 answers = [4, 5, 3, 4, 2, 4, 5]
4 m = st.median(answers)
5 print("Median code:", m, "->", codes[m])    # median is safe for ordinal data
```

OUTPUT

```
Median code: 4 -> Agree
```

Exercise 1.5 (solve by hand, then check with Python)

1. Name the scale: (a) province of birth, (b) height in cm, (c) customer rating Poor/Fair/Good/Excellent, (d) year of birth, (e) number of children, (f) BS/MS/PhD education level.
2. Which scales allow us to compute “twice as much”? Explain with an example.
3. Why is it not sensible to find the average of shirt numbers?

Multiple Choice Questions – 1.5

1. Blood group is measured on a/an:

(a) nominal scale	(b) ordinal scale
(c) interval scale	(d) ratio scale
2. Ranks in a class are measured on a/an:

(a) nominal scale	(b) ordinal scale
(c) interval scale	(d) ratio scale
3. Which scale has a true zero?

(a) nominal	(b) ordinal
(c) interval	(d) ratio
4. Temperature in °C is on a/an:

(a) nominal scale	(b) ordinal scale
(c) interval scale	(d) ratio scale

Summary of 1.5

- NOIR: Nominal (labels), Ordinal (order), Interval (equal gaps), Ratio (true zero).
- Only ratio data allow statements like “twice as much”.
- The scale decides which statistics are sensible (e.g. mode for nominal, median for ordinal, mean for interval/ratio).

1.6 Tabular and Graphical Presentation of Data

Key Concepts

Frequency distribution: a table showing how many times (frequency f) each value or class occurs.

- **Class interval** e.g. 10–19; **class limits** 10 and 19; **class boundaries** 9.5 and 19.5; **class width** = 10.
- **Mid-point** $x = \frac{\text{lower limit} + \text{upper limit}}{2}$.
- **Relative frequency** $= \frac{f}{\sum f}$; **Cumulative frequency** = running total of f .

Common graphs

- **Bar chart** – categories, separate bars of equal width.
- **Pie chart** – parts of a whole; angle of a sector $= \frac{f}{\text{total}} \times 360^\circ$.
- **Histogram** – grouped continuous data; bars touch each other (drawn on class boundaries).
- **Frequency polygon** – join mid-points of tops of histogram bars.
- **Line graph** – change over time. **Ogive** – graph of cumulative frequency.

Example 1.6.1: Making a grouped frequency table

Marks (out of 50) of 20 students:

12, 25, 37, 18, 42, 29, 33, 15, 27, 39, 21, 45, 31, 24, 36, 19, 28, 41, 34, 26

Make a frequency table with classes 10–19, 20–29, 30–39, 40–49.

Solution.

Step 1: Find the range: smallest = 12, largest = 45; range = 33. Four classes of width 10 cover 10–49.

▷ *Why? Classes must cover every value with no gaps or overlaps.*

Step 2: Sort each mark into its class:

10–19: 12, 18, 15, 19 (4 marks)

20–29: 25, 29, 27, 21, 24, 28, 26 (7 marks)

30–39: 37, 33, 39, 31, 36, 34 (6 marks)

40–49: 42, 45, 41 (3 marks)

Step 3: Check the total: $4 + 7 + 6 + 3 = 20$. ✓

Step 4: Boundaries: subtract 0.5 from lower limits and add 0.5 to upper limits (to close the gaps between 19 and 20, etc.).

Step 5: Mid-point $= \frac{\text{lower} + \text{upper}}{2}$, e.g. $\frac{10 + 19}{2} = 14.5$.

Step 6: Relative frequency $= f \div 20$, e.g. $4 \div 20 = 0.20$; cumulative frequency is a running total.

Class	Boundaries	Mid-point	f	Relative f	Cumulative f
10–19	9.5–19.5	14.5	4	0.20	4
20–29	19.5–29.5	24.5	7	0.35	11
30–39	29.5–39.5	34.5	6	0.30	17
40–49	39.5–49.5	44.5	3	0.15	20
Total			20	1.00	

Interpretation: the most common class is 20–29 (35% of students); 17 students scored below 39.5.

>>> Python Solution

Example 1.6.1

CODE

```

1 marks = [12, 25, 37, 18, 42, 29, 33, 15, 27, 39,
2         21, 45, 31, 24, 36, 19, 28, 41, 34, 26]
3 classes = [(10, 19), (20, 29), (30, 39), (40, 49)]
4 n = len(marks)
5 cum = 0
6 print(f"{'Class':<8}{'Bound.':<12}{'Mid':>5}{'f':>4}{'Rel f':>7}{'Cum f':>7}")
7 for lo, hi in classes:
8     f = sum(1 for m in marks if lo <= m <= hi)
9     cum += f
10    cls = f"{lo}-{hi}"
11    bound = f"{lo - 0.5}-{hi + 0.5}"
12    print(f"{cls:<8}{bound:<12}{(lo + hi) / 2:>5}{f:>4}{f / n:>7.2f}{cum:>7}")
13 print("Total:", n)

```

OUTPUT

```

Class Bound.      Mid  f Rel f Cum f
10-19  9.5-19.5    14.5 4  0.20    4
20-29  19.5-29.5   24.5 7  0.35   11
30-39  29.5-39.5   34.5 6  0.30   17
40-49  39.5-49.5   44.5 3  0.15   20
Total: 20

```

Example 1.6.2: Frequency table for discrete data

Goals scored by a team in 20 matches: 0, 1, 2, 1, 3, 0, 2, 1, 1, 4, 2, 0, 1, 3, 2, 1, 0, 2, 1, 1.

Make a frequency table and describe the data.

Solution.

Step 1: The values are few and separate (0 to 4), so list each value instead of making classes.

Step 2: Count each value carefully (cross them out as you go):

Goals x	0	1	2	3	4	Total
Frequency f	4	8	5	2	1	20

Step 3: Check: $4 + 8 + 5 + 2 + 1 = 20$ matches. ✓

Step 4: The most common result is 1 goal (8 matches); only 3 matches had 3 or more goals.

>>> Python Solution

Example 1.6.2

CODE

```

1 goals = [0, 1, 2, 1, 3, 0, 2, 1, 1, 4, 2, 0, 1, 3, 2, 1, 0, 2, 1, 1]
2 freq = {}
3 for g in goals:
4     freq[g] = freq.get(g, 0) + 1
5 for g in sorted(freq):
6     print(f"Goals {g}: {freq[g]}")
7 print("Total matches:", sum(freq.values()))
8 print("Most common:", max(freq, key=freq.get), "goal")

```

OUTPUT

```

Goals 0: 4
Goals 1: 8
Goals 2: 5
Goals 3: 2
Goals 4: 1
Total matches: 20
Most common: 1 goal

```

Example 1.6.3: Drawing a histogram and frequency polygon

Draw a histogram for the marks table in Example 1.6.1.

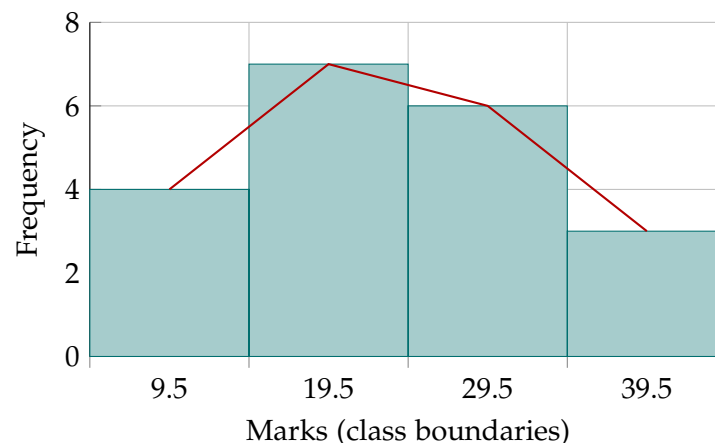
Solution.

Step 1: Put class **boundaries** on the x -axis and frequency on the y -axis.

▷ *Why? Using boundaries makes the bars touch, showing the data are continuous.*

Step 2: Draw each bar from one boundary to the next with height equal to its frequency.

Step 3: For the frequency polygon, mark the mid-point of the top of each bar and join with straight lines.



>>> Python Solution

Example 1.6.3

CODE

```

1 import matplotlib.pyplot as plt
2 marks = [12, 25, 37, 18, 42, 29, 33, 15, 27, 39,
3          21, 45, 31, 24, 36, 19, 28, 41, 34, 26]
4 bounds = [9.5, 19.5, 29.5, 39.5, 49.5]

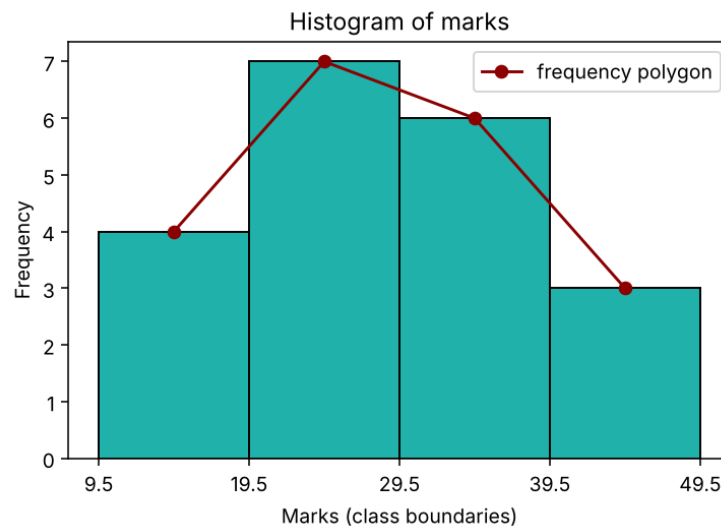
```

```

5 plt.figure(figsize=(6, 3.8))
6 counts, _, _ = plt.hist(marks, bins=bounds, color="lightseagreen",
    ↪ edgecolor="black")
7 mids = [14.5, 24.5, 34.5, 44.5]
8 plt.plot(mids, counts, "o-", color="darkred", label="frequency polygon")
9 plt.xticks(bounds)
10 plt.xlabel("Marks (class boundaries)"); plt.ylabel("Frequency")
11 plt.title("Histogram of marks"); plt.legend()
12 plt.show()

```

OUTPUT (FIGURE)



Example 1.6.4: Ogive (cumulative frequency curve)

Draw an ogive for the marks table and use it to find how many students scored below 29.5 and above 39.5.

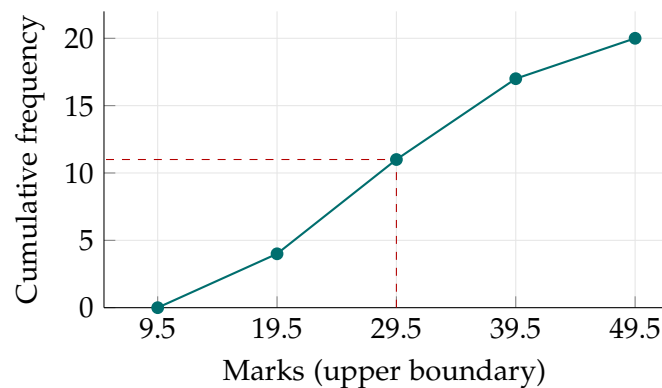
Solution.

Step 1: Plot cumulative frequency against the **upper** boundary: $(9.5, 0)$, $(19.5, 4)$, $(29.5, 11)$, $(39.5, 17)$, $(49.5, 20)$.

▷ Why? Cumulative frequency counts all values below the upper boundary.

Step 2: Join the points.

Step 3: Below 29.5: read 11 students. Above 39.5: $20 - 17 = 3$ students.



>>> Python Solution

Example 1.6.4

CODE

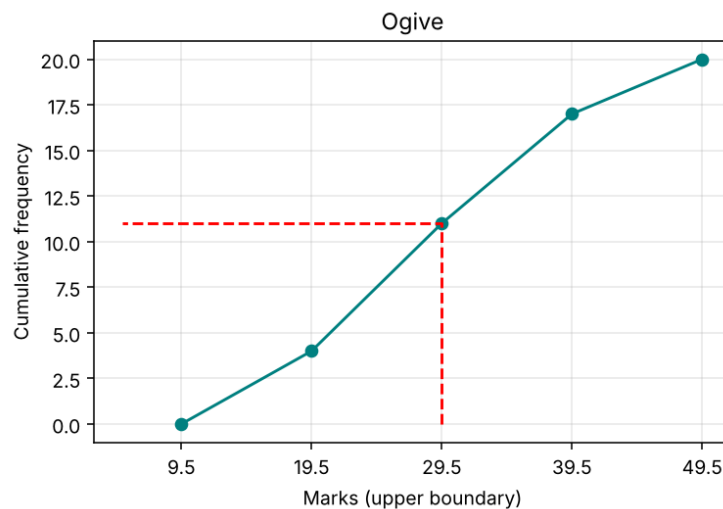
```

1 import matplotlib.pyplot as plt
2 upper = [9.5, 19.5, 29.5, 39.5, 49.5]
3 cum_f = [0, 4, 11, 17, 20]
4 plt.figure(figsize=(6, 3.8))
5 plt.plot(upper, cum_f, "o-", color="teal")
6 plt.plot([29.5, 29.5, 5], [0, 11, 11], "r--")
7 plt.xticks(upper); plt.grid(alpha=0.3)
8 plt.xlabel("Marks (upper boundary)"); plt.ylabel("Cumulative frequency")
9 plt.title("Ogive")
10 plt.show()
11 print("Below 29.5:", cum_f[2], " Above 39.5:", cum_f[-1] - cum_f[3])

```

OUTPUT

Below 29.5: 11 Above 39.5: 3



Example 1.6.5: Pie chart

A family's monthly budget of Rs. 60 000: Food 24 000, Rent 18 000, Transport 6 000, Education 9 000, Savings 3 000. Draw a pie chart.

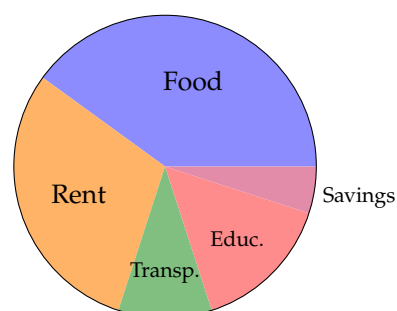
Solution.

Step 1: The whole budget corresponds to the full circle, 360° .

Step 2: Angle of each sector = $\frac{\text{amount}}{60\,000} \times 360^\circ$, e.g. Food: $\frac{24\,000}{60\,000} \times 360 = 0.4 \times 360 = 144^\circ$.

Step 3: Check the angles add to 360° , then draw them with a protractor.

Item	Amount	Angle
Food	24 000	144°
Rent	18 000	108°
Transport	6 000	36°
Education	9 000	54°
Savings	3 000	18°
Total	60 000	$360^\circ \checkmark$



Interpretation: food takes the largest share (40%); savings are only 5%.

>>> Python Solution

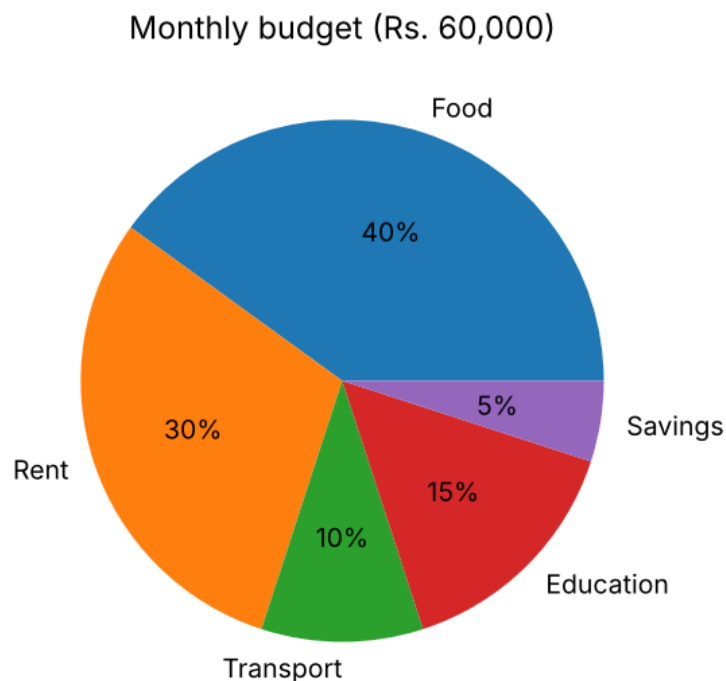
Example 1.6.5

CODE

```
1 import matplotlib.pyplot as plt
2 budget = {"Food": 24000, "Rent": 18000, "Transport": 6000,
3           "Education": 9000, "Savings": 3000}
4 total = sum(budget.values())
5 for item, amount in budget.items():
6     print(f"{item:<10} {amount / total * 360:>5.0f} degrees")
7 plt.figure(figsize=(4.5, 4.5))
8 plt.pie(budget.values(), labels=budget.keys(), autopct="%1.0f%%", startangle=0,
9         counterclock=True)
10 plt.title("Monthly budget (Rs. 60,000)")
11 plt.show()
```

OUTPUT

Food	144 degrees
Rent	108 degrees
Transport	36 degrees
Education	54 degrees
Savings	18 degrees



Example 1.6.6: Reading a pie chart backwards

In a pie chart of 500 students, the sector for “Arts” has angle 72° . How many students study Arts?

Solution.

Step 1: The fraction of the circle is $\frac{72}{360} = \frac{1}{5}$.

Step 2: The same fraction of students: $\frac{1}{5} \times 500 = 100$ students.

>>> Python Solution

Example 1.6.6

CODE

```
1 angle, total_students = 72, 500
2 arts = angle / 360 * total_students
3 print("Students in Arts:", arts)
```

OUTPUT

```
Students in Arts: 100.0
```

Example 1.6.7: Bar chart

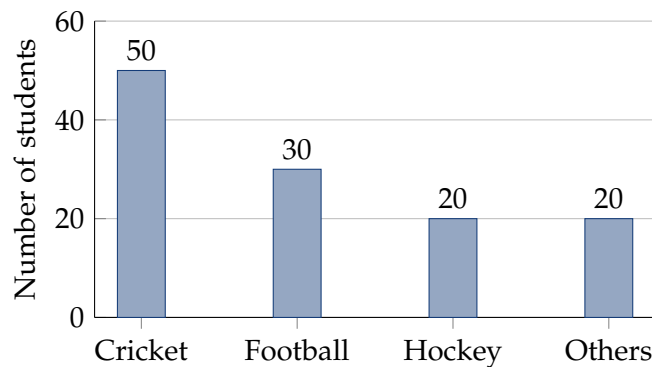
Favourite sport of 120 students: Cricket 50, Football 30, Hockey 20, Others 20. Draw a bar chart.

Solution.

Step 1: Put the categories on the x -axis and the number of students on the y -axis, starting at 0.

▷ Why? Starting the axis at zero keeps bar heights honest.

Step 2: Draw bars of equal width with gaps between them.



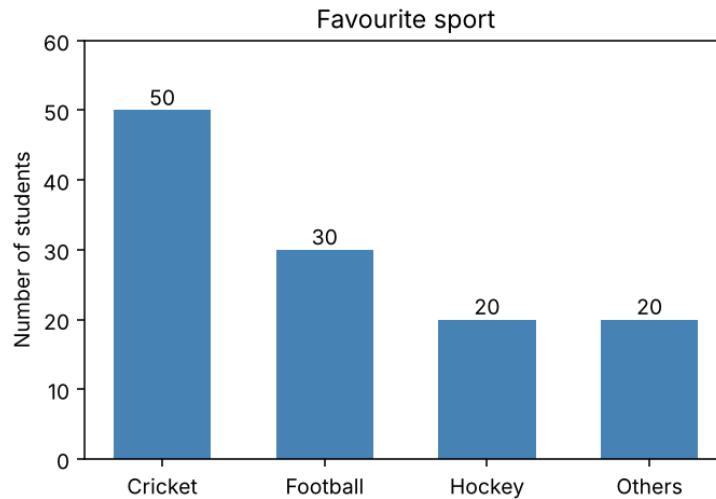
>>> Python Solution

Example 1.6.7

CODE

```
1 import matplotlib.pyplot as plt
2 sports = ["Cricket", "Football", "Hockey", "Others"]
3 students = [50, 30, 20, 20]
4 plt.figure(figsize=(5.5, 3.6))
5 bars = plt.bar(sports, students, color="steelblue", width=0.6)
6 plt.bar_label(bars)
7 plt.ylim(0, 60) # start the axis at zero
8 plt.ylabel("Number of students"); plt.title("Favourite sport")
9 plt.show()
```

OUTPUT (FIGURE)



Example 1.6.8: Line graph

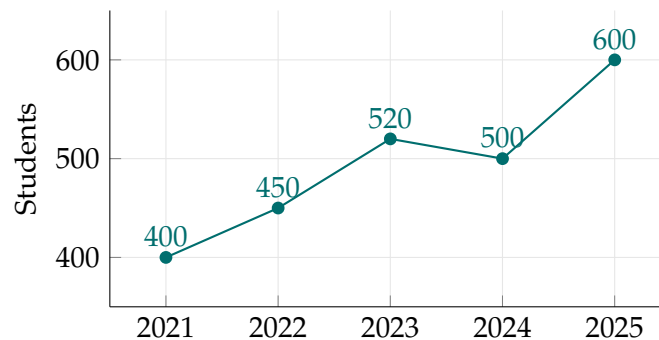
Enrolment in a college: 2021: 400, 2022: 450, 2023: 520, 2024: 500, 2025: 600. Draw a line graph and interpret it.

Solution.

Step 1: Time (years) goes on the x -axis; enrolment on the y -axis.

Step 2: Plot the five points and join them in order.

Step 3: Read the changes: +50, +70, -20, +100.



Interpretation: enrolment grew overall by 200 (50%), with one small fall in 2024 and the biggest rise in 2025.

>>> Python Solution

Example 1.6.8

CODE

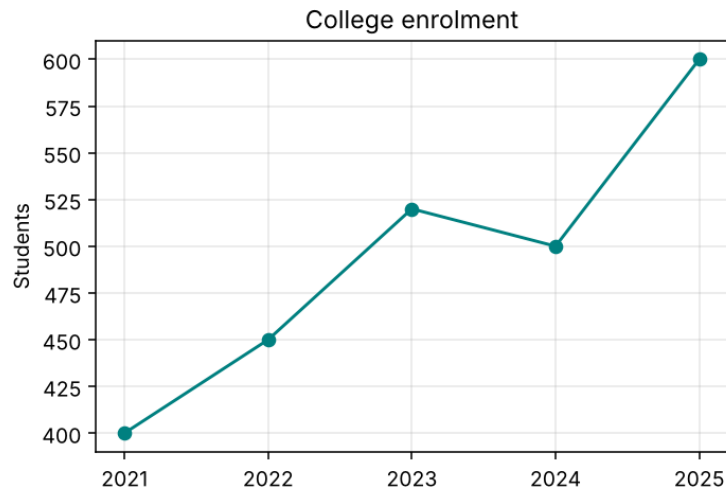
```
1 import matplotlib.pyplot as plt
2 years = [2021, 2022, 2023, 2024, 2025]
3 enrol = [400, 450, 520, 500, 600]
4 changes = [b - a for a, b in zip(enrol, enrol[1:])]
5 print("Year-to-year changes:", changes)
6 print(f"Overall growth: {enrol[-1] - enrol[0]} ({(enrol[-1] - enrol[0]) /
   ↪ enrol[0]:.0%})")
7 plt.figure(figsize=(5.5, 3.6))
8 plt.plot(years, enrol, "o-", color="teal")
9 plt.xticks(years); plt.grid(alpha=0.3)
```

```
10 plt.ylabel("Students"); plt.title("College enrolment")
11 plt.show()
```

OUTPUT

Year-to-year changes: [50, 70, -20, 100]

Overall growth: 200 (50%)

**Example 1.6.9: Choosing the right graph**

Choose a suitable graph for: (a) market share of mobile networks, (b) monthly temperature over a year, (c) heights of 100 students, (d) students in each department.

Solution.

Step 1: (a) Parts of a whole $\Rightarrow$ **pie chart**.

Step 2: (b) Change over time $\Rightarrow$ **line graph**.

Step 3: (c) Continuous data in classes $\Rightarrow$ **histogram**.

Step 4: (d) Separate categories $\Rightarrow$ **bar chart**.

>>> Python Solution**Example 1.6.9****CODE**

```
1 def choose_graph(purpose):
2     rules = {"parts of a whole": "pie chart", "change over time": "line graph",
3             "continuous data in classes": "histogram", "separate categories":
4             "\u2192 "bar chart"}
5     return rules[purpose]
6
7 tasks = {"market share of networks": "parts of a whole",
8         "monthly temperature": "change over time",
9         "heights of 100 students": "continuous data in classes",
10        "students per department": "separate categories"}
11 for task, purpose in tasks.items():
12     print(f"{task:<26} -> {choose_graph(purpose)}")
```

OUTPUT

```
market share of networks    -> pie chart
monthly temperature        -> line graph
```

heights of 100 students -> histogram
 students per department -> bar chart

Exercise 1.6 (solve by hand, then check with Python)

1. The number of children in 15 families is: 2, 3, 1, 4, 2, 3, 2, 5, 1, 2, 3, 0, 2, 4, 3. Make a frequency table and find the most common number of children.
2. In a class of 120 students, the favourite sports are: Cricket 50, Football 30, Hockey 20, Others 20. Find the angle of each sector for a pie chart.
3. For the class 30–39, write the class boundaries, class width and mid-point.
4. What is the main difference between a bar chart and a histogram?

Multiple Choice Questions – 1.6

1. In a pie chart, the total angle is:

(a) 180°	(b) 90°
(c) 360°	(d) 100°
2. The mid-point of the class 20–29 is:

(a) 24	(b) 24.5
(c) 25	(d) 9
3. A histogram is used for:

(a) nominal data	(b) grouped continuous data
(c) only percentages	(d) ranks
4. Cumulative frequency of the last class always equals:

(a) zero	(b) the largest frequency
(c) the total frequency	(d) the class width
5. A quantity worth 25% of the total gets a pie-chart angle of:

(a) 25°	(b) 45°
(c) 90°	(d) 120°

Summary of 1.6

- Organise raw data into a frequency table: classes, frequencies, mid-points, relative and cumulative frequencies.
- Bar chart for categories; pie chart for parts of a whole; histogram for grouped continuous data; line graph for time.
- Pie-chart angle = $\frac{f}{\text{total}} \times 360^\circ$.
- Always label axes, give a title, and state what the graph shows.

1.7 Quantitative Reasoning Exercises Using Number Knowledge

A Problem-Solving Plan

1. **Understand** – what is given, what is asked, which units?
2. **Plan** – which idea fits: ratio, percentage, rate, area, unit conversion...?
3. **Solve** – work step by step, keeping units.
4. **Check** – is the answer reasonable? Estimate first and compare.

Example 1.7.1: Estimation

Estimate (a) 49.8×20.1 (b) $\frac{612}{29.7}$ without a calculator.

Solution.

Step 1: Round each number to an easy value: $49.8 \approx 50$, $20.1 \approx 20$.

Step 2: (a) $50 \times 20 = 1000$ (exact: 1000.98).

Step 3: (b) $612 \approx 600$ and $29.7 \approx 30$: $600 \div 30 = 20$ (exact: 20.6).

▷ Why? An estimate shows whether a calculator answer is sensible – e.g. 10 009.8 would clearly be a typing error.

>>> Python Solution

Example 1.7.1

CODE

```
1 print("(a) estimate:", 50 * 20, " exact:", round(49.8 * 20.1, 2))
2 print("(b) estimate:", 600 / 30, " exact:", round(612 / 29.7, 1))
```

OUTPUT

```
(a) estimate: 1000 exact: 1000.98
(b) estimate: 20.0 exact: 20.6
```

Example 1.7.2: Electricity bill (slab rates)

The first 100 units cost Rs. 10 per unit and every unit above 100 costs Rs. 15. Find the bill for 250 units.

Solution.

Step 1: Split the units into slabs: first 100 units, then the remaining $250 - 100 = 150$ units.

Step 2: First slab: $100 \times 10 = \text{Rs. } 1000$.

Step 3: Second slab: $150 \times 15 = \text{Rs. } 2250$.

Step 4: Total bill = $1000 + 2250 = \text{Rs. } 3250$.

▷ Why? Charging all 250 units at Rs. 15 (= Rs. 3750) is a common mistake.

>>> Python Solution

Example 1.7.2

CODE

```
1 units = 250
2 first = min(units, 100) * 10
3 rest = max(units - 100, 0) * 15
```

```

4 print("First 100 units: Rs.", first)
5 print("Remaining units: Rs.", rest)
6 print("Total bill      : Rs.", first + rest)

```

OUTPUT

```

First 100 units: Rs. 1000
Remaining units: Rs. 2250
Total bill      : Rs. 3250

```

Example 1.7.3: Fuel cost of a trip

A town is 360 km from Quetta. A car gives 12 km/L and petrol costs Rs. 270/L. Find the fuel cost of the trip.

Solution.

Step 1: Fuel needed = $\frac{\text{distance}}{\text{fuel rate}} = \frac{360}{12} = 30$ litres.

Step 2: Cost = $30 \times 270 = \text{Rs. } 8100$.

Step 3: For a return trip, double it: Rs. 16 200.

>>> Python Solution**Example 1.7.3****CODE**

```

1 distance, km_per_litre, price = 360, 12, 270
2 litres = distance / km_per_litre
3 cost = litres * price
4 print("Fuel needed:", litres, "litres")
5 print("One way: Rs.", cost, "   Return trip: Rs.", 2 * cost)

```

OUTPUT

```

Fuel needed: 30.0 litres
One way: Rs. 8100.0   Return trip: Rs. 16200.0

```

Example 1.7.4: Absolute versus relative change

Shop A's sales grew from 200 to 250 items; Shop B's grew from 1000 to 1100. Which grew more?

Solution.

Step 1: Absolute increase: A = 50, B = 100. B grew more in *number*.

Step 2: Relative increase: A = $\frac{50}{200} \times 100 = 25\%$, B = $\frac{100}{1000} \times 100 = 10\%$.

Step 3: A grew faster in *percentage*.

▷ Why? Both statements are true; a fair report states which measure it uses.

>>> Python Solution**Example 1.7.4****CODE**

```

1 shops = {"A": (200, 250), "B": (1000, 1100)}
2 for name, (old, new) in shops.items():
3     print(f"Shop {name}: increase {new - old}, relative {(new - old) / old:.0%}")

```

OUTPUT

Shop A: increase 50, relative 25%
 Shop B: increase 100, relative 10%

Example 1.7.5: Simple interest

Find the simple interest on Rs. 50 000 at 8% per year for 3 years.

Solution.

Step 1: Interest for one year = 8% of 50 000 = Rs. 4000.

Step 2: Simple interest is the same every year: $3 \times 4000 = \text{Rs. } 12\,000$. (Formula: $I = \frac{PRT}{100}$.)

Step 3: Total amount = $50\,000 + 12\,000 = \text{Rs. } 62\,000$.

>>> Python Solution

Example 1.7.5

CODE

```
1 P, R, T = 50000, 8, 3
2 I = P * R * T / 100
3 print("Interest = Rs.", I)
4 print("Amount = Rs.", P + I)
```

OUTPUT

```
Interest = Rs. 12000.0
Amount = Rs. 62000.0
```

Example 1.7.6: Simple versus compound interest

Rs. 10 000 is invested for 2 years at 10% per year. Compare simple and compound interest.

Solution.

Step 1: Simple: 10% of 10 000 = 1000 each year; after 2 years: $10\,000 + 2000 = \text{Rs. } 12\,000$.

Step 2: Compound, year 1: $10\,000 + 1000 = 11\,000$.

Step 3: Compound, year 2: interest is on 11 000: 1100; total = Rs. 12 100.

▷ Why? In compound interest, interest also earns interest. Formula: $A = P \left(1 + \frac{r}{100}\right)^n = 10\,000(1.1)^2 = 12\,100$.

>>> Python Solution

Example 1.7.6

CODE

```
1 P, r, n = 10000, 10, 2
2 simple = P + P * r * n / 100
3 compound = P * (1 + r / 100) ** n
4 print(f"Simple : Rs. {simple:,.0f}")
5 print(f"Compound: Rs. {compound:,.0f}")
6 for year in range(1, n + 1):
7     print(f" after year {year}: {P * 1.1 ** year:,.0f}")
```

OUTPUT

```
Simple : Rs. 12,000
Compound: Rs. 12,100
 after year 1: 11,000
 after year 2: 12,100
```

Example 1.7.7: Will the mobile data last?

A student has a 25 GB monthly package. After 20 days she has used 18.5 GB. At this rate, will it last 30 days?

Solution.

Step 1: Daily use = $18.5 \div 20 = 0.925$ GB per day.

Step 2: Use in 30 days = $0.925 \times 30 = 27.75$ GB, which is more than 25 GB.

Step 3: Days the package lasts = $25 \div 0.925 \approx 27$ days. So **no**, it runs out about 3 days early.

>>> Python Solution**Example 1.7.7****CODE**

```
1 used, days, package = 18.5, 20, 25
2 per_day = used / days
3 print("Daily use:", per_day, "GB")
4 print("Needed for 30 days:", round(per_day * 30, 2), "GB")
5 print("Package lasts about", int(package / per_day), "days")
```

OUTPUT

```
Daily use: 0.925 GB
Needed for 30 days: 27.75 GB
Package lasts about 27 days
```

Example 1.7.8: Scaling a recipe

A recipe for 4 people needs 300 g of rice. How much rice is needed for 10 people?

Solution.

Step 1: Rice per person = $300 \div 4 = 75$ g.

Step 2: For 10 people: $75 \times 10 = 750$ g.

Step 3: *Check with ratio:* $10 \div 4 = 2.5$ times as many people, so $2.5 \times 300 = 750$ g. ✓

>>> Python Solution**Example 1.7.8****CODE**

```
1 rice_for_4 = 300
2 per_person = rice_for_4 / 4
3 print("Per person:", per_person, "g; for 10 people:", per_person * 10, "g")
```

OUTPUT

```
Per person: 75.0 g; for 10 people: 750.0 g
```

Example 1.7.9: Making sense of large numbers

A city of 2.4 million people spends on average Rs. 50 per person per day on transport. Find the yearly total.

Solution.

Step 1: Daily total = $2.4 \times 10^6 \times 50 = 120 \times 10^6 = 1.2 \times 10^8$ rupees (120 million).

Step 2: Yearly total = $1.2 \times 10^8 \times 365 = 438 \times 10^8 = 4.38 \times 10^{10}$ rupees.

Step 3: In words: about Rs. 43.8 billion a year.

▷ Why? Scientific notation keeps track of the zeros in large calculations.

>>> Python Solution

Example 1.7.9

CODE

```
1 people = 2.4e6
2 daily = people * 50
3 yearly = daily * 365
4 print(f"Daily : {daily:.1e} rupees")
5 print(f"Yearly: {yearly:.2e} rupees = Rs. {yearly / 1e9:.1f} billion")
```

OUTPUT

```
Daily : 1.2e+08 rupees
Yearly: 4.38e+10 rupees = Rs. 43.8 billion
```

Example 1.7.10: The average-speed trap

A car goes 60 km to a town at 30 km/h and returns at 60 km/h. Is the average speed 45 km/h?

Solution.

Step 1: Time going = $60 \div 30 = 2$ h; time returning = $60 \div 60 = 1$ h.

Step 2: Total distance = 120 km; total time = 3 h.

Step 3: Average speed = $\frac{\text{total distance}}{\text{total time}} = \frac{120}{3} = 40$ km/h, **not** 45.

▷ Why? The car spends more time at the slower speed, so the slower speed counts more.

>>> Python Solution

Example 1.7.10

CODE

```
1 d = 60
2 t_go, t_back = d / 30, d / 60
3 average = (2 * d) / (t_go + t_back)
4 print("Times:", t_go, "h and", t_back, "h")
5 print("Average speed =", average, "km/h (not", (30 + 60) / 2, ")")
```

OUTPUT

```
Times: 2.0 h and 1.0 h
Average speed = 40.0 km/h (not 45.0 )
```

Exercise 1.7 (solve by hand, then check with Python)

1. Estimate 198×5.03 .
2. A mobile package charges Rs. 1.5 per minute. How much do 3 hours of calls cost?
3. A bus covers 150 km in 2.5 hours. Find its average speed. How long will it take for 240 km at the same speed?
4. Find the simple interest on Rs. 20 000 at 5% per annum for 4 years.
5. A town's population grew from 80 000 to 92 000. Find the percentage increase.
6. A room is 5 m by 4 m. Tiles cost Rs. 1200 per m². Find the cost of tiling the floor.

Multiple Choice Questions – 1.7

1. The best estimate of 31×49 is:
(a) 150 (b) 1500
(c) 15 000 (d) 300
2. Simple interest on Rs. 1000 at 10% for 2 years is:
(a) Rs. 100 (b) Rs. 200
(c) Rs. 210 (d) Rs. 20
3. A car uses 5 L for 60 km. Fuel needed for 180 km:
(a) 10 L (b) 12 L
(c) 15 L (d) 18 L
4. Sales rise from 50 to 60. The percentage increase is:
(a) 10% (b) 16.7%
(c) 20% (d) 60%

Summary of 1.7

- Use Understand – Plan – Solve – Check for every word problem.
- Estimate first to catch calculation errors.
- Distinguish absolute change (difference) from relative change (percentage).
- Simple interest: $I = \frac{PRT}{100}$.

UNIT 2

Fundamental Mathematical Concepts

2.1 Basics of Geometry (Lines, Angles, Circles, Polygons)

Key Concepts

Lines

- **Line** – extends without end in both directions; **ray** – one end point; **line segment** – two end points.
- **Parallel lines** never meet; **perpendicular lines** meet at 90° .

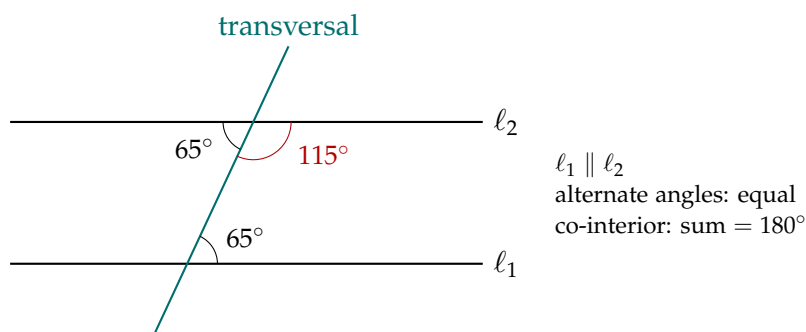
Angles

- Acute ($< 90^\circ$), right ($= 90^\circ$), obtuse (90° – 180°), straight ($= 180^\circ$), reflex (180° – 360°).
- **Complementary** angles add to 90° ; **supplementary** angles add to 180° .
- **Vertically opposite** angles are equal; angles at a point add to 360° .
- When a transversal cuts two parallel lines: *corresponding* angles are equal, *alternate* angles are equal, *co-interior* angles add to 180° .

Triangles: angles add to 180° . **Pythagoras theorem** (right triangle): $c^2 = a^2 + b^2$, where c is the hypotenuse.

Circles: radius r , diameter $d = 2r$, chord, arc, sector. Arc length $= \frac{\theta}{360^\circ} \times 2\pi r$, sector area $= \frac{\theta}{360^\circ} \times \pi r^2$.

Polygons (n sides): sum of interior angles $= (n - 2) \times 180^\circ$; for a *regular* polygon, each interior angle $= \frac{(n - 2) \times 180^\circ}{n}$ and each exterior angle $= \frac{360^\circ}{n}$.



Example 2.1.1: Complement and supplement

Find the complement and the supplement of 35° .

Solution.

Step 1: Complementary angles add to 90° : complement $= 90^\circ - 35^\circ = 55^\circ$.

Step 2: Supplementary angles add to 180° : $\text{supplement} = 180^\circ - 35^\circ = 145^\circ$.

▷ Why? Memory aid: *Complement – Corner* (90°); *Supplement – Straight line* (180°).

>>> Python Solution

Example 2.1.1

CODE

```
1 angle = 35
2 print("Complement :", 90 - angle, "degrees")
3 print("Supplement :", 180 - angle, "degrees")
```

OUTPUT

```
Complement : 55 degrees
Supplement : 145 degrees
```

Example 2.1.2: Intersecting lines

Two straight lines cross. One of the angles formed is 40° . Find the other three angles.

Solution.

Step 1: The angle opposite the 40° angle is **vertically opposite**, so it is also 40° .

Step 2: The angle next to 40° lies on a straight line with it: $180^\circ - 40^\circ = 140^\circ$.

Step 3: Its opposite angle is also 140° .

Step 4: Check: $40 + 140 + 40 + 140 = 360^\circ$ (angles at a point). ✓

>>> Python Solution

Example 2.1.2

CODE

```
1 a = 40
2 opposite = a                # vertically opposite angles are equal
3 adjacent = 180 - a          # angles on a straight line
4 angles = [a, adjacent, opposite, adjacent]
5 print("Angles:", angles, " Sum:", sum(angles))
```

OUTPUT

```
Angles: [40, 140, 40, 140] Sum: 360
```

Example 2.1.3: Angles at a point

Three angles at a point are x , $2x$ and $3x$. Find each angle.

Solution.

Step 1: Angles round a point add to 360° : $x + 2x + 3x = 360^\circ$.

Step 2: Collect like terms: $6x = 360^\circ \Rightarrow x = 60^\circ$.

Step 3: The angles are 60° , 120° and 180° .

>>> Python Solution

Example 2.1.3

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 sol = solve(Eq(x + 2*x + 3*x, 360), x)[0]
```

```
4 print("x =", sol, " Angles:", [sol, 2*sol, 3*sol])
```

OUTPUT

```
x = 60 Angles: [60, 120, 180]
```

Example 2.1.4: Angles of a triangle

Two angles of a triangle are 50° and 60° . Find the third angle.

Solution.

Step 1: The angles of any triangle add to 180° .

Step 2: Add the known angles: $50^\circ + 60^\circ = 110^\circ$.

Step 3: Third angle = $180^\circ - 110^\circ = 70^\circ$.

>>> Python Solution

Example 2.1.4

CODE

```
1 known = [50, 60]
2 third = 180 - sum(known)
3 print("Third angle =", third, "degrees")
```

OUTPUT

```
Third angle = 70 degrees
```

Example 2.1.5: Triangle angles with algebra

The angles of a triangle are x , $x + 20^\circ$ and $2x$. Find them.

Solution.

Step 1: Angle sum: $x + (x + 20) + 2x = 180$.

Step 2: Simplify: $4x + 20 = 180 \Rightarrow 4x = 160 \Rightarrow x = 40$.

Step 3: Angles: $40^\circ, 60^\circ, 80^\circ$. Check: $40 + 60 + 80 = 180$. ✓

>>> Python Solution

Example 2.1.5

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 sol = solve(Eq(x + (x + 20) + 2*x, 180), x)[0]
4 angles = [sol, sol + 20, 2*sol]
5 print("x =", sol, " Angles:", angles, " Sum:", sum(angles))
```

OUTPUT

```
x = 40 Angles: [40, 60, 80] Sum: 180
```

Example 2.1.6: Exterior angle of a triangle

Two interior angles of a triangle are 55° and 65° . Find the exterior angle at the third vertex.

Solution.

Step 1: Third interior angle = $180^\circ - (55^\circ + 65^\circ) = 60^\circ$.

Step 2: The exterior angle is on a straight line with it: $180^\circ - 60^\circ = 120^\circ$.

Step 3: *Shortcut:* an exterior angle equals the sum of the two opposite interior angles: $55^\circ + 65^\circ = 120^\circ$. ✓

>>> Python Solution

Example 2.1.6

CODE

```
1 a, b = 55, 65
2 third = 180 - (a + b)
3 print("Third interior angle:", third)
4 print("Exterior angle      :", 180 - third)
5 print("Shortcut a + b      :", a + b)
```

OUTPUT

```
Third interior angle: 60
Exterior angle      : 120
Shortcut a + b      : 120
```

Example 2.1.7: Pythagoras theorem

The two shorter sides of a right triangle are 6 cm and 8 cm. Find the hypotenuse.

Solution.

Step 1: The hypotenuse c is the longest side, opposite the right angle: $c^2 = a^2 + b^2$.

Step 2: $c^2 = 6^2 + 8^2 = 36 + 64 = 100$.

Step 3: Take the square root: $c = \sqrt{100} = 10$ cm.

>>> Python Solution

Example 2.1.7

CODE

```
1 import math
2 a, b = 6, 8
3 c = math.sqrt(a**2 + b**2)
4 print("c^2 =", a**2 + b**2, " c =", c, "cm")
5 print("Using math.hypot:", math.hypot(a, b))
```

OUTPUT

```
c^2 = 100  c = 10.0 cm
Using math.hypot: 10.0
```

Example 2.1.8: Is it a right triangle?

Test whether triangles with sides (a) 7, 24, 25 (b) 5, 6, 8 are right-angled.

Solution.

Step 1: Compare the square of the longest side with the sum of squares of the other two.

Step 2: (a) $7^2 + 24^2 = 49 + 576 = 625$ and $25^2 = 625$. Equal $\Rightarrow$ **right-angled**.

Step 3: (b) $5^2 + 6^2 = 25 + 36 = 61$ but $8^2 = 64$. Not equal $\Rightarrow$ not right-angled.

▷ Why? Since $64 > 61$, the largest angle is more than 90° : the triangle is obtuse.

>>> Python Solution

Example 2.1.8

CODE

```

1 def triangle_type(a, b, c):
2     a, b, c = sorted([a, b, c])          # c is now the longest side
3     if a**2 + b**2 == c**2:
4         return "right-angled"
5     return "obtuse" if c**2 > a**2 + b**2 else "acute"
6
7 for sides in [(7, 24, 25), (5, 6, 8)]:
8     print(sides, "->", triangle_type(*sides))

```

OUTPUT

```

(7, 24, 25) -> right-angled
(5, 6, 8) -> obtuse

```

Example 2.1.9: Regular hexagon

Find the sum of interior angles, each interior angle and each exterior angle of a regular hexagon.

Solution.

Step 1: A hexagon has $n = 6$ sides. It can be cut into $n - 2 = 4$ triangles from one corner.

Step 2: Sum = $(6 - 2) \times 180^\circ = 720^\circ$.

Step 3: Each interior angle (all equal in a regular polygon) = $720^\circ \div 6 = 120^\circ$.

Step 4: Each exterior angle = $360^\circ \div 6 = 60^\circ$. Check: $120^\circ + 60^\circ = 180^\circ$. ✓

>>> Python Solution

Example 2.1.9

CODE

```

1 n = 6
2 total = (n - 2) * 180
3 print("Sum of interior angles:", total)
4 print("Each interior angle   :", total / n)
5 print("Each exterior angle   :", 360 / n)

```

OUTPUT

```

Sum of interior angles: 720
Each interior angle   : 120.0
Each exterior angle   : 60.0

```

Example 2.1.10: Finding the number of sides

The interior angles of a polygon add to 1440° . How many sides does it have?

Solution.

Step 1: Use the formula: $(n - 2) \times 180 = 1440$.

Step 2: Divide by 180: $n - 2 = 8$.

Step 3: So $n = 10$: the polygon is a decagon.

>>> Python Solution

Example 2.1.10

CODE

```
1 from sympy import symbols, Eq, solve
2 n = symbols("n")
3 print("Number of sides:", solve(Eq((n - 2) * 180, 1440), n)[0])
```

OUTPUT

```
Number of sides: 10
```

Example 2.1.11: Parallel lines

In the figure above, one angle is 65° . Find the alternate angle and the co-interior angle.

Solution.

Step 1: Alternate angles ("Z" shape) between parallel lines are equal: 65° .

Step 2: Co-interior angles ("C" shape) are supplementary: $180^\circ - 65^\circ = 115^\circ$.

>>> Python Solution

Example 2.1.11

CODE

```
1 given = 65
2 print("Alternate angle :", given)
3 print("Co-interior angle:", 180 - given)
```

OUTPUT

```
Alternate angle : 65
Co-interior angle: 115
```

Example 2.1.12: Parallel lines with algebra

Two corresponding angles on parallel lines are $3x + 10^\circ$ and $5x - 30^\circ$. Find x and the angle.

Solution.

Step 1: Corresponding angles ("F" shape) are equal: $3x + 10 = 5x - 30$.

Step 2: Move x -terms to one side: $10 + 30 = 5x - 3x \Rightarrow 40 = 2x \Rightarrow x = 20$.

Step 3: Angle = $3(20) + 10 = 70^\circ$. Check: $5(20) - 30 = 70^\circ$. ✓

>>> Python Solution

Example 2.1.12

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 sol = solve(Eq(3*x + 10, 5*x - 30), x)[0]
4 print("x =", sol, " angle =", 3*sol + 10, "degrees")
```

OUTPUT

```
x = 20  angle = 70 degrees
```

Example 2.1.13: Arc length and sector area

A circle has radius 7 cm. Find the arc length and area of a sector with angle 90° ($\pi = \frac{22}{7}$).

Solution.

Step 1: A sector of 90° is $\frac{90}{360} = \frac{1}{4}$ of the whole circle.

Step 2: Full circumference $= 2 \times \frac{22}{7} \times 7 = 44$ cm, so arc length $= \frac{1}{4} \times 44 = 11$ cm.

Step 3: Full area $= \frac{22}{7} \times 49 = 154$ cm², so sector area $= \frac{1}{4} \times 154 = 38.5$ cm².

>>> Python Solution

Example 2.1.13

CODE

```
1 from fractions import Fraction
2 pi = Fraction(22, 7)
3 r, theta = 7, 90
4 part = Fraction(theta, 360)
5 print("Fraction of circle:", part)
6 print("Arc length :", part * 2 * pi * r, "cm")
7 print("Sector area:", float(part * pi * r**2), "cm^2")
```

OUTPUT

```
Fraction of circle: 1/4
Arc length : 11 cm
Sector area: 38.5 cm^2
```

Example 2.1.14: Wheel and distance

A bicycle wheel has radius 35 cm. How far does the bicycle travel in 100 revolutions? ($\pi = \frac{22}{7}$)

Solution.

Step 1: In one revolution the wheel rolls a distance equal to its circumference.

Step 2: $C = 2 \times \frac{22}{7} \times 35 = 2 \times 22 \times 5 = 220$ cm.

Step 3: In 100 revolutions: $100 \times 220 = 22\,000$ cm = 220 m.

>>> Python Solution

Example 2.1.14

CODE

```
1 from fractions import Fraction
2 C = 2 * Fraction(22, 7) * 35           # circumference in cm
3 distance = 100 * C
4 print("Circumference:", C, "cm")
5 print("Distance:", distance, "cm =", distance / 100, "m")
```

OUTPUT

```
Circumference: 220 cm
Distance: 22000 cm = 220 m
```

Exercise 2.1 (solve by hand, then check with Python)

1. Find the supplement of 72° .
2. Two angles of a triangle are 45° and 85° . Find the third angle.
3. The hypotenuse of a right triangle is 13 cm and one side is 5 cm. Find the other side.
4. Find each interior angle of a regular octagon.

5. Each exterior angle of a regular polygon is 30° . How many sides does it have?
6. Find the circumference of a circle of radius 21 cm ($\pi = \frac{22}{7}$).

Multiple Choice Questions – 2.1

1. An angle of 120° is:

(a) acute	(b) right
(c) obtuse	(d) reflex
2. The sum of interior angles of a pentagon is:

(a) 360°	(b) 540°
(c) 720°	(d) 180°
3. A triangle with sides 3, 4, 5 is:

(a) equilateral	(b) right-angled
(c) obtuse	(d) isosceles
4. Co-interior angles between parallel lines are:

(a) equal	(b) complementary
(c) supplementary	(d) each 90°
5. The complement of 40° is:

(a) 50°	(b) 140°
(c) 40°	(d) 320°

Summary of 2.1

- Complementary = 90° , supplementary = 180° , triangle angles = 180° .
- Parallel lines: corresponding and alternate angles equal; co-interior add to 180° .
- Pythagoras: $c^2 = a^2 + b^2$.
- Polygon: interior sum $(n - 2)180^\circ$; exterior angles always add to 360° .
- Arc and sector are the fraction $\frac{\theta}{360^\circ}$ of the full circumference and area.

2.2 Sets and their Operations

Key Concepts

A **set** is a well-defined collection of distinct objects (elements). We write $a \in A$ for “ a is an element of A ”.

- **Roster form:** $A = \{1, 2, 3, 4, 5\}$. **Set-builder form:** $A = \{x \mid x \in \mathbb{N}, x \leq 5\}$.
- **Empty set** $\emptyset = \{ \}$; **singleton** – one element; **finite / infinite** sets.
- $A \subseteq B$ (**subset**) if every element of A is in B . A set with n elements has 2^n subsets; the set of all subsets is the **power set** $P(A)$.
- **Universal set** U contains all elements under discussion.

Operations

Union	$A \cup B$	elements in A or B (or both)
Intersection	$A \cap B$	elements in <i>both</i> A and B
Difference	$A - B$	elements in A but not in B
Complement	A'	elements of U not in A

Counting formula: $n(A \cup B) = n(A) + n(B) - n(A \cap B)$.

De Morgan's laws: $(A \cup B)' = A' \cap B'$ and $(A \cap B)' = A' \cup B'$.

Example 2.2.1: Roster and set-builder forms

(a) Write $\{x \mid x \text{ is a vowel of the English alphabet}\}$ in roster form. (b) Write $\{1, 4, 9, 16, 25\}$ in set-builder form. (c) Write $\{x \mid x \in \mathbb{N}, x < 6\}$ in roster form.

Solution.

Step 1: (a) List every element: $\{a, e, i, o, u\}$.

Step 2: (b) Find the rule: $1 = 1^2, 4 = 2^2, \dots, 25 = 5^2$. So $\{x^2 \mid x \in \mathbb{N}, x \leq 5\}$.

▷ *Why? Set-builder form describes the rule; roster form lists the elements.*

Step 3: (c) Natural numbers less than 6: $\{1, 2, 3, 4, 5\}$.

>>> Python Solution

Example 2.2.1

CODE

```
1 import string
2 vowels = {ch for ch in string.ascii_lowercase if ch in "aeiou"}
3 print("(a)", sorted(vowels))
4 squares = {x**2 for x in range(1, 6)}
5 print("(b) {x^2 : x = 1..5} =", sorted(squares))
6 print("(c)", {x for x in range(1, 6)})
```

OUTPUT

```
(a) ['a', 'e', 'i', 'o', 'u']
(b) {x^2 : x = 1..5} = [1, 4, 9, 16, 25]
(c) {1, 2, 3, 4, 5}
```

Example 2.2.2: Types of sets

Describe each set: (a) $\{x \mid x \in \mathbb{N}, x < 1\}$ (b) $\{0\}$ (c) $\{1, 2, 3\}$ and $\{a, b, c\}$ (d) $\mathbb{N}$.

Solution.

Step 1: (a) No natural number is less than 1, so it is the **empty set** $\emptyset$.

Step 2: (b) It has one element (zero), so it is a **singleton**, not empty.

▷ *Why? $\{0\}$ contains something – the number 0. $\emptyset$ contains nothing.*

Step 3: (c) Both have 3 elements: **equivalent** sets (same number of elements) but not equal.

Step 4: (d) The natural numbers never end: an **infinite** set.

>>> Python Solution

Example 2.2.2

CODE

```
1 a = {x for x in range(1, 10) if x < 1}          # natural numbers less than 1
2 print("(a)", a, " empty?", len(a) == 0)
```

```

3 b = {0}
4 print("(b)", b, " elements:", len(b))
5 A, B = {1, 2, 3}, {"a", "b", "c"}
6 print("(c) equivalent?", len(A) == len(B), " equal?", A == B)

```

OUTPUT

```

(a) set() empty? True
(b) {0} elements: 1
(c) equivalent? True equal? False

```

Example 2.2.3: Subsets and power sets

List all subsets of (a) $\{a, b\}$ (b) $\{1, 2, 3\}$.

Solution.

Step 1: (a) Subsets with 0 elements: $\emptyset$; with 1: $\{a\}, \{b\}$; with 2: $\{a, b\}$. Total $4 = 2^2$.

Step 2: So $P(\{a, b\}) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$.

Step 3: (b) 0 elements: $\emptyset$; 1 element: $\{1\}, \{2\}, \{3\}$; 2 elements: $\{1, 2\}, \{1, 3\}, \{2, 3\}$; 3 elements: $\{1, 2, 3\}$.

Step 4: Total $1 + 3 + 3 + 1 = 8 = 2^3$. ✓

▷ Why? Each element is either in or out of a subset: 2 choices per element.

>>> Python Solution**Example 2.2.3****CODE**

```

1 from itertools import combinations
2
3 def power_set(s):
4     s = sorted(s)
5     return [set(c) for r in range(len(s) + 1) for c in combinations(s, r)]
6
7 for S in [{"a", "b"}, {1, 2, 3}]:
8     P = power_set(S)
9     print(f"Subsets of {sorted(S)} ({len(P)} = 2^{len(S)}):")
10    print(" ", [sorted(p) for p in P])

```

OUTPUT

```

Subsets of ['a', 'b'] (4 = 2^2):
  [], ['a'], ['b'], ['a', 'b']]
Subsets of [1, 2, 3] (8 = 2^3):
  [], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]

```

Example 2.2.4: Set operations

Let $U = \{1, 2, \dots, 10\}$, $A = \{1, 2, 3, 4, 5\}$, $B = \{2, 4, 6, 8, 10\}$. Find $A \cup B$, $A \cap B$, $A - B$, $B - A$, A' , B' .

Solution.

Step 1: Union ("or"): everything in A or B , each written once: $A \cup B = \{1, 2, 3, 4, 5, 6, 8, 10\}$.

Step 2: Intersection ("and"): elements in both: $A \cap B = \{2, 4\}$.

Step 3: Difference: in A but not in B : $A - B = \{1, 3, 5\}$; in B but not in A : $B - A = \{6, 8, 10\}$.

Step 4: Complement: elements of U outside the set: $A' = \{6, 7, 8, 9, 10\}$; $B' = \{1, 3, 5, 7, 9\}$.

>>> Python Solution

Example 2.2.4

CODE

```
1 U = set(range(1, 11))
2 A = {1, 2, 3, 4, 5}
3 B = {2, 4, 6, 8, 10}
4 print("A u B =", A | B)
5 print("A n B =", A & B)
6 print("A - B =", A - B, "   B - A =", sorted(B - A))    # sorted() lists in order
7 print("A'      =", U - A)
8 print("B'      =", U - B)
```

OUTPUT

```
A u B = {1, 2, 3, 4, 5, 6, 8, 10}
A n B = {2, 4}
A - B = {1, 3, 5}   B - A = [6, 8, 10]
A'      = {6, 7, 8, 9, 10}
B'      = {1, 3, 5, 7, 9}
```

Example 2.2.5: Difference as intersection with a complement

Using the sets above, show that $A - B = A \cap B'$.

Solution.

Step 1: $A \cap B' = \{1, 2, 3, 4, 5\} \cap \{1, 3, 5, 7, 9\} = \{1, 3, 5\}$.

Step 2: From the previous example, $A - B = \{1, 3, 5\}$.

Step 3: They are equal. ✓

▷ Why? "In A but not in B" means "in A and in B'".

>>> Python Solution

Example 2.2.5

CODE

```
1 U = set(range(1, 11))
2 A, B = {1, 2, 3, 4, 5}, {2, 4, 6, 8, 10}
3 print("A - B =", A - B)
4 print("A n B' =", A & (U - B))
5 print("Equal?", A - B == A & (U - B))
```

OUTPUT

```
A - B = {1, 3, 5}
A n B' = {1, 3, 5}
Equal? True
```

Example 2.2.6: Verifying De Morgan's law

Using the sets above, verify $(A \cup B)' = A' \cap B'$.

Solution.

Step 1: LHS: $A \cup B = \{1, 2, 3, 4, 5, 6, 8, 10\}$; elements of U left out are 7 and 9, so $(A \cup B)' = \{7, 9\}$.

Step 2: RHS: $A' \cap B' = \{6, 7, 8, 9, 10\} \cap \{1, 3, 5, 7, 9\} = \{7, 9\}$.

Step 3: LHS = RHS. ✓

>>> Python Solution

Example 2.2.6

CODE

```
1 U = set(range(1, 11))
2 A, B = {1, 2, 3, 4, 5}, {2, 4, 6, 8, 10}
3 lhs = U - (A | B)
4 rhs = (U - A) & (U - B)
5 print("(A u B)' =", sorted(lhs))
6 print("A' n B' =", sorted(rhs))
7 print("De Morgan holds:", lhs == rhs)
```

OUTPUT

```
(A u B)' = [7, 9]
A' n B' = [7, 9]
De Morgan holds: True
```

Example 2.2.7: Survey problem with a Venn diagram

Of 60 students, 35 like tea, 30 like coffee and 12 like both. How many like (a) at least one, (b) neither, (c) only tea, (d) only coffee?

Solution.

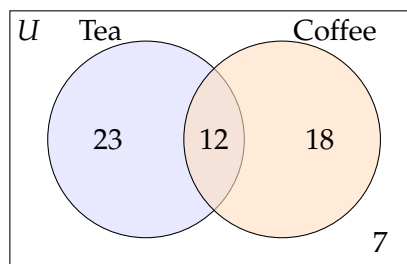
Step 1: Always fill the overlap first: 12 like both.

Step 2: Only tea = $35 - 12 = 23$; only coffee = $30 - 12 = 18$.

▷ Why? The 35 tea-lovers include the 12 who like both.

Step 3: At least one = $23 + 12 + 18 = 53$. (Formula: $35 + 30 - 12 = 53$.)

Step 4: Neither = $60 - 53 = 7$.



Check: $23 + 12 + 18 + 7 = 60$. ✓

>>> Python Solution

Example 2.2.7

CODE

```
1 total, tea, coffee, both = 60, 35, 30, 12
2 only_tea = tea - both
3 only_coffee = coffee - both
4 at_least_one = tea + coffee - both
5 neither = total - at_least_one
6 print("Only tea:", only_tea, " Only coffee:", only_coffee)
7 print("At least one:", at_least_one, " Neither:", neither)
8 print("Check:", only_tea + both + only_coffee + neither)
```

OUTPUT

```
Only tea: 23  Only coffee: 18
At least one: 53  Neither: 7
Check: 60
```

Example 2.2.8: Three sets

In a group of 100 students, 45 study Maths, 40 Physics and 35 Chemistry; 15 study Maths and Physics, 10 Maths and Chemistry, 12 Physics and Chemistry, and 5 study all three. How many study (a) at least one subject, (b) none, (c) only Maths?

Solution.

Step 1: For three sets: $n(M \cup P \cup C) = n(M) + n(P) + n(C) - n(M \cap P) - n(M \cap C) - n(P \cap C) + n(M \cap P \cap C)$.

Step 2: $= 45 + 40 + 35 - 15 - 10 - 12 + 5 = 88$.

▷ Why? Pairs are subtracted to remove double counting; the centre is then added back once.

Step 3: None $= 100 - 88 = 12$.

Step 4: Only Maths $= 45 - 15 - 10 + 5 = 25$ (remove both overlaps, add back the centre removed twice).

>>> Python Solution**Example 2.2.8****CODE**

```
1 total = 100
2 M, P, C = 45, 40, 35
3 MP, MC, PC, MPC = 15, 10, 12, 5
4 union = M + P + C - MP - MC - PC + MPC
5 print("At least one subject:", union)
6 print("None           :", total - union)
7 print("Only Maths      :", M - MP - MC + MPC)
```

OUTPUT

```
At least one subject: 88
None                : 12
Only Maths          : 25
```

Exercise 2.2 (solve by hand, then check with Python)

1. Let $U = \{1, 2, \dots, 12\}$, $P = \{1, 2, 3, 4, 6, 12\}$, $Q = \{2, 4, 6, 8, 10, 12\}$. Find $P \cup Q$, $P \cap Q$, $P - Q$ and Q' .
2. Write $\{x \mid x \text{ is an even natural number less than } 12\}$ in roster form.
3. How many subsets does a set with 4 elements have?
4. In a group of 100 people, 60 read newspaper A, 45 read newspaper B and 20 read both. How many read neither?
5. Verify $(A \cap B)' = A' \cup B'$ for $U = \{1, 2, 3, 4, 5\}$, $A = \{1, 2, 3\}$, $B = \{2, 3, 4\}$.

Multiple Choice Questions – 2.2

- If $A = \{1, 2, 3\}$ and $B = \{3, 4\}$, then $A \cap B =$
 - $\{1, 2, 3, 4\}$
 - $\{3\}$
 - $\{1, 2\}$
 - $\emptyset$
- The number of subsets of $\{a, b, c\}$ is:
 - 3
 - 6
 - 8
 - 9
- $(A \cup B)'$ equals:
 - $A' \cup B'$
 - $A' \cap B'$
 - $A \cap B$
 - $A - B$
- If $n(A) = 10$, $n(B) = 8$, $n(A \cap B) = 3$, then $n(A \cup B) =$
 - 18
 - 15
 - 21
 - 13
- The set of even prime numbers is a/an:
 - empty set
 - singleton set
 - infinite set
 - universal set

Summary of 2.2

- Union = “or”, intersection = “and”, complement = “not”.
- $n(A \cup B) = n(A) + n(B) - n(A \cap B)$ – subtract the overlap so it is not counted twice.
- A set of n elements has 2^n subsets.
- Venn diagrams: fill the overlap first, then the “only” regions, then the outside.

2.3 Relations, Functions and their Graphs

Key Concepts

- Ordered pair** (a, b) – order matters: $(2, 3) \neq (3, 2)$.
- Cartesian product** $A \times B = \{(a, b) \mid a \in A, b \in B\}$; $n(A \times B) = n(A) \cdot n(B)$.
- A **relation** from A to B is any subset of $A \times B$. **Domain** = set of first elements; **range** = set of second elements.
- A **function** is a relation in which *each input has exactly one output*. We write $y = f(x)$.
- Vertical line test**: a graph is a function if no vertical line cuts it more than once.
- Linear function** $f(x) = mx + c$: graph is a straight line with slope m and y -intercept c .
Slope $m = \frac{y_2 - y_1}{x_2 - x_1}$.
- Quadratic function** $f(x) = ax^2 + bx + c$: graph is a **parabola**; opens up if $a > 0$, down if $a < 0$. Vertex at $x = -\frac{b}{2a}$.

Example 2.3.1: Cartesian product

$A = \{1, 2\}$, $B = \{a, b, c\}$. Find $A \times B$.

Solution.

Step 1: Pair every element of A (first) with every element of B (second).

Step 2: With 1: $(1, a), (1, b), (1, c)$. With 2: $(2, a), (2, b), (2, c)$.

Step 3: $A \times B = \{(1, a), (1, b), (1, c), (2, a), (2, b), (2, c)\}$; $n(A \times B) = 2 \times 3 = 6$.

>>> Python Solution**Example 2.3.1****CODE**

```
1 from itertools import product
2 A = [1, 2]
3 B = ["a", "b", "c"]
4 AxB = list(product(A, B))
5 print(AxB)
6 print("n(A x B) =", len(AxB))
```

OUTPUT

```
[(1, 'a'), (1, 'b'), (1, 'c'), (2, 'a'), (2, 'b'), (2, 'c')]
n(A x B) = 6
```

Example 2.3.2: A relation given by a rule

$A = \{1, 2, 3\}$, $B = \{2, 3, 4, 5, 6\}$. Find the relation $R = \{(a, b) \mid b = 2a\}$, its domain and range.

Solution.

Step 1: Apply the rule to each a : $a = 1 \rightarrow b = 2$; $a = 2 \rightarrow b = 4$; $a = 3 \rightarrow b = 6$. All these b 's are in B .

Step 2: $R = \{(1, 2), (2, 4), (3, 6)\}$.

Step 3: Domain (first elements) = $\{1, 2, 3\}$; range (second elements) = $\{2, 4, 6\}$.

>>> Python Solution**Example 2.3.2****CODE**

```
1 A = {1, 2, 3}
2 B = {2, 3, 4, 5, 6}
3 R = {(a, 2*a) for a in A if 2*a in B}
4 print("R =", sorted(R))
5 print("Domain =", {a for a, b in R})
6 print("Range =", {b for a, b in R})
```

OUTPUT

```
R = [(1, 2), (2, 4), (3, 6)]
Domain = {1, 2, 3}
Range = {2, 4, 6}
```

Example 2.3.3: Function or not?

Which relation is a function? $R_1 = \{(1, 2), (2, 3), (3, 4)\}$, $R_2 = \{(1, 2), (1, 3), (2, 4)\}$.

Solution.

Step 1: Look at the first elements (inputs). In a function, no input may appear with two different outputs.

Step 2: R_1 : inputs 1, 2, 3 each appear once $\Rightarrow$ **function**. Domain $\{1, 2, 3\}$, range $\{2, 3, 4\}$.

Step 3: R_2 : input 1 gives 2 and 3 $\Rightarrow$ **not a function**.

>>> Python Solution

Example 2.3.3

CODE

```
1 def is_function(relation):
2     inputs = [a for a, b in relation]
3     return len(inputs) == len(set(inputs))    # no input repeated
4
5 R1 = {(1, 2), (2, 3), (3, 4)}
6 R2 = {(1, 2), (1, 3), (2, 4)}
7 print("R1 is a function:", is_function(R1))
8 print("R2 is a function:", is_function(R2))
```

OUTPUT

```
R1 is a function: True
R2 is a function: False
```

Example 2.3.4: Vertical line test

Is the graph of (a) $y = x^2$ (b) the circle $x^2 + y^2 = 4$ a function?

Solution.

Step 1: (a) Any vertical line $x = k$ meets the parabola once (at $y = k^2$) $\Rightarrow$ **function**.

Step 2: (b) The vertical line $x = 0$ meets the circle at $y = 2$ and $y = -2 \Rightarrow$ **not a function**.

▷ Why? Two points on one vertical line means one input with two outputs.

>>> Python Solution

Example 2.3.4

CODE

```
1 from sympy import symbols, solve
2 x, y = symbols("x y")
3 for name, curve in [("y = x^2", y - x**2), ("x^2 + y^2 = 4", x**2 + y**2 - 4)]:
4     ys = solve(curve.subs(x, 0), y)    # vertical line x = 0
5     verdict = "function" if len(ys) <= 1 else "NOT a function"
6     print(f"{name:<14} meets x = 0 at y = {ys} -> {verdict}")
```

OUTPUT

```
y = x^2      meets x = 0 at y = [0] -> function
x^2 + y^2 = 4 meets x = 0 at y = [-2, 2] -> NOT a function
```

Example 2.3.5: Evaluating a function

If $f(x) = 3x - 2$, find $f(0)$, $f(2)$, $f(-1)$, and the value of x for which $f(x) = 10$.

Solution.

Step 1: Replace x by the given number: $f(0) = 3(0) - 2 = -2$; $f(2) = 3(2) - 2 = 4$;
 $f(-1) = 3(-1) - 2 = -5$.

▷ Why? Use brackets when substituting negatives, to keep signs correct.

Step 2: For $f(x) = 10$: solve $3x - 2 = 10 \Rightarrow 3x = 12 \Rightarrow x = 4$.

>>> Python Solution

Example 2.3.5

CODE

```
1 from sympy import symbols, Eq, solve
2 f = lambda x: 3*x - 2
3 for v in [0, 2, -1]:
4     print(f"f({v}) = {f(v)}")
5 x = symbols("x")
6 print(f"f(x) = 10 when x =", solve(Eq(3*x - 2, 10), x))
```

OUTPUT

```
f(0) = -2
f(2) = 4
f(-1) = -5
f(x) = 10 when x = [4]
```

Example 2.3.6: Evaluating a quadratic function

If $g(x) = x^2 - 3x + 2$, find $g(0)$, $g(1)$ and $g(-2)$.

Solution.

Step 1: $g(0) = 0 - 0 + 2 = 2$.

Step 2: $g(1) = 1 - 3 + 2 = 0$ (so $x = 1$ is a root).

Step 3: $g(-2) = (-2)^2 - 3(-2) + 2 = 4 + 6 + 2 = 12$.

▷ Why? $(-2)^2 = +4$, and $-3 \times (-2) = +6$.

>>> Python Solution

Example 2.3.6

CODE

```
1 g = lambda x: x**2 - 3*x + 2
2 for v in [0, 1, -2]:
3     print(f"g({v}) = {g(v)}")
```

OUTPUT

```
g(0) = 2
g(1) = 0
g(-2) = 12
```

Example 2.3.7: Slope and equation of a line

Find the slope of the line through $(1, 3)$ and $(3, 7)$ and write its equation.

Solution.

Step 1: Slope $m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{7 - 3}{3 - 1} = \frac{4}{2} = 2$.

▷ Why? Slope = rise ÷ run: how much y changes when x increases by 1.

Step 2: Point-slope form: $y - y_1 = m(x - x_1) \Rightarrow y - 3 = 2(x - 1)$.

Step 3: Simplify: $y = 2x - 2 + 3 = 2x + 1$. Check with $(3, 7)$: $2(3) + 1 = 7$. ✓

>>> Python Solution

Example 2.3.7

CODE

```

1 from sympy import symbols, Rational, expand
2 x1, y1, x2, y2 = 1, 3, 3, 7
3 m = Rational(y2 - y1, x2 - x1)
4 x = symbols("x")
5 line = expand(y1 + m * (x - x1))
6 print("Slope m =", m)
7 print("Equation: y =", line)
8 print("Check at x = 3:", line.subs(x, 3))

```

OUTPUT

```

Slope m = 2
Equation: y = 2*x + 1
Check at x = 3: 7

```

Example 2.3.8: Graph of a linear function

Draw the graph of $y = 2x + 1$.

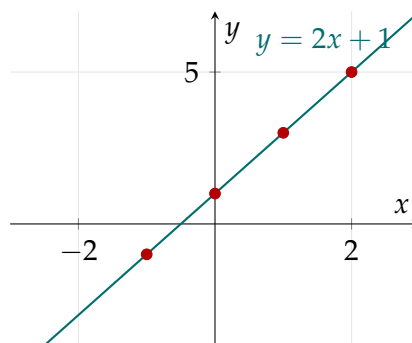
Solution.

Step 1: Make a table:

x	-1	0	1	2
y	-1	1	3	5

Step 2: Plot the points and join them with a straight line.

Step 3: Read the features: slope $m = 2$ (up 2 for each 1 to the right), y -intercept = 1.



>>> Python Solution

Example 2.3.8

CODE

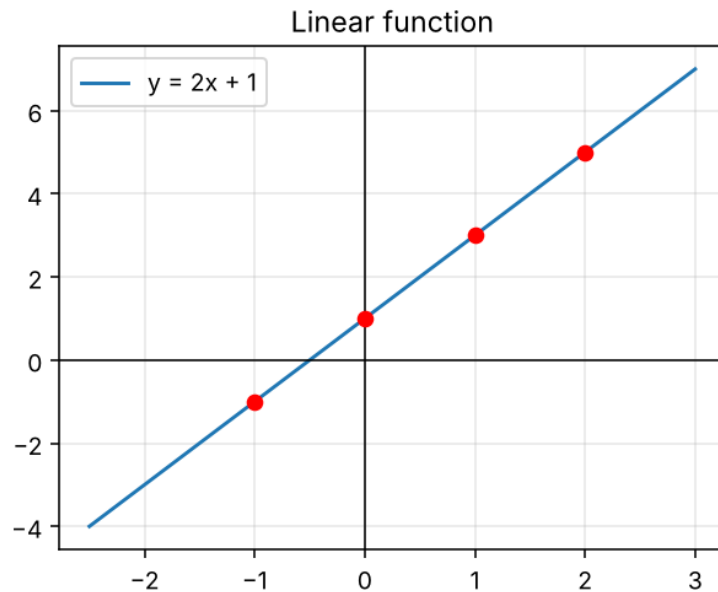
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 xs = [-1, 0, 1, 2]
4 print("Table:", [(x, 2*x + 1) for x in xs])
5 x = np.linspace(-2.5, 3, 50)
6 plt.figure(figsize=(5, 3.8))
7 plt.plot(x, 2*x + 1, label="y = 2x + 1")
8 plt.scatter(xs, [2*v + 1 for v in xs], color="red", zorder=3)
9 plt.axhline(0, color="black", lw=0.8); plt.axvline(0, color="black", lw=0.8)
10 plt.grid(alpha=0.3); plt.legend(); plt.title("Linear function")
11 plt.show()

```

OUTPUT

```
Table: [(-1, -1), (0, 1), (1, 3), (2, 5)]
```



Example 2.3.9: Graph of a quadratic function

Draw the graph of $f(x) = x^2 - 4$ and find its vertex and x -intercepts.

Solution.

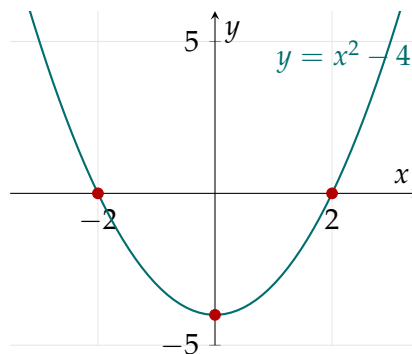
Step 1: Table:

x	-3	-2	-1	0	1	2	3
y	5	0	-3	-4	-3	0	5

Step 2: The values are symmetric about $x = 0$, so the vertex is at $x = -\frac{b}{2a} = 0$: vertex $(0, -4)$.

Step 3: $a = 1 > 0$, so the parabola opens upward and the vertex is the lowest point.

Step 4: x -intercepts: $x^2 - 4 = 0 \Rightarrow x^2 = 4 \Rightarrow x = \pm 2$.



>>> Python Solution

Example 2.3.9

CODE

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 f = lambda x: x**2 - 4
4 print("Table:", [(x, f(x)) for x in range(-3, 4)])
5 x = np.linspace(-3.2, 3.2, 100)
6 plt.figure(figsize=(5, 3.8))
7 plt.plot(x, f(x), label="y = x^2 - 4")
```

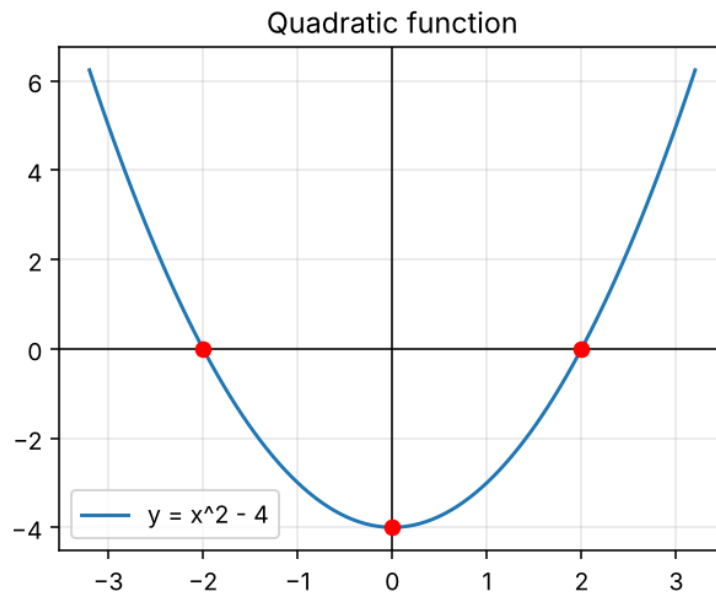
```

8 plt.scatter([-2, 2, 0], [0, 0, -4], color="red", zorder=3)
9 plt.axhline(0, color="black", lw=0.8); plt.axvline(0, color="black", lw=0.8)
10 plt.grid(alpha=0.3); plt.legend(); plt.title("Quadratic function")
11 plt.show()

```

OUTPUT

Table: $[(-3, 5), (-2, 0), (-1, -3), (0, -4), (1, -3), (2, 0), (3, 5)]$

**Example 2.3.10: Key features of a parabola**

For $y = x^2 - 4x + 3$ find the vertex, the y -intercept and the x -intercepts.

Solution.

Step 1: $a = 1, b = -4, c = 3$. Vertex $x = -\frac{b}{2a} = -\frac{-4}{2} = 2$; $y = 4 - 8 + 3 = -1$. Vertex $(2, -1)$.

Step 2: y -intercept: put $x = 0$: $y = 3$.

Step 3: x -intercepts: $x^2 - 4x + 3 = 0 \Rightarrow (x - 1)(x - 3) = 0 \Rightarrow x = 1, 3$.

▷ Why? The vertex lies exactly halfway between the roots: $\frac{1+3}{2} = 2$. ✓

>>> Python Solution**Example 2.3.10****CODE**

```

1 from sympy import symbols, solve, Rational
2 x = symbols("x")
3 a, b, c = 1, -4, 3
4 y = a*x**2 + b*x + c
5 xv = Rational(-b, 2*a)
6 print("Vertex      :", (xv, y.subs(x, xv)))
7 print("y-intercept :", y.subs(x, 0))
8 print("x-intercepts:", solve(y, x))

```

OUTPUT

```

Vertex      : (2, -1)
y-intercept : 3

```

x-intercepts: [1, 3]

Example 2.3.11: Domain of a function

Find the domain of (a) $f(x) = \frac{1}{x-2}$ (b) $g(x) = \sqrt{x-1}$.

Solution.

Step 1: (a) Division by zero is not allowed: $x - 2 \neq 0 \Rightarrow x \neq 2$. Domain: all real numbers except 2.

Step 2: (b) The square root of a negative number is not real: $x - 1 \geq 0 \Rightarrow x \geq 1$. Domain: $x \geq 1$.

>>> Python Solution

Example 2.3.11

CODE

```
1 from sympy import symbols, sqrt, S
2 from sympy.calculus.util import continuous_domain
3 x = symbols("x")
4 print("Domain of 1/(x-2) :", continuous_domain(1/(x - 2), x, S.Reals))
5 print("Domain of sqrt(x-1):", continuous_domain(sqrt(x - 1), x, S.Reals))
```

OUTPUT

```
Domain of 1/(x-2) : Union(Interval.open(-oo, 2), Interval.open(2, oo))
Domain of sqrt(x-1): Interval(1, oo)
```

Example 2.3.12: A function in daily life

A taxi charges Rs. 100 plus Rs. 30 per km: $f(d) = 100 + 30d$. Find the fare for 8 km, and the distance for a fare of Rs. 400.

Solution.

Step 1: $f(8) = 100 + 30 \times 8 = 100 + 240 = \text{Rs. } 340$.

Step 2: For Rs. 400: $100 + 30d = 400 \Rightarrow 30d = 300 \Rightarrow d = 10$ km.

Step 3: Meaning: the slope 30 is the *rate* (Rs per km) and 100 is the fixed starting charge.

>>> Python Solution

Example 2.3.12

CODE

```
1 from sympy import symbols, Eq, solve
2 fare = lambda d: 100 + 30*d
3 print("Fare for 8 km = Rs.", fare(8))
4 d = symbols("d")
5 print("Distance for Rs. 400 =", solve(Eq(100 + 30*d, 400), d)[0], "km")
```

OUTPUT

```
Fare for 8 km = Rs. 340
Distance for Rs. 400 = 10 km
```

Exercise 2.3 (solve by hand, then check with Python)

1. If $f(x) = x^2 + 1$, find $f(3)$ and $f(-2)$.
2. Find the domain and range of $\{(1,4), (2,5), (3,6)\}$. Is it a function?
3. Is $\{(2,3), (2,5), (4,6)\}$ a function? Give a reason.
4. Write the slope and y -intercept of $y = -3x + 7$.
5. Find the slope of the line through $(1,2)$ and $(3,8)$.
6. A mobile plan costs $C = 500 + 2m$ rupees for m minutes. Find the cost for 150 minutes.

Multiple Choice Questions – 2.3

1. If A has 3 elements and B has 4, then $A \times B$ has:

(a) 7 elements	(b) 12 elements
(c) 64 elements	(d) 81 elements
2. The slope of $y = 5x - 3$ is:

(a) -3	(b) 5
(c) 3	(d) -5
3. The graph of $y = x^2$ is a:

(a) straight line	(b) circle
(c) parabola	(d) triangle
4. If $f(x) = 2x + 3$, then $f(4) =$

(a) 8	(b) 9
(c) 11	(d) 14
5. Which test checks whether a graph is a function?

(a) horizontal line test	(b) vertical line test
(c) slope test	(d) area test

Summary of 2.3

- A function gives exactly one output for every input.
- $y = mx + c$: straight line, slope m (rate of change), intercept c (starting value).
- $y = ax^2 + bx + c$: parabola with vertex at $x = -\frac{b}{2a}$.
- To draw a graph: make a table, plot points, join smoothly, label.

2.4 Exponents, Factoring and Simplifying Algebraic Expressions**Laws of Exponents (for non-zero bases)**

$$\begin{array}{lll}
 a^m \cdot a^n = a^{m+n} & \frac{a^m}{a^n} = a^{m-n} & (a^m)^n = a^{mn} \\
 (ab)^n = a^n b^n & a^0 = 1 & a^{-n} = \frac{1}{a^n} \\
 a^{1/n} = \sqrt[n]{a} & a^{m/n} = (\sqrt[n]{a})^m & \left(\frac{a}{b}\right)^n = \frac{a^n}{b^n}
 \end{array}$$

Algebraic Expressions and Factoring

- **Like terms** have the same variables with the same powers ($3x^2$ and $-5x^2$); only like terms can be added.
- **Simplifying:** remove brackets (distributive law $a(b + c) = ab + ac$), then collect like terms.
- **Factoring** is the reverse of expanding. Methods:
 1. Common factor: $6x^2 + 9x = 3x(2x + 3)$.
 2. Difference of squares: $a^2 - b^2 = (a - b)(a + b)$.
 3. Perfect squares: $a^2 \pm 2ab + b^2 = (a \pm b)^2$.
 4. Trinomial $x^2 + bx + c$: find two numbers whose *product* is c and *sum* is b .
 5. $ax^2 + bx + c$: split the middle term using two numbers with product ac and sum b .

Example 2.4.1: Product, quotient and power laws

Simplify (a) $2^3 \cdot 2^4$ (b) $(x^3)^2$ (c) $\frac{x^5 y^3}{x^2 y}$ (d) 5^0 .

Solution.

Step 1: (a) Same base, multiplying $\Rightarrow$ add powers: $2^{3+4} = 2^7 = 128$.

▷ Why? $2^3 \cdot 2^4 = (2 \cdot 2 \cdot 2)(2 \cdot 2 \cdot 2 \cdot 2)$: seven 2's.

Step 2: (b) Power of a power $\Rightarrow$ multiply powers: $x^{3 \times 2} = x^6$.

Step 3: (c) Same base, dividing $\Rightarrow$ subtract powers, one letter at a time: $x^{5-2} y^{3-1} = x^3 y^2$.

Step 4: (d) Any non-zero number to the power 0 is 1: $5^0 = 1$.

>>> Python Solution

Example 2.4.1

CODE

```
1 from sympy import symbols, simplify
2 x, y = symbols("x y")
3 print("(a) 2^3 * 2^4 =", 2**3 * 2**4, "= 2^7")
4 print("(b) (x^3)^2 =", (x**3)**2)
5 print("(c) x^5 y^3 / (x^2 y) =", simplify(x**5 * y**3 / (x**2 * y)))
6 print("(d) 5^0 =", 5**0)
```

OUTPUT

```
(a) 2^3 * 2^4 = 128 = 2^7
(b) (x^3)^2 = x**6
(c) x^5 y^3 / (x^2 y) = x**3*y**2
(d) 5^0 = 1
```

Example 2.4.2: Negative and fractional powers

Simplify (a) 2^{-3} (b) $(\frac{2}{3})^{-2}$ (c) $27^{2/3}$ (d) $16^{3/4}$.

Solution.

Step 1: (a) Negative power means reciprocal: $2^{-3} = \frac{1}{2^3} = \frac{1}{8}$.

Step 2: (b) Flip the fraction and make the power positive: $(\frac{3}{2})^2 = \frac{9}{4}$.

Step 3: (c) The denominator 3 means cube root; the numerator 2 means square: $\left(\sqrt[3]{27}\right)^2 = 3^2 = 9$.

▷ Why? Take the root first – the numbers stay small.

Step 4: (d) $\left(\sqrt[4]{16}\right)^3 = 2^3 = 8$.

>>> Python Solution

Example 2.4.2

CODE

```
1 from sympy import Rational, root
2 print("(a) 2^-3      =", Rational(2)**-3)
3 print("(b) (2/3)^-2   =", Rational(2, 3)**-2)
4 print("(c) 27^(2/3)   =", Rational(27)**Rational(2, 3))
5 print("(d) 16^(3/4)   =", Rational(16)**Rational(3, 4))
```

OUTPUT

```
(a) 2^-3      = 1/8
(b) (2/3)^-2   = 9/4
(c) 27^(2/3)   = 9
(d) 16^(3/4)   = 8
```

Example 2.4.3: Combining several laws

Simplify $\frac{(2x^3y^2)^3}{4x^4y}$.

Solution.

Step 1: Expand the bracket: $(2)^3(x^3)^3(y^2)^3 = 8x^9y^6$.

Step 2: Divide numbers: $8 \div 4 = 2$.

Step 3: Subtract powers: $x^{9-4} = x^5$, $y^{6-1} = y^5$.

Step 4: Result: $2x^5y^5$.

>>> Python Solution

Example 2.4.3

CODE

```
1 from sympy import symbols, simplify
2 x, y = symbols("x y")
3 expr = (2*x**3*y**2)**3 / (4*x**4*y)
4 print("Bracket expanded:", (2*x**3*y**2)**3)
5 print("Simplified      :", simplify(expr))
```

OUTPUT

```
Bracket expanded: 8*x**9*y**6
Simplified      : 2*x**5*y**5
```

Example 2.4.4: Simplifying an expression

Simplify $3(2x + 4) - 2(x - 1)$.

Solution.

Step 1: Expand each bracket: $3(2x + 4) = 6x + 12$ and $-2(x - 1) = -2x + 2$.

▷ Why? The -2 multiplies both terms: $-2 \times -1 = +2$. Forgetting this is the most common error.

Step 2: Write together: $6x + 12 - 2x + 2$.

Step 3: Collect like terms: $(6x - 2x) + (12 + 2) = 4x + 14$.

>>> Python Solution

Example 2.4.4

CODE

```
1 from sympy import symbols, expand
2 x = symbols("x")
3 print("3(2x+4) =", expand(3*(2*x + 4)))
4 print("-2(x-1) =", expand(-2*(x - 1)))
5 print("Answer =", expand(3*(2*x + 4) - 2*(x - 1)))
```

OUTPUT

```
3(2x+4) = 6*x + 12
-2(x-1) = 2 - 2*x
Answer = 4*x + 14
```

Example 2.4.5: Expanding products

Expand (a) $(x + 3)(x - 5)$ (b) $(2x - 1)^2$.

Solution.

Step 1: (a) Multiply each term of the first bracket by each term of the second (FOIL): $x \cdot x + x(-5) + 3x + 3(-5) = x^2 - 5x + 3x - 15 = x^2 - 2x - 15$.

Step 2: (b) $(2x - 1)^2 = (2x - 1)(2x - 1) = 4x^2 - 2x - 2x + 1 = 4x^2 - 4x + 1$.

▷ Why? $(a - b)^2 = a^2 - 2ab + b^2$, not $a^2 - b^2$.

>>> Python Solution

Example 2.4.5

CODE

```
1 from sympy import symbols, expand
2 x = symbols("x")
3 print("(a)", expand((x + 3)*(x - 5)))
4 print("(b)", expand((2*x - 1)**2))
```

OUTPUT

```
(a) x**2 - 2*x - 15
(b) 4*x**2 - 4*x + 1
```

Example 2.4.6: Common factor, difference of squares, perfect square

Factorise (a) $6x^2 + 9x$ (b) $x^2 - 25$ (c) $x^2 + 6x + 9$.

Solution.

Step 1: (a) HCF of $6x^2$ and $9x$ is $3x$. Divide each term by $3x$: $2x$ and 3 . So $6x^2 + 9x = 3x(2x + 3)$.

Step 2: (b) Two perfect squares with a minus sign: $x^2 - 5^2 = (x - 5)(x + 5)$.

Step 3: (c) First and last terms are squares ($x^2, 3^2$) and the middle term is $2 \times x \times 3$: $(x + 3)^2$.

>>> Python Solution

Example 2.4.6

CODE

```
1 from sympy import symbols, factor
2 x = symbols("x")
3 for e in [6*x**2 + 9*x, x**2 - 25, x**2 + 6*x + 9]:
4     print(f"{str(e):<14} = {factor(e)}")
```

OUTPUT

```
6*x**2 + 9*x    = 3*x*(2*x + 3)
x**2 - 25       = (x - 5)*(x + 5)
x**2 + 6*x + 9  = (x + 3)**2
```

Example 2.4.7: Factorising trinomials

Factorise (a) $x^2 + 5x + 6$ (b) $x^2 - x - 12$.

Solution.

Step 1: (a) Find two numbers with product +6 and sum +5. List factor pairs of 6: (1, 6), (2, 3). $2 + 3 = 5$. ✓

Step 2: So $x^2 + 5x + 6 = (x + 2)(x + 3)$.

Step 3: (b) Product -12, sum -1. The product is negative, so the signs differ; the larger number is negative: -4 and 3.

Step 4: So $x^2 - x - 12 = (x - 4)(x + 3)$. Check: $x^2 + 3x - 4x - 12 = x^2 - x - 12$. ✓

>>> Python Solution

Example 2.4.7

CODE

```
1 from sympy import symbols, factor
2 x = symbols("x")
3
4 def find_pair(product, total):
5     for p in range(-abs(product), abs(product) + 1):
6         if p != 0 and product % p == 0 and p + product // p == total:
7             return p, product // p
8
9 print("x^2+5x+6 : numbers", find_pair(6, 5), "->", factor(x**2 + 5*x + 6))
10 print("x^2-x-12 : numbers", find_pair(-12, -1), "->", factor(x**2 - x - 12))
```

OUTPUT

```
x^2+5x+6 : numbers (2, 3) -> (x + 2)*(x + 3)
x^2-x-12 : numbers (-4, 3) -> (x - 4)*(x + 3)
```

Example 2.4.8: Splitting the middle term

Factorise $2x^2 + 7x + 3$.

Solution.

Step 1: Multiply the first and last coefficients: $ac = 2 \times 3 = 6$.

Step 2: Find two numbers with product 6 and sum 7: 6 and 1.

Step 3: Split the middle term: $2x^2 + 6x + x + 3$.

Step 4: Group in pairs: $2x(x + 3) + 1(x + 3)$.

Step 5: Take out the common bracket: $(2x + 1)(x + 3)$.

>>> Python Solution

Example 2.4.8

CODE

```
1 from sympy import symbols, factor
2 x = symbols("x")
3 a, b, c = 2, 7, 3
4 print("ac =", a * c, "-> numbers 6 and 1 (6 x 1 = 6, 6 + 1 = 7)")
5 print("Factorised:", factor(a*x**2 + b*x + c))
```

OUTPUT

```
ac = 6 -> numbers 6 and 1 (6 x 1 = 6, 6 + 1 = 7)
Factorised: (x + 3)*(2*x + 1)
```

Example 2.4.9: Splitting with negative terms

Factorise $6x^2 - x - 2$.

Solution.

Step 1: $ac = 6 \times (-2) = -12$. Need product -12 , sum -1 : -4 and 3 .

Step 2: Split: $6x^2 - 4x + 3x - 2$.

Step 3: Group: $2x(3x - 2) + 1(3x - 2)$.

Step 4: Factor: $(2x + 1)(3x - 2)$. Check: $6x^2 - 4x + 3x - 2 = 6x^2 - x - 2$. ✓

>>> Python Solution

Example 2.4.9

CODE

```
1 from sympy import symbols, factor, expand
2 x = symbols("x")
3 f = factor(6*x**2 - x - 2)
4 print("Factorised:", f)
5 print("Check      :", expand(f))
```

OUTPUT

```
Factorised: (2*x + 1)*(3*x - 2)
Check      : 6*x**2 - x - 2
```

Example 2.4.10: Factorising completely

Factorise completely (a) $2x^2 - 18$ (b) $x^4 - 16$.

Solution.

Step 1: (a) Take the common factor first: $2(x^2 - 9)$; then use difference of squares: $2(x - 3)(x + 3)$.

▷ Why? Always look for a common factor first.

Step 2: (b) $x^4 - 16 = (x^2)^2 - 4^2 = (x^2 - 4)(x^2 + 4)$.

Step 3: $x^2 - 4$ factorises again: $(x - 2)(x + 2)(x^2 + 4)$. ($x^2 + 4$ cannot be factorised over real numbers.)

>>> Python Solution

Example 2.4.10

CODE

```
1 from sympy import symbols, factor
2 x = symbols("x")
3 print("(a) 2x^2 - 18 =", factor(2*x**2 - 18))
4 print("(b) x^4 - 16 =", factor(x**4 - 16))
```

OUTPUT

```
(a) 2x^2 - 18 = 2*(x - 3)*(x + 3)
(b) x^4 - 16 = (x - 2)*(x + 2)*(x**2 + 4)
```

Example 2.4.11: Factorising by grouping

Factorise $x^3 + 2x^2 + 3x + 6$.

Solution.

Step 1: Group in pairs: $(x^3 + 2x^2) + (3x + 6)$.

Step 2: Factor each pair: $x^2(x + 2) + 3(x + 2)$.

Step 3: Take out the common bracket: $(x^2 + 3)(x + 2)$.

>>> Python Solution

Example 2.4.11

CODE

```
1 from sympy import symbols, factor
2 x = symbols("x")
3 print("x^3 + 2x^2 + 3x + 6 =", factor(x**3 + 2*x**2 + 3*x + 6))
```

OUTPUT

```
x^3 + 2x^2 + 3x + 6 = (x + 2)*(x**2 + 3)
```

Example 2.4.12: Simplifying algebraic fractions

Simplify (a) $\frac{x^2 - 9}{x + 3}$ (b) $\frac{x^2 + 5x + 6}{x^2 - 9}$.

Solution.

Step 1: Factorise top and bottom completely before cancelling.

Step 2: (a) $\frac{(x - 3)(x + 3)}{x + 3} = x - 3, (x \neq -3)$.

Step 3: (b) $\frac{(x + 2)(x + 3)}{(x - 3)(x + 3)} = \frac{x + 2}{x - 3}, (x \neq \pm 3)$.

▷ Why? Only common factors cancel – never cancel terms joined by + or –.

>>> Python Solution

Example 2.4.12

CODE

```
1 from sympy import symbols, cancel, factor
2 x = symbols("x")
3 print("(a)", cancel((x**2 - 9)/(x + 3)))
4 e = (x**2 + 5*x + 6)/(x**2 - 9)
5 print("(b) factored:", factor(e))
6 print("    simplified:", cancel(e))
```

OUTPUT

(a) $x - 3$
 (b) factored: $(x + 2)/(x - 3)$
 simplified: $(x + 2)/(x - 3)$

Exercise 2.4 (solve by hand, then check with Python)

1. Simplify: (a) $3^2 \cdot 3^3$ (b) $(2a^2)^3$ (c) $16^{1/2}$ (d) 4^{-2} .
2. Simplify $5(x - 2) + 3(2x + 1)$.
3. Factorise: (a) $x^2 - 49$ (b) $4x^2 - 12x$ (c) $x^2 + 7x + 12$ (d) $x^2 - 2x - 15$ (e) $3x^2 + 10x + 8$.
4. Simplify $\frac{x^2 - 4}{x - 2}$.

Multiple Choice Questions – 2.4

1. $a^5 \div a^2 =$
 (a) a^{10} (b) a^7
 (c) a^3 (d) $a^{2.5}$
2. $7^0 =$
 (a) 0 (b) 1
 (c) 7 (d) undefined
3. $x^2 - 16$ factorises as:
 (a) $(x - 4)^2$ (b) $(x - 4)(x + 4)$
 (c) $(x - 8)(x + 2)$ (d) $(x + 4)^2$
4. $x^2 + 8x + 15 =$
 (a) $(x + 3)(x + 5)$ (b) $(x + 1)(x + 15)$
 (c) $(x - 3)(x - 5)$ (d) $(x + 2)(x + 6)$
5. $8^{1/3} =$
 (a) 2 (b) 4
 (c) 24 (d) $\frac{8}{3}$

Summary of 2.4

- Same base: multiply $\rightarrow$ add powers, divide $\rightarrow$ subtract powers, power of power $\rightarrow$ multiply.
- $a^0 = 1$, $a^{-n} = 1/a^n$, $a^{1/n} = \sqrt[n]{a}$.
- Factor in this order: common factor $\rightarrow$ special products $\rightarrow$ trinomial / splitting the middle term.
- Always check a factorisation by expanding it again.

2.5 Linear and Quadratic Equations and Inequalities (Algebraic and Graphical)

Key Concepts

- **Linear equation** $ax + b = 0$: do the same operation on both sides to isolate x .
- **System of two linear equations**: solve by *substitution* or *elimination*; graphically, the solution is the **intersection point** of the two lines.
- **Quadratic equation** $ax^2 + bx + c = 0$ ($a \neq 0$): solve by factoring or by the **quadratic formula**

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Discriminant $D = b^2 - 4ac$: $D > 0$ two real roots; $D = 0$ one repeated root; $D < 0$ no real roots.

Graphically, the roots are where the parabola $y = ax^2 + bx + c$ cuts the x -axis.

- **Inequalities** ($<, \leq, >, \geq$) are solved like equations, **except**: multiplying or dividing by a *negative* number **reverses** the inequality sign.
- **Quadratic inequality**: find the roots, then use the graph – for $a > 0$ the parabola is *below* the axis between the roots and *above* it outside the roots.

Example 2.5.1: Linear equations

Solve (a) $3x + 5 = 20$ (b) $\frac{x}{2} + 3 = 7$.

Solution.

Step 1: (a) Subtract 5 from both sides: $3x = 15$. Divide both sides by 3: $x = 5$.

▷ Why? *Doing the same thing to both sides keeps the equation balanced.*

Step 2: Check: $3(5) + 5 = 20$. ✓

Step 3: (b) Subtract 3: $\frac{x}{2} = 4$. Multiply by 2: $x = 8$.

>>> Python Solution

Example 2.5.1

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 print("(a) x =", solve(Eq(3*x + 5, 20), x)[0])
4 print("(b) x =", solve(Eq(x/2 + 3, 7), x)[0])
5 print("Check (a):", 3*5 + 5 == 20)
```

OUTPUT

```
(a) x = 5
(b) x = 8
Check (a): True
```

Example 2.5.2: Brackets on both sides

Solve $3(x - 2) = 2(x + 4)$.

Solution.

Step 1: Expand: $3x - 6 = 2x + 8$.

Step 2: Bring x -terms to the left: $3x - 2x - 6 = 8 \Rightarrow x - 6 = 8$.

Step 3: Add 6: $x = 14$. Check: $3(12) = 36$ and $2(18) = 36$. ✓

>>> Python Solution

Example 2.5.2

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 sol = solve(Eq(3*(x - 2), 2*(x + 4)), x)[0]
4 print("x =", sol)
5 print("Check: LHS =", 3*(sol - 2), " RHS =", 2*(sol + 4))
```

OUTPUT

```
x = 14
Check: LHS = 36  RHS = 36
```

Example 2.5.3: Equations with fractions

Solve $\frac{x+1}{3} - \frac{x-1}{4} = 1$.

Solution.

Step 1: Clear fractions by multiplying every term by the LCM 12: $4(x+1) - 3(x-1) = 12$.

Step 2: Expand carefully: $4x + 4 - 3x + 3 = 12$.

Step 3: Simplify: $x + 7 = 12 \Rightarrow x = 5$.

Step 4: Check: $\frac{6}{3} - \frac{4}{4} = 2 - 1 = 1$. ✓

>>> Python Solution

Example 2.5.3

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 sol = solve(Eq((x + 1)/3 - (x - 1)/4, 1), x)[0]
4 print("x =", sol)
5 print("Check:", (sol + 1)/3 - (sol - 1)/4)
```

OUTPUT

```
x = 5
Check: 1
```

Example 2.5.4: Word problem

The sum of two consecutive integers is 41. Find them.

Solution.

Step 1: Let the smaller integer be x ; the next one is $x + 1$.

Step 2: Translate: $x + (x + 1) = 41$.

Step 3: Solve: $2x + 1 = 41 \Rightarrow 2x = 40 \Rightarrow x = 20$.

Step 4: The numbers are 20 and 21 (check: $20 + 21 = 41$).

>>> Python Solution

Example 2.5.4

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 n = solve(Eq(x + (x + 1), 41), x)[0]
4 print("The integers are", n, "and", n + 1)
```

OUTPUT

```
The integers are 20 and 21
```

Example 2.5.5: Age problem

A father is three times as old as his son. In 10 years he will be twice as old as his son. Find their present ages.

Solution.

Step 1: Let the son's age be s ; father = $3s$.

Step 2: In 10 years: son $s + 10$, father $3s + 10$. Translate: $3s + 10 = 2(s + 10)$.

Step 3: Solve: $3s + 10 = 2s + 20 \Rightarrow s = 10$.

Step 4: Son 10 years, father 30 years. Check: in 10 years $40 = 2 \times 20$. ✓

>>> Python Solution

Example 2.5.5

CODE

```
1 from sympy import symbols, Eq, solve
2 s = symbols("s")
3 son = solve(Eq(3*s + 10, 2*(s + 10)), s)[0]
4 print("Son:", son, "years   Father:", 3*son, "years")
5 print("In 10 years:", 3*son + 10, "=", 2, "x", son + 10)
```

OUTPUT

```
Son: 10 years   Father: 30 years
In 10 years: 40 = 2 x 20
```

Example 2.5.6: System of equations – elimination and graph

Solve $x + y = 10$ and $x - y = 2$.

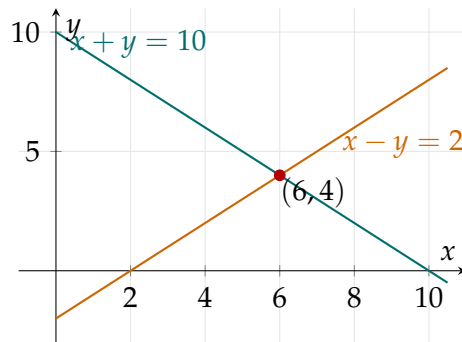
Solution.

Step 1: *Elimination:* the y -terms have opposite signs, so add the equations: $2x = 12 \Rightarrow x = 6$.

▷ *Why? Adding removes y , leaving one equation in one unknown.*

Step 2: Substitute in $x + y = 10$: $6 + y = 10 \Rightarrow y = 4$.

Step 3: *Graphically:* the lines $y = 10 - x$ and $y = x - 2$ meet at $(6, 4)$.



>>> Python Solution

Example 2.5.6

CODE

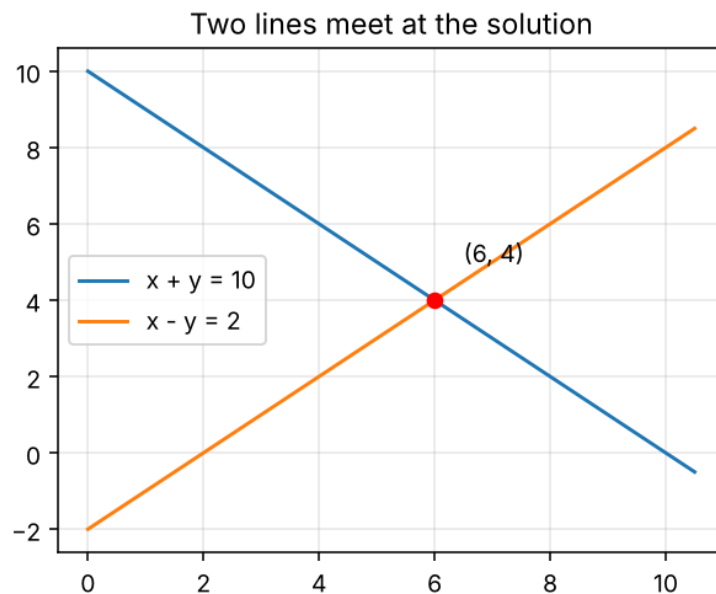
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sympy import symbols, Eq, solve
4 x, y = symbols("x y")
5 print(solve([Eq(x + y, 10), Eq(x - y, 2)], [x, y]))
6 xs = np.linspace(0, 10.5, 50)
7 plt.figure(figsize=(5, 3.8))
8 plt.plot(xs, 10 - xs, label="x + y = 10")
9 plt.plot(xs, xs - 2, label="x - y = 2")
10 plt.scatter([6], [4], color="red", zorder=3)
11 plt.annotate("(6, 4)", (6, 4), xytext=(6.5, 5))
12 plt.grid(alpha=0.3); plt.legend(); plt.title("Two lines meet at the solution")
13 plt.show()

```

OUTPUT

```
{x: 6, y: 4}
```



Example 2.5.7: System by substitution

Solve $y = 2x - 1$ and $3x + 2y = 12$.

Solution.

Step 1: y is already alone in the first equation, so substitute it into the second: $3x + 2(2x - 1) = 12$.

Step 2: Expand and solve: $3x + 4x - 2 = 12 \Rightarrow 7x = 14 \Rightarrow x = 2$.

Step 3: Back-substitute: $y = 2(2) - 1 = 3$. Solution $(2, 3)$.

Step 4: Check in the second equation: $3(2) + 2(3) = 12$. ✓

>>> Python Solution

Example 2.5.7

CODE

```
1 from sympy import symbols, Eq, solve
2 x, y = symbols("x y")
3 print(solve([Eq(y, 2*x - 1), Eq(3*x + 2*y, 12)], [x, y]))
```

OUTPUT

```
{x: 2, y: 3}
```

Example 2.5.8: Word problem with two unknowns

2 pens and 3 notebooks cost Rs.210; 1 pen and 2 notebooks cost Rs.130. Find the price of each.

Solution.

Step 1: Let a pen cost p and a notebook n : $2p + 3n = 210$ and $p + 2n = 130$.

Step 2: From the second: $p = 130 - 2n$. Substitute: $2(130 - 2n) + 3n = 210$.

Step 3: $260 - 4n + 3n = 210 \Rightarrow -n = -50 \Rightarrow n = 50$.

Step 4: $p = 130 - 100 = 30$. A pen costs Rs.30 and a notebook Rs.50.

>>> Python Solution

Example 2.5.8

CODE

```
1 from sympy import symbols, Eq, solve
2 p, n = symbols("p n")
3 sol = solve([Eq(2*p + 3*n, 210), Eq(p + 2*n, 130)], [p, n])
4 print("Pen = Rs.", sol[p], " Notebook = Rs.", sol[n])
```

OUTPUT

```
Pen = Rs. 30 Notebook = Rs. 50
```

Example 2.5.9: Quadratic equation by factoring and graph

Solve $x^2 - 5x + 6 = 0$.

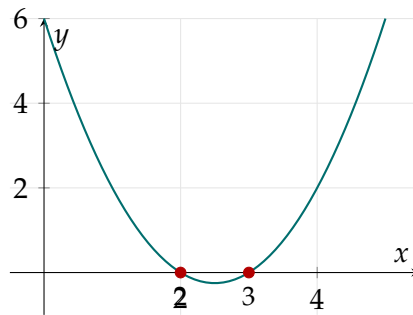
Solution.

Step 1: Factor: two numbers with product +6 and sum -5: -2 and -3. $(x - 2)(x - 3) = 0$.

Step 2: If a product is zero, one factor must be zero: $x - 2 = 0$ or $x - 3 = 0$.

▷ Why? This is the zero-product rule.

Step 3: So $x = 2$ or $x = 3$. Graphically, $y = x^2 - 5x + 6$ cuts the x -axis at 2 and 3.



>>> Python Solution

Example 2.5.9

CODE

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sympy import symbols, factor, solve
4 x = symbols("x")
5 print("Factorised:", factor(x**2 - 5*x + 6))
6 print("Roots      :", solve(x**2 - 5*x + 6, x))
7 xs = np.linspace(0, 5, 100)
8 plt.figure(figsize=(5, 3.5))
9 plt.plot(xs, xs**2 - 5*xs + 6)
10 plt.scatter([2, 3], [0, 0], color="red", zorder=3)
11 plt.axhline(0, color="black", lw=0.8)
12 plt.grid(alpha=0.3); plt.title("y = x^2 - 5x + 6 cuts the x-axis at 2 and 3")
13 plt.show()

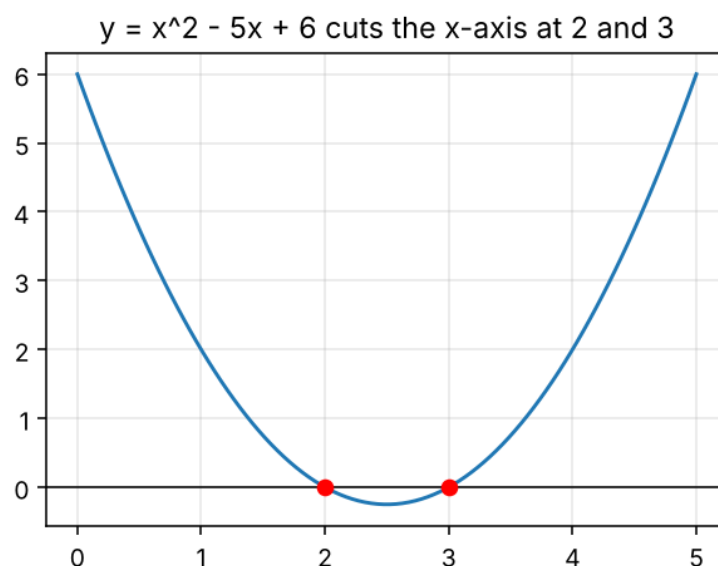
```

OUTPUT

```

Factorised: (x - 3)*(x - 2)
Roots      : [2, 3]

```



Example 2.5.10: Quadratic formula (rational roots)

Solve $2x^2 + 3x - 2 = 0$.

Solution.

Step 1: Identify $a = 2, b = 3, c = -2$.

Step 2: Discriminant: $D = b^2 - 4ac = 9 - 4(2)(-2) = 9 + 16 = 25 > 0 \Rightarrow$ two real roots.

Step 3: $x = \frac{-b \pm \sqrt{D}}{2a} = \frac{-3 \pm 5}{4}$.

Step 4: $x = \frac{-3+5}{4} = \frac{1}{2}$ or $x = \frac{-3-5}{4} = -2$.

>>> Python Solution

Example 2.5.10

CODE

```
1 import math
2 a, b, c = 2, 3, -2
3 D = b**2 - 4*a*c
4 x1 = (-b + math.sqrt(D)) / (2*a)
5 x2 = (-b - math.sqrt(D)) / (2*a)
6 print("D =", D)
7 print("x =", x1, "or", x2)
```

OUTPUT

```
D = 25
x = 0.5 or -2.0
```

Example 2.5.11: Quadratic formula (irrational roots)

Solve $x^2 + 2x - 4 = 0$.

Solution.

Step 1: $a = 1, b = 2, c = -4$; $D = 4 - 4(1)(-4) = 4 + 16 = 20$.

Step 2: $x = \frac{-2 \pm \sqrt{20}}{2}$. Simplify $\sqrt{20} = \sqrt{4 \times 5} = 2\sqrt{5}$.

Step 3: $x = \frac{-2 \pm 2\sqrt{5}}{2} = -1 \pm \sqrt{5}$, i.e. $x \approx 1.236$ or $x \approx -3.236$.

▷ Why? $D = 20$ is not a perfect square, so the roots are irrational and the equation does not factorise nicely.

>>> Python Solution

Example 2.5.11

CODE

```
1 from sympy import symbols, solve, N
2 x = symbols("x")
3 roots = solve(x**2 + 2*x - 4, x)
4 print("Discriminant:", 2**2 - 4*1*(-4))
5 print("Exact roots :", roots)
6 print("Decimals      :", [round(float(N(r)), 3) for r in roots])
```

OUTPUT

```
Discriminant: 20
Exact roots : [-1 + sqrt(5), -sqrt(5) - 1]
Decimals      : [1.236, -3.236]
```

Example 2.5.12: Nature of roots

Without solving, describe the roots of (a) $x^2 - 6x + 9 = 0$ (b) $x^2 + x + 2 = 0$.

Solution.

Step 1: (a) $D = 36 - 36 = 0$: one repeated root, $x = -\frac{b}{2a} = 3$. (Indeed $(x - 3)^2 = 0$.)

Step 2: (b) $D = 1 - 8 = -7 < 0$: no real roots – the parabola does not touch the x -axis.

>>> Python Solution**Example 2.5.12****CODE**

```
1 def nature(a, b, c):
2     D = b**2 - 4*a*c
3     if D > 0: return D, "two real roots"
4     if D == 0: return D, "one repeated root"
5     return D, "no real roots"
6
7 print("x^2 - 6x + 9 :", nature(1, -6, 9))
8 print("x^2 + x + 2 :", nature(1, 1, 2))
```

OUTPUT

```
x^2 - 6x + 9 : (0, 'one repeated root')
x^2 + x + 2 : (-7, 'no real roots')
```

Example 2.5.13: Linear inequalities

Solve (a) $2x - 3 < 7$ (b) $-3x \leq 12$ (c) $-1 < 2x + 1 \leq 7$.

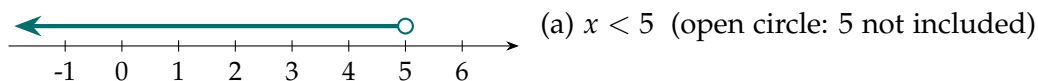
Solution.

Step 1: (a) Add 3: $2x < 10$; divide by 2: $x < 5$.

Step 2: (b) Divide by -3 and **reverse** the sign: $x \geq -4$.

▷ Why? Example: $2 < 3$, but $-2 > -3$. Multiplying by a negative flips the order.

Step 3: (c) Do the same to all three parts: subtract 1: $-2 < 2x \leq 6$; divide by 2: $-1 < x \leq 3$.

**>>> Python Solution****Example 2.5.13****CODE**

```
1 from sympy import symbols, solve_univariate_inequality
2 x = symbols("x", real=True)
3 print("(a)", solve_univariate_inequality(2*x - 3 < 7, x))
4 print("(b)", solve_univariate_inequality(-3*x <= 12, x))
5 print("(c)", solve_univariate_inequality(2*x + 1 > -1, x), "and",
6     solve_univariate_inequality(2*x + 1 <= 7, x))
```

OUTPUT

```
(a) x < 5
(b) -4 <= x
(c) -1 < x and x <= 3
```

Example 2.5.14: Inequality word problem

A student needs an average of at least 60 in three tests. He scored 52 and 58 in the first two. What must he score in the third?

Solution.

Step 1: Let the third score be x . "At least" means $\geq$: $\frac{52 + 58 + x}{3} \geq 60$.

Step 2: Multiply by 3: $110 + x \geq 180$.

Step 3: Subtract 110: $x \geq 70$. He needs at least 70.

>>> Python Solution**Example 2.5.14****CODE**

```
1 from sympy import symbols, solve_univariate_inequality
2 x = symbols("x", real=True)
3 print(solve_univariate_inequality((52 + 58 + x)/3 >= 60, x))
4 print("Minimum score needed:", 3*60 - (52 + 58))
```

OUTPUT

```
70 <= x
Minimum score needed: 70
```

Example 2.5.15: Quadratic inequalities

Solve (a) $x^2 - 5x + 6 < 0$ (b) $x^2 - 4 \geq 0$.

Solution.

Step 1: (a) Find the roots of $x^2 - 5x + 6 = 0$: $x = 2, 3$. They split the number line into three parts.

Step 2: Test a point in each part: $x = 0$: $6 > 0$; $x = 2.5$: $6.25 - 12.5 + 6 = -0.25 < 0$; $x = 4$: $2 > 0$.

Step 3: Only the middle part is negative: $2 < x < 3$.

Step 4: (b) Roots ± 2 . Test $x = 0$: $-4 < 0$ (fails); test $x = 3$: $5 \geq 0$ (works). Solution: $x \leq -2$ or $x \geq 2$.

>>> Python Solution**Example 2.5.15****CODE**

```
1 from sympy import symbols, solve_univariate_inequality
2 f = lambda t: t**2 - 5*t + 6
3 for t in [0, 2.5, 4]:
4     print(f"f({t}) = {f(t)}", "negative" if f(t) < 0 else "positive")
5 x = symbols("x", real=True)
6 print("(a)", solve_univariate_inequality(x**2 - 5*x + 6 < 0, x))
7 print("(b)", solve_univariate_inequality(x**2 - 4 >= 0, x))
```

OUTPUT

```
f(0) = 6 positive
f(2.5) = -0.25 negative
f(4) = 2 positive
(a) (2 < x) & (x < 3)
(b) (2 <= x) | (x <= -2)
```

Exercise 2.5 (solve by hand, then check with Python)

1. Solve: (a) $5x - 7 = 18$ (b) $2(x + 3) = x + 10$.
2. Solve the system $2x + y = 7$, $x - y = 2$.
3. Solve by factoring: (a) $x^2 - 7x + 12 = 0$ (b) $x^2 + 2x - 8 = 0$.
4. Solve using the quadratic formula: $x^2 - 4x + 1 = 0$.
5. Solve: (a) $3x + 2 > 11$ (b) $-2x + 5 \geq 1$.
6. Solve $x^2 - 9 < 0$.
7. Without solving, state the nature of roots of $x^2 + x + 1 = 0$.

Multiple Choice Questions – 2.5

1. The solution of $4x - 8 = 0$ is:
 (a) -2 (b) 2
 (c) 4 (d) 8
2. If $-2x > 6$, then:
 (a) $x > -3$ (b) $x < -3$
 (c) $x > 3$ (d) $x < 3$
3. The roots of $x^2 - 9 = 0$ are:
 (a) 3 only (b) ± 3
 (c) 9 (d) ± 9
4. If $b^2 - 4ac < 0$, the quadratic equation has:
 (a) two real roots (b) one real root
 (c) no real roots (d) three roots
5. Graphically, the solution of two linear equations is:
 (a) the y -intercept (b) the slope
 (c) the intersection point (d) the origin

Summary of 2.5

- Linear equation: isolate x ; always substitute back to check.
- Two equations: elimination/substitution; graph $\Rightarrow$ intersection point.
- Quadratic: factor or use $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$; roots = x -intercepts.
- Inequalities: flip the sign when multiplying/dividing by a negative number.
- Quadratic inequality ($a > 0$): " < 0 " between the roots; " > 0 " outside the roots.

2.6 Quantitative Reasoning Exercises Using Mathematical Concepts**From words to mathematics**

1. Choose a letter for the unknown and write what it means (with units).
2. Translate the sentences into an equation, inequality, set or formula.
3. Solve, then interpret the answer in the words of the question.

Example 2.6.1: Comparing mobile plans

Plan A: Rs. 300 per month + Rs. 2 per minute. Plan B: Rs. 500 per month + Rs. 1 per minute. When do both plans cost the same? Which is cheaper for heavy users?

Solution.

Step 1: Let m = minutes used. Cost A = $300 + 2m$; cost B = $500 + m$.

Step 2: Equal cost: $300 + 2m = 500 + m \Rightarrow m = 200$ minutes (both cost Rs. 700).

Step 3: Test a larger value, $m = 300$: A = 900, B = 800. So **Plan B** is cheaper above 200 minutes; Plan A below.

>>> Python Solution**Example 2.6.1****CODE**

```
1 cost_A = lambda m: 300 + 2*m
2 cost_B = lambda m: 500 + m
3 for m in [100, 200, 300]:
4     cheaper = "A" if cost_A(m) < cost_B(m) else ("B" if cost_B(m) < cost_A(m) else
5         ↪ "same")
6     print(f"{m} min: A = {cost_A(m)}, B = {cost_B(m)} -> {cheaper}")
```

OUTPUT

```
100 min: A = 500, B = 600 -> A
200 min: A = 700, B = 700 -> same
300 min: A = 900, B = 800 -> B
```

Example 2.6.2: Garden dimensions

A rectangular garden has perimeter 40 m and its length is 4 m more than its width. Find its area.

Solution.

Step 1: Let width = w ; then length = $w + 4$.

Step 2: Perimeter: $2(w + w + 4) = 40 \Rightarrow 4w + 8 = 40 \Rightarrow w = 8$ m; length = 12 m.

Step 3: Area = $12 \times 8 = 96 \text{ m}^2$.

>>> Python Solution**Example 2.6.2****CODE**

```
1 from sympy import symbols, Eq, solve
2 w = symbols("w")
3 width = solve(Eq(2*(w + w + 4), 40), w)[0]
4 length = width + 4
5 print("Width:", width, " Length:", length, " Area:", width * length, "m^2")
```

OUTPUT

```
Width: 8 Length: 12 Area: 96 m^2
```

Example 2.6.3: Largest area for a fixed fence

A farmer has 40 m of fencing to make a rectangular pen. What dimensions give the largest area?

Solution.

Step 1: If one side is x , the other is $\frac{40-2x}{2} = 20 - x$.

Step 2: Area $A = x(20 - x) = -x^2 + 20x$, a downward parabola.

Step 3: Maximum at the vertex: $x = -\frac{b}{2a} = -\frac{20}{2(-1)} = 10$.

Step 4: Sides $10 \text{ m} \times 10 \text{ m}$ (a square), area 100 m^2 .

>>> Python Solution

Example 2.6.3

CODE

```
1 # Area A = x(20 - x) = -x^2 + 20x, a downward parabola
2 best = -20 / (2 * -1) # vertex x = -b/(2a)
3 print("Best side:", best, "m Area:", best * (20 - best), "m^2")
4 for side in [6, 8, 10, 12, 14]:
5     print(f" side {side:>2}: area {side * (20 - side)}")
```

OUTPUT

```
Best side: 10.0 m Area: 100.0 m^2
side  6: area 84
side  8: area 96
side 10: area 100
side 12: area 96
side 14: area 84
```

Example 2.6.4: Height of a ball

A ball's height (m) after t seconds is $h = -5t^2 + 20t$. Find the maximum height and when it lands.

Solution.

Step 1: $a = -5 < 0$, so the parabola opens downward and the vertex is the highest point.

Step 2: Time of maximum: $t = -\frac{b}{2a} = -\frac{20}{2(-5)} = 2 \text{ s}$.

Step 3: Maximum height: $h(2) = -5(4) + 20(2) = -20 + 40 = 20 \text{ m}$.

Step 4: Lands when $h = 0$: $-5t(t - 4) = 0 \Rightarrow t = 0$ (start) or $t = 4 \text{ s}$.

>>> Python Solution

Example 2.6.4

CODE

```
1 from sympy import symbols, solve, Rational
2 t = symbols("t")
3 h = -5*t**2 + 20*t
4 t_max = Rational(-20, 2 * -5) # vertex t = -b/(2a)
5 print("Max height at t =", t_max, "s, height =", h.subs(t, t_max), "m")
6 print("Lands at t =", solve(h, t))
```

OUTPUT

```
Max height at t = 2 s, height = 20 m
Lands at t = [0, 4]
```

Example 2.6.5: Ladder against a wall

A 10 m ladder leans against a wall with its foot 6 m from the wall. How high does it reach?

Solution.

Step 1: Wall, ground and ladder form a right triangle; the ladder is the hypotenuse.

Step 2: $h^2 + 6^2 = 10^2 \Rightarrow h^2 = 100 - 36 = 64$.

Step 3: $h = 8$ m.

>>> Python Solution**Example 2.6.5****CODE**

```
1 import math
2 print("Height reached:", math.sqrt(10**2 - 6**2), "m")
```

OUTPUT

```
Height reached: 8.0 m
```

Example 2.6.6: Size of a TV screen

A “32-inch” TV has a diagonal of 32 inches and a width-to-height ratio 16 : 9. Find its width and height.

Solution.

Step 1: Let width = $16k$ and height = $9k$.

Step 2: Pythagoras: $(16k)^2 + (9k)^2 = 32^2 \Rightarrow 256k^2 + 81k^2 = 1024 \Rightarrow 337k^2 = 1024$.

Step 3: $k^2 = \frac{1024}{337} \approx 3.039 \Rightarrow k \approx 1.743$.

Step 4: Width $\approx 16 \times 1.743 \approx 27.9$ in; height $\approx 9 \times 1.743 \approx 15.7$ in.

>>> Python Solution**Example 2.6.6****CODE**

```
1 import math
2 k = math.sqrt(32**2 / (16**2 + 9**2))
3 print(f"k = {k:.3f}")
4 print(f"Width = {16*k:.1f} inches")
5 print(f"Height = {9*k:.1f} inches")
```

OUTPUT

```
k = 1.743
Width = 27.9 inches
Height = 15.7 inches
```

Example 2.6.7: Break-even point

A small business has cost $C = 2000 + 50x$ and revenue $R = 90x$ for x items. How many items must be sold to break even?

Solution.

Step 1: Break even means no profit, no loss: $R = C$.

Step 2: $90x = 2000 + 50x \Rightarrow 40x = 2000 \Rightarrow x = 50$ items.

Step 3: Profit = $R - C = 40x - 2000$, which is positive when $x > 50$.

>>> Python Solution

Example 2.6.7

CODE

```
1 from sympy import symbols, Eq, solve
2 x = symbols("x")
3 print("Break-even at", solve(Eq(90*x, 2000 + 50*x), x)[0], "items")
4 for items in [40, 50, 60]:
5     print(f" {items} items: profit = {90*items - (2000 + 50*items)}")
```

OUTPUT

```
Break-even at 50 items
40 items: profit = -400
50 items: profit = 0
60 items: profit = 400
```

Example 2.6.8: Ticket sales

30 tickets were sold: adults at Rs.200 and children at Rs.100. Total takings were Rs.4500. How many of each?

Solution.

Step 1: Let a = adult tickets and c = child tickets: $a + c = 30$ and $200a + 100c = 4500$.

Step 2: Divide the second equation by 100: $2a + c = 45$.

Step 3: Subtract the first equation: $(2a + c) - (a + c) = 45 - 30 \Rightarrow a = 15$; so $c = 15$.

>>> Python Solution

Example 2.6.8

CODE

```
1 from sympy import symbols, Eq, solve
2 a, c = symbols("a c")
3 print(solve([Eq(a + c, 30), Eq(200*a + 100*c, 4500)], [a, c]))
```

OUTPUT

```
{a: 15, c: 15}
```

Example 2.6.9: Budget inequality

A student has Rs.5000. A bag costs Rs.1400 and notebooks cost Rs.120 each. How many notebooks can she buy with the bag?

Solution.

Step 1: Let n = number of notebooks. Total cost must not exceed 5000: $1400 + 120n \leq 5000$.

Step 2: $120n \leq 3600 \Rightarrow n \leq 30$.

Step 3: She can buy at most 30 notebooks.

>>> Python Solution

Example 2.6.9

CODE

```
1 from sympy import symbols, solve_univariate_inequality
2 n = symbols("n", real=True)
```

```
3 print(solve_univariate_inequality(1400 + 120*n <= 5000, n))
4 print("Maximum notebooks:", (5000 - 1400) // 120)
```

OUTPUT

```
n <= 30
Maximum notebooks: 30
```

Exercise 2.6 (solve by hand, then check with Python)

1. A tutor charges Rs. 1600 registration + Rs. 800 per class; another charges Rs. 1000 per class with no registration. After how many classes are the costs equal?
2. The length of a rectangle is twice its width and its perimeter is 36 cm. Find its area.
3. The product of two consecutive positive integers is 56. Find them.
4. A stone's height is $h = -5t^2 + 30t$. Find its maximum height.
5. In a class of 40 students, 25 study Physics, 20 study Chemistry and 8 study both. How many study neither?
6. A rickshaw fare is $F = 50 + 25d$. For which distances is the fare at most Rs. 300?

Multiple Choice Questions – 2.6

1. If $2x + 5 = x + 12$, then $x =$

(a) 5	(b) 7
(c) 12	(d) 17
2. The maximum of $h = -t^2 + 6t$ occurs at $t =$

(a) 2	(b) 3
(c) 6	(d) 9
3. Two numbers add to 30 and differ by 6. The larger number is:

(a) 12	(b) 15
(c) 18	(d) 24
4. A 13 m ladder with its foot 5 m from a wall reaches a height of:

(a) 8 m	(b) 12 m
(c) 18 m	(d) 10 m

Summary of 2.6

- Real problems become equations: define the variable, translate, solve, interpret.
- "Same cost", "break even" $\Rightarrow$ set two expressions equal.
- "Maximum/minimum" of a quadratic $\Rightarrow$ vertex $t = -\frac{b}{2a}$.
- Right-angle situations $\Rightarrow$ Pythagoras theorem.

UNIT 3

Fundamental Statistical Concepts

3.1 Population and Sample

Key Concepts

- **Population** – the *whole* group we want to study (all students of a college, all voters of Balochistan).
- **Sample** – a *part* of the population actually observed.
- **Census** – data collected from every member of the population; **sample survey** – from a sample only.
- **Parameter** – a number describing the population (population mean μ , SD σ).
Statistic – a number describing a sample (sample mean $\bar{x}$, SD s). *Memory tip: Parameter–Population, Statistic–Sample.*
- **Why sample?** Saves time and money; sometimes testing destroys the item (e.g. testing bulbs).

Sampling methods

Simple random	every member has an equal chance (lottery method, random numbers)
Systematic	choose every k -th member, $k = \frac{N}{n}$, after a random start
Stratified	divide population into groups (strata) and sample from each, usually in proportion
Cluster	divide into clusters (e.g. classes, villages), randomly select whole clusters
Convenience	whoever is easy to reach – quick but often biased

Sampling bias occurs when the sample does not represent the population fairly.

Example 3.1.1: Identifying population, sample, parameter and statistic

To find the average height of the students of a college, the heights of 80 students are measured.

Solution.

Step 1: Population: the whole group we want to know about – all students of the college.

Step 2: Sample: the part actually measured – the 80 students.

Step 3: Parameter: the mean height of *all* students (μ). It is unknown.

Step 4: Statistic: the mean height of the 80 students ($\bar{x}$). We calculate it and use it to estimate μ .

>>> Python Solution

Example 3.1.1

CODE

```

1 import random
2 import statistics as st
3 random.seed(5)
4 # Imagine a college of 1200 students with heights (cm) -- the POPULATION
5 population = [round(random.gauss(165, 7), 1) for _ in range(1200)]
6 sample = random.sample(population, 80)           # the SAMPLE of 80 students
7 mu = st.mean(population)                         # PARAMETER (usually unknown)
8 x_bar = st.mean(sample)                         # STATISTIC (we can calculate)
9 print(f"Population size {len(population)}, sample size {len(sample)}")
10 print(f"Parameter mu      = {mu:.2f} cm")
11 print(f"Statistic x-bar = {x_bar:.2f} cm (an estimate of mu)")

```

OUTPUT

```

Population size 1200, sample size 80
Parameter mu      = 164.84 cm
Statistic x-bar = 165.74 cm (an estimate of mu)

```

Example 3.1.2: Census or sample?

Should we use a census or a sample to (a) count the population of Pakistan, (b) test the life of light bulbs from a factory, (c) check the blood sugar of a patient?

Solution.

Step 1: (a) The government needs a count of every person: **census**.

Step 2: (b) Testing a bulb's life burns it out; testing all bulbs would leave none to sell: **sample**.

Step 3: (c) A few drops of blood represent all the blood: **sample**.

▷ Why? A sample is chosen when a census is too costly, too slow or destroys the items.

>>> Python Solution

Example 3.1.2

CODE

```

1 situations = {
2     "count the population of Pakistan": ("every person must be counted", "census"),
3     "test the life of light bulbs":      ("testing destroys the bulb", "sample"),
4     "check a patient's blood sugar":     ("a few drops represent all blood",
5     ↪ "sample"),
6 }
7 for task, (reason, method) in situations.items():
8     print(f"{task:<34} -> {method:<7} ({reason})")

```

OUTPUT

```

count the population of Pakistan -> census (every person must be counted)
test the life of light bulbs      -> sample (testing destroys the bulb)
check a patient's blood sugar     -> sample (a few drops represent all blood)

```

Example 3.1.3: Simple random sampling

A class has 40 students. Explain how to select a simple random sample of 5, and find the chance that a particular student is chosen.

Solution.

Step 1: Write the numbers 1 to 40 on identical slips, mix them well in a box.

Step 2: Draw 5 slips without looking (and without putting them back).

▷ *Why? Every student, and every group of 5, has the same chance – no personal choice is involved.*

Step 3: The chance that a given student is chosen $= \frac{5}{40} = \frac{1}{8}$.

>>> Python Solution

Example 3.1.3

CODE

```
1 import random
2 random.seed(7)
3 slips = list(range(1, 41))           # 40 numbered slips
4 chosen = random.sample(slips, 5)     # draw 5 without replacement
5 print("Chosen roll numbers:", sorted(chosen))
6 print("Chance a given student is chosen:", 5 / 40)
```

OUTPUT

```
Chosen roll numbers: [4, 5, 10, 21, 26]
Chance a given student is chosen: 0.125
```

Example 3.1.4: Systematic sampling

A college has 500 students listed by roll number. Select a systematic sample of 50.

Solution.

Step 1: Sampling interval $k = \frac{N}{n} = \frac{500}{50} = 10$.

Step 2: Choose a random start between 1 and 10, say 7.

Step 3: Then take every 10th roll number: 7, 17, 27, 37, ..., 497 (50 students).

▷ *Why? Systematic sampling is easy when a list exists, but beware if the list has a repeating pattern.*

>>> Python Solution

Example 3.1.4

CODE

```
1 import random
2 N, n = 500, 50
3 k = N // n
4 random.seed(3)
5 start = random.randint(1, k)
6 sample = list(range(start, N + 1, k))
7 print("k =", k, " random start =", start)
8 print("First five:", sample[:5], "... last:", sample[-1])
9 print("Sample size:", len(sample))
```

OUTPUT

```
k = 10  random start = 4
First five: [4, 14, 24, 34, 44] ... last: 494
Sample size: 50
```

Example 3.1.5: Stratified sampling

A college has 600 students: 360 science and 240 arts. Take a proportional stratified sample of 50.

Solution.

Step 1: Each group (stratum) gets a share proportional to its size: $\text{share} = \frac{\text{stratum size}}{N} \times n$.

Step 2: Science: $\frac{360}{600} \times 50 = 0.6 \times 50 = 30$ students.

Step 3: Arts: $\frac{240}{600} \times 50 = 0.4 \times 50 = 20$ students.

Step 4: Check: $30 + 20 = 50$. Then choose randomly within each group.

>>> Python Solution**Example 3.1.5****CODE**

```
1 strata = {"Science": 360, "Arts": 240}
2 N = sum(strata.values())
3 n = 50
4 for name, size in strata.items():
5     print(f"{name:<8}: {size} / {N} x {n} = {size / N * n:.0f} students")
```

OUTPUT

```
Science : 360 / 600 x 50 = 30 students
Arts    : 240 / 600 x 50 = 20 students
```

Example 3.1.6: Cluster sampling

A college has 30 sections of about 40 students each. Describe a cluster sample and compare it with a stratified sample.

Solution.

Step 1: Treat each section as a *cluster*. Randomly choose, say, 3 sections.

Step 2: Survey *every* student in those 3 sections: about $3 \times 40 = 120$ students.

Step 3: Difference: in **stratified** sampling we take some members from *every* group; in **cluster** sampling we take *all* members of a *few* groups.

▷ Why? Cluster sampling saves travel and time when groups are spread out (e.g. villages).

>>> Python Solution**Example 3.1.6****CODE**

```
1 import random
2 random.seed(2)
3 sections = [f"Section {i}" for i in range(1, 31)]
4 chosen = random.sample(sections, 3)
5 print("Clusters chosen:", chosen)
6 print("Students surveyed:", 3 * 40, "(everyone in the chosen sections)")
```

OUTPUT

```
Clusters chosen: ['Section 28', 'Section 2', 'Section 3']
Students surveyed: 120 (everyone in the chosen sections)
```

Example 3.1.7: Name the method

Name the sampling method: (a) every 5th customer leaving a shop is asked; (b) students are split into boys and girls and a random sample is taken from each; (c) names are drawn from a hat; (d) people standing outside a mall are asked.

Solution.

Step 1: (a) Fixed interval $\Rightarrow$ **systematic**.

Step 2: (b) Groups, sample from each $\Rightarrow$ **stratified**.

Step 3: (c) Equal chance for all $\Rightarrow$ **simple random**.

Step 4: (d) Whoever is nearby $\Rightarrow$ **convenience** (likely biased).

>>> Python Solution**Example 3.1.7****CODE**

```
1 def method(description):
2     rules = {"every": "systematic", "groups": "stratified",
3             "hat": "simple random", "nearby": "convenience"}
4     for word, name in rules.items():
5         if word in description:
6             return name
7
8 for d in ["every 5th customer", "boys and girls groups, random from each",
9         "names drawn from a hat", "people nearby outside a mall"]:
10    print(f"{d:<40} -> {method(d)}")
```

OUTPUT

```
every 5th customer                -> systematic
boys and girls groups, random from each -> stratified
names drawn from a hat            -> simple random
people nearby outside a mall      -> convenience
```

Example 3.1.8: Spotting bias

A TV channel runs an online poll asking “Do you use the internet daily?” and reports that 95% of Pakistanis do.

Solution.

Step 1: Who could answer? Only people who are *already online*.

Step 2: People without internet access had no chance to be included.

Step 3: So the sample over-represents internet users: the result is **biased** and cannot be generalised to all Pakistanis.

>>> Python Solution**Example 3.1.8****CODE**

```
1 # A population of 1000 people: how often do they use the internet?
2 population = ["daily"] * 500 + ["sometimes"] * 200 + ["never"] * 300
3 true_daily = population.count("daily") / len(population)
4 # An ONLINE poll only reaches people who use the internet at all
5 reached = [p for p in population if p != "never"]
6 poll_daily = reached.count("daily") / len(reached)
```

```

7 print(f"True share using it daily: {true_daily:.0%}")
8 print(f"Online poll says          : {poll_daily:.0%}  <- biased upward")

```

OUTPUT

```

True share using it daily: 50%
Online poll says          : 71%  <- biased upward

```

Exercise 3.1 (solve by hand, then check with Python)

1. A factory makes 1200 bulbs a day. Find the sampling interval k for a systematic sample of 60 bulbs.
2. An office has 1000 employees: 600 men and 400 women. How many of each should be in a proportional stratified sample of 80?
3. State whether each is a parameter or statistic: (a) average income of all households in Pakistan (from a census); (b) average marks of 40 randomly chosen students.
4. Why might asking only your friends about a new canteen be a poor sample?
5. Give two reasons why we usually study a sample instead of the whole population.

Multiple Choice Questions – 3.1

1. A numerical value describing a sample is called a:

(a) parameter	(b) statistic
(c) population	(d) census
2. Choosing every 10th name from a list is:

(a) simple random	(b) stratified
(c) systematic	(d) cluster
3. Data collected from every member of a population is a:

(a) sample	(b) census
(c) statistic	(d) stratum
4. The population mean is usually denoted by:

(a) $\bar{x}$	(b) s
(c) μ	(d) n
5. Which method is most likely to be biased?

(a) simple random	(b) stratified
(c) systematic	(d) convenience

Summary of 3.1

- Population = whole group (parameter); sample = part of it (statistic).
- Random, systematic, stratified and cluster methods reduce bias; convenience samples often add bias.
- A good sample must represent the population; a big but biased sample is still unreliable.

3.2 Measures of Central Tendency, Dispersion and Data Interpretation

Central Tendency (the “centre” of data)

- **Mean (average):** $\bar{x} = \frac{\sum x}{n}$; grouped data: $\bar{x} = \frac{\sum fx}{\sum f}$ (x = mid-point); weighted mean: $\bar{x}_w = \frac{\sum wx}{\sum w}$.
- **Median:** middle value of the *ordered* data – position $\left(\frac{n+1}{2}\right)$ th. If n is even, take the average of the two middle values.
- **Mode:** the value that occurs most often (there may be no mode or more than one).

Dispersion (the “spread” of data)

- **Range** = largest – smallest.
- **Quartiles:** Q_1 at position $\frac{n+1}{4}$, Q_2 (median), Q_3 at position $\frac{3(n+1)}{4}$ of ordered data; **inter-quartile range** $IQR = Q_3 - Q_1$.
- **Variance:** population $\sigma^2 = \frac{\sum (x - \bar{x})^2}{n}$; sample $s^2 = \frac{\sum (x - \bar{x})^2}{n - 1}$.
- **Standard deviation (SD)** = $\sqrt{\text{variance}}$ – the typical distance of values from the mean.
- **Coefficient of variation:** $CV = \frac{SD}{\text{mean}} \times 100\%$ – used to compare consistency.

Example 3.2.1: Mean, median, mode and range

Find the mean, median, mode and range of 3, 5, 7, 7, 8.

Solution.

Step 1: Mean: add all values and divide by how many there are: $\frac{3 + 5 + 7 + 7 + 8}{5} = \frac{30}{5} = 6$.

Step 2: Median: the data are already in order; $n = 5$, so the median is the $\frac{5+1}{2} = 3$ rd value = 7.

Step 3: Mode: the value that appears most often = 7.

Step 4: Range = largest – smallest = $8 - 3 = 5$.

>>> Python Solution

Example 3.2.1

CODE

```
1 import statistics as st
2 data = [3, 5, 7, 7, 8]
3 print("Mean :", st.mean(data))
4 print("Median:", st.median(data))
5 print("Mode :", st.mode(data))
6 print("Range :", max(data) - min(data))
```

OUTPUT

```
Mean : 6
Median: 7
```

```
Mode : 7
Range : 5
```

Example 3.2.2: Median with an even number of values

Find the median of 4, 9, 2, 7, 5, 8.

Solution.

Step 1: First put the data in order: 2, 4, 5, 7, 8, 9 ($n = 6$).

▷ Why? The median is meaningless unless the data are ordered.

Step 2: The median position is $\frac{6+1}{2} = 3.5$: halfway between the 3rd and 4th values (5 and 7).

Step 3: Median = $\frac{5+7}{2} = 6$.

>>> Python Solution

Example 3.2.2

CODE

```
1 import statistics as st
2 data = [4, 9, 2, 7, 5, 8]
3 print("Ordered:", sorted(data))
4 print("Median :", st.median(data))
```

OUTPUT

```
Ordered: [2, 4, 5, 7, 8, 9]
Median : 6.0
```

Example 3.2.3: The effect of an outlier

Data: 4, 5, 6, 7, 8. Find the mean and median. Then replace 8 by 58 and find them again.

Solution.

Step 1: Original: mean = $\frac{30}{5} = 6$; median = 6.

Step 2: New data 4, 5, 6, 7, 58: mean = $\frac{80}{5} = 16$; median is still the 3rd value = 6.

Step 3: One extreme value moved the mean from 6 to 16 but did not change the median.

▷ Why? The median depends only on the middle position, so it resists outliers.

>>> Python Solution

Example 3.2.3

CODE

```
1 import statistics as st
2 before = [4, 5, 6, 7, 8]
3 after = [4, 5, 6, 7, 58]
4 for name, d in [("original", before), ("with outlier", after)]:
5     print(f"{name:<13} mean = {st.mean(d):>4}, median = {st.median(d)}")
```

OUTPUT

```
original      mean =    6, median = 6
with outlier  mean =   16, median = 6
```

Example 3.2.4: Variance and standard deviation

Find the variance and SD of 3, 5, 7, 7, 8 (mean = 6).

Solution.

Step 1: Find each deviation from the mean, $x - \bar{x}$, then square it (squaring removes minus signs).

x	$x - \bar{x}$	$(x - \bar{x})^2$
3	-3	9
5	-1	1
7	1	1
7	1	1
8	2	4
$\Sigma = 0$		$\Sigma = 16$

Step 2: Check: the deviations add to 0. ✓

Step 3: Population variance $\sigma^2 = \frac{16}{5} = 3.2$; SD $\sigma = \sqrt{3.2} \approx 1.79$.

Step 4: Sample variance $s^2 = \frac{16}{5-1} = 4$; SD $s = \sqrt{4} = 2$.

▷ Why? Dividing by $n - 1$ for a sample corrects for the sample usually being less spread out than the population.

>>> Python Solution**Example 3.2.4****CODE**

```
1 import statistics as st
2 data = [3, 5, 7, 7, 8]
3 m = st.mean(data)
4 print(f'{'x':>3}{'x - mean':>10}{'squared':>9}")
5 for x in data:
6     print(f'{'x':>3}{'x - m':>10}{'(x - m)**2':>9}")
7 print("Sum of squares:", sum((x - m)**2 for x in data))
8 print(f"Population variance = {st.pvariance(data)}, SD = {st.pstdev(data):.2f}")
9 print(f"Sample variance      = {st.variance(data)}, SD = {st.stdev(data):.2f}")
```

OUTPUT

```
x  x - mean  squared
3    -3      9
5    -1      1
7     1      1
7     1      1
8     2      4
Sum of squares: 16
Population variance = 3.2, SD = 1.79
Sample variance     = 4, SD = 2.00
```

Example 3.2.5: Shortcut formula for variance

Use $\sigma^2 = \frac{\sum x^2}{n} - \bar{x}^2$ to find the SD of 2, 4, 4, 4, 5, 5, 7, 9.

Solution.

Step 1: $n = 8$, $\sum x = 40$, so $\bar{x} = 5$.

Step 2: $\sum x^2 = 4 + 16 + 16 + 16 + 25 + 25 + 49 + 81 = 232$.

Step 3: $\sigma^2 = \frac{232}{8} - 5^2 = 29 - 25 = 4$, so $\sigma = 2$.

▷ Why? This formula avoids computing every deviation – useful with a calculator.

>>> Python Solution

Example 3.2.5

CODE

```
1 data = [2, 4, 4, 4, 5, 5, 7, 9]
2 n = len(data)
3 mean = sum(data) / n
4 sum_sq = sum(x**2 for x in data)
5 var = sum_sq / n - mean**2
6 print("Sum of x^2 =", sum_sq)
7 print("Variance =", var, " SD =", var ** 0.5)
```

OUTPUT

```
Sum of x^2 = 232
Variance = 4.0  SD = 2.0
```

Example 3.2.6: Quartiles and IQR

Find Q_1 , Q_2 , Q_3 and the IQR of 2, 4, 5, 7, 8, 9, 11.

Solution.

Step 1: The data are ordered and $n = 7$, so $n + 1 = 8$.

Step 2: Q_1 is at position $\frac{8}{4} = 2$: the 2nd value = 4.

Step 3: Q_2 (median) at position $\frac{8}{2} = 4$: 7. Q_3 at position $\frac{3 \times 8}{4} = 6$: 9.

Step 4: $IQR = Q_3 - Q_1 = 9 - 4 = 5$: the middle half of the data spans 5 units.

>>> Python Solution

Example 3.2.6

CODE

```
1 import statistics as st
2 data = [2, 4, 5, 7, 8, 9, 11]
3 q1, q2, q3 = st.quantiles(data, n=4)      # (n+1) method
4 print("Q1 =", q1, " Q2 =", q2, " Q3 =", q3)
5 print("IQR =", q3 - q1)
```

OUTPUT

```
Q1 = 4.0  Q2 = 7.0  Q3 = 9.0
IQR = 5.0
```

Example 3.2.7: Mean, median and mode from a frequency table

Use the goals table of Example 1.6.2 (x : 0,1,2,3,4; f : 4,8,5,2,1) to find the mean, median and mode.

Solution.

Step 1: Multiply each value by its frequency: $fx = 0, 8, 10, 6, 4$; $\sum fx = 28$, $\sum f = 20$.

Step 2: Mean = $\frac{\sum fx}{\sum f} = \frac{28}{20} = 1.4$ goals.

Step 3: Median: $n = 20$, so we need the 10th and 11th values. Cumulative frequencies: 4, 12, ... – values 5 to 12 are all 1. So median = 1.

Step 4: Mode = 1 (highest frequency 8).

>>> Python Solution

Example 3.2.7

CODE

```
1 values = [0, 1, 2, 3, 4]
2 freq = [4, 8, 5, 2, 1]
3 n = sum(freq)
4 mean = sum(x * f for x, f in zip(values, freq)) / n
5 expanded = [x for x, f in zip(values, freq) for _ in range(f)] # all 20 values
6 expanded.sort()
7 median = (expanded[n//2 - 1] + expanded[n//2]) / 2
8 mode = values[freq.index(max(freq))]
9 print("Mean =", mean, " Median =", median, " Mode =", mode)
```

OUTPUT

```
Mean = 1.4  Median = 1.0  Mode = 1
```

Example 3.2.8: Mean of grouped data

Find the mean of the marks table of Example 1.6.1.

Solution.

Step 1: We do not know the individual marks inside each class, so we use each class **mid-point** as its representative.

Step 2: Multiply mid-point by frequency and add.

Class	Mid-point x	f	fx
10–19	14.5	4	58.0
20–29	24.5	7	171.5
30–39	34.5	6	207.0
40–49	44.5	3	133.5
		$\sum f = 20$	$\sum fx = 570$

Step 3: $\bar{x} = \frac{570}{20} = 28.5$.

▷ Why? The exact mean of the 20 raw marks is $582/20 = 29.1$; grouped data give a close approximation.

>>> Python Solution

Example 3.2.8

CODE

```
1 mids = [14.5, 24.5, 34.5, 44.5]
2 freq = [4, 7, 6, 3]
3 fx = [m * f for m, f in zip(mids, freq)]
```

```

4 print("fx values:", fx)
5 print("Grouped mean =", sum(fx) / sum(freq))
6 raw = [12, 25, 37, 18, 42, 29, 33, 15, 27, 39, 21, 45, 31, 24, 36, 19, 28, 41, 34,
    ↪ 26]
7 print("Exact mean of raw data =", sum(raw) / len(raw))

```

OUTPUT

```

fx values: [58.0, 171.5, 207.0, 133.5]
Grouped mean = 28.5
Exact mean of raw data = 29.1

```

Example 3.2.9: Median and mode of grouped data

For the same marks table, estimate the median and mode using

$$\text{Median} = L + \frac{\frac{n}{2} - cf}{f} \times h, \quad \text{Mode} = L + \frac{f_1 - f_0}{2f_1 - f_0 - f_2} \times h.$$

Solution.

Step 1: $\frac{n}{2} = 10$. Cumulative frequencies 4, 11, ...: the 10th value lies in class 20–29 (the *median class*).

Step 2: For this class: $L = 19.5$ (lower boundary), $cf = 4$ (before it), $f = 7$, $h = 10$.

Step 3: Median = $19.5 + \frac{10 - 4}{7} \times 10 = 19.5 + 8.57 \approx 28.07$.

Step 4: Modal class is also 20–29 ($f_1 = 7$), with $f_0 = 4$ before and $f_2 = 6$ after.

Step 5: Mode = $19.5 + \frac{7 - 4}{14 - 4 - 6} \times 10 = 19.5 + \frac{3}{4} \times 10 = 27$.

>>> Python Solution**Example 3.2.9****CODE**

```

1 lower = [9.5, 19.5, 29.5, 39.5]
2 freq = [4, 7, 6, 3]
3 h, n = 10, sum(freq)
4 cum = 0
5 for i, f in enumerate(freq):           # find the median class
6     if cum + f >= n / 2:
7         break
8     cum += f
9 median = lower[i] + (n / 2 - cum) / f * h
10 j = freq.index(max(freq))              # modal class
11 f0, f1, f2 = freq[j - 1], freq[j], freq[j + 1]
12 mode = lower[j] + (f1 - f0) / (2 * f1 - f0 - f2) * h
13 print(f"Median = {median:.2f}")
14 print(f"Mode   = {mode:.2f}")

```

OUTPUT

```

Median = 28.07
Mode   = 27.00

```

Example 3.2.10: Weighted mean – GPA

A student's courses: 3 credits (grade point 4), 3 credits (3), 2 credits (3.5), 1 credit (2). Find the GPA.

Solution.

Step 1: Courses with more credits should count more, so use a **weighted mean** with credits as weights.

Step 2: $\sum wx = 3(4) + 3(3) + 2(3.5) + 1(2) = 12 + 9 + 7 + 2 = 30$.

Step 3: $\sum w = 3 + 3 + 2 + 1 = 9$.

Step 4: $\text{GPA} = \frac{30}{9} \approx 3.33$.

>>> Python Solution**Example 3.2.10****CODE**

```
1 credits = [3, 3, 2, 1]
2 points = [4, 3, 3.5, 2]
3 gpa = sum(c * p for c, p in zip(credits, points)) / sum(credits)
4 print(f"GPA = {gpa:.2f}")
```

OUTPUT

```
GPA = 3.33
```

Example 3.2.11: Combined mean

Class A (30 students) has mean 60; class B (20 students) has mean 70. Find the mean of all 50 students.

Solution.

Step 1: Total marks of A = $30 \times 60 = 1800$; of B = $20 \times 70 = 1400$.

Step 2: Combined mean = $\frac{1800 + 1400}{50} = \frac{3200}{50} = 64$.

▷ Why? It is not $\frac{60+70}{2} = 65$, because the classes have different sizes.

>>> Python Solution**Example 3.2.11****CODE**

```
1 sizes = [30, 20]
2 means = [60, 70]
3 combined = sum(n * m for n, m in zip(sizes, means)) / sum(sizes)
4 print("Combined mean =", combined, " (simple average of means would be",
    ↪ sum(means) / 2, ")")
```

OUTPUT

```
Combined mean = 64.0 (simple average of means would be 65.0 )
```

Example 3.2.12: Interpreting spread – who is more consistent?

Batsman A scored 40, 50, 60; batsman B scored 10, 50, 90. Compare them.

Solution.

Step 1: Means: $\bar{x}_A = \frac{150}{3} = 50$, $\bar{x}_B = \frac{150}{3} = 50$ – the same.

Step 2: SD of A: $\sqrt{\frac{(-10)^2 + 0^2 + 10^2}{3}} = \sqrt{\frac{200}{3}} \approx 8.16$.

Step 3: SD of B: $\sqrt{\frac{(-40)^2 + 0^2 + 40^2}{3}} = \sqrt{\frac{3200}{3}} \approx 32.66$.

Step 4: CV: A = $\frac{8.16}{50} \times 100 \approx 16.3\%$; B = $\frac{32.66}{50} \times 100 \approx 65.3\%$.

Interpretation: same average, but A has a much smaller spread, so **A is more consistent**.

>>> Python Solution

Example 3.2.12

CODE

```
1 import statistics as st
2 A = [40, 50, 60]
3 B = [10, 50, 90]
4 for name, d in [("A", A), ("B", B)]:
5     m, sd = st.mean(d), st.pstdev(d)
6     print(f"Batsman {name}: mean = {m}, SD = {sd:.2f}, CV = {sd / m:.1%}")
```

OUTPUT

```
Batsman A: mean = 50, SD = 8.16, CV = 16.3%
Batsman B: mean = 50, SD = 32.66, CV = 65.3%
```

Example 3.2.13: Comparing variables with different units

Heights of students: mean 160 cm, SD 8 cm. Weights: mean 55 kg, SD 11 kg. Which varies more?

Solution.

Step 1: We cannot compare 8 cm with 11 kg directly – different units.

Step 2: CV (height) = $\frac{8}{160} \times 100 = 5\%$; CV (weight) = $\frac{11}{55} \times 100 = 20\%$.

Step 3: Weights vary more (relative to their mean).

▷ Why? CV has no units, so it compares spreads fairly.

>>> Python Solution

Example 3.2.13

CODE

```
1 cv = lambda sd, mean: sd / mean * 100
2 print(f"Height CV = {cv(8, 160):.0f}%")
3 print(f"Weight CV = {cv(11, 55):.0f}%")
```

OUTPUT

```
Height CV = 5%
Weight CV = 20%
```

Exercise 3.2 (solve by hand, then check with Python)

1. For the data 6, 2, 9, 4, 4 find the mean, median, mode and range.
2. For the same data, find the population variance and SD, and the sample variance and SD.
3. Find the median of 10, 3, 7, 12.
4. Find the mean of the grouped data: classes 0–10, 10–20, 20–30 with frequencies 5, 10, 5.

5. A test has weight 2 and the final exam weight 3. A student scores 70 in the test and 80 in the final. Find the weighted mean.
6. Two classes both have mean 60; Class X has SD 5 and Class Y has SD 15. Which class performed more uniformly?

Multiple Choice Questions – 3.2

1. Which measure is most affected by an extreme value?

(a) median	(b) mode
(c) mean	(d) IQR
2. The median of 1, 2, 3, 4, 100 is:

(a) 3	(b) 22
(c) 4	(d) 100
3. The mode of 2, 3, 3, 5, 5, 5 is:

(a) 3	(b) 5
(c) 3.83	(d) 2
4. Standard deviation measures:

(a) centre	(b) spread
(c) the largest value	(d) the most frequent value
5. The sum of deviations from the mean $\sum(x - \bar{x})$ is always:

(a) positive	(b) negative
(c) zero	(d) one

Summary of 3.2

- Centre: mean (uses all values, sensitive to outliers), median (middle, resistant to outliers), mode (most frequent).
- Spread: range, IQR, variance, SD; larger SD $\Rightarrow$ more variable data.
- CV compares consistency of data sets with different means or units.
- Always report a measure of centre *with* a measure of spread.

3.3 Rules of Counting (Multiplicative, Permutation and Combination)

Important Formulas

- **Multiplicative rule:** if one task can be done in m ways and a second in n ways, both can be done in $m \times n$ ways.
- **Factorial:** $n! = n \times (n-1) \times \cdots \times 2 \times 1$; $0! = 1$.
- **Permutation** (*order matters* – arrangements): ${}^nP_r = \frac{n!}{(n-r)!}$.
- **Combination** (*order does not matter* – selections): ${}^nC_r = \binom{n}{r} = \frac{n!}{r!(n-r)!}$.
- Arrangements of n objects with p alike of one kind, q alike of another: $\frac{n!}{p!q!}$.

Ask yourself: “Does the order matter?” Yes $\Rightarrow$ permutation (ranks, posts, codes); No $\Rightarrow$

combination (teams, committees).

Example 3.3.1: Multiplicative rule

A student has 3 shirts and 4 trousers. How many different outfits are possible?

Solution.

Step 1: Choosing an outfit is two tasks: pick a shirt (3 ways), then pick trousers (4 ways).

Step 2: Each shirt can go with each of the 4 trousers: $3 \times 4 = 12$ outfits.

>>> Python Solution

Example 3.3.1

CODE

```
1 from itertools import product
2 shirts = ["S1", "S2", "S3"]
3 trousers = ["T1", "T2", "T3", "T4"]
4 outfits = list(product(shirts, trousers))
5 print("First few:", outfits[:4])
6 print("Number of outfits:", len(outfits), "=", len(shirts), "x", len(trousers))
```

OUTPUT

```
First few: [('S1', 'T1'), ('S1', 'T2'), ('S1', 'T3'), ('S1', 'T4')]
Number of outfits: 12 = 3 x 4
```

Example 3.3.2: Addition rule versus multiplication rule

From Quetta to Karachi one can travel by 3 bus services or 2 train services. (a) In how many ways can one travel? (b) In how many ways can one go and return, using any service each way?

Solution.

Step 1: (a) Bus *or* train – only one choice is made: $3 + 2 = 5$ ways.

▷ Why? “Or” (one task, alternative options) $\Rightarrow$ add.

Step 2: (b) Going *and* returning – two tasks: $5 \times 5 = 25$ ways.

▷ Why? “And” (several tasks in sequence) $\Rightarrow$ multiply.

>>> Python Solution

Example 3.3.2

CODE

```
1 buses, trains = 3, 2
2 one_way = buses + trains          # one choice: OR -> add
3 round_trip = one_way * one_way    # two tasks: AND -> multiply
4 print("One way   :", one_way)
5 print("Round trip:", round_trip)
```

OUTPUT

```
One way   : 5
Round trip: 25
```

Example 3.3.3: Factorials

Evaluate (a) $5!$ (b) $\frac{7!}{5!}$ (c) $\frac{10!}{8!2!}$.

Solution.

Step 1: (a) $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Step 2: (b) Cancel the common part: $\frac{7 \times 6 \times 5!}{5!} = 7 \times 6 = 42$.

Step 3: (c) $\frac{10 \times 9 \times 8!}{8! \times 2} = \frac{90}{2} = 45$.

▷ Why? Do not multiply out large factorials – cancel first.

>>> Python Solution**Example 3.3.3****CODE**

```
1 import math
2 print("5!      =", math.factorial(5))
3 print("7!/5!    =", math.factorial(7) // math.factorial(5))
4 print("10!/(8! 2!) =", math.factorial(10) // (math.factorial(8) *
    ↪ math.factorial(2)))
```

OUTPUT

```
5!      = 120
7!/5!    = 42
10!/(8! 2!) = 45
```

Example 3.3.4: PIN codes

How many 4-digit PINs can be made from the digits 0–9 (a) if digits may repeat, (b) if no digit repeats?

Solution.

Step 1: Think of 4 boxes to fill: $\square \square \square \square$.

Step 2: (a) Each box can take any of 10 digits: $10 \times 10 \times 10 \times 10 = 10^4 = 10\,000$.

Step 3: (b) The first box has 10 choices, the next only 9 (one digit used), then 8, then 7: $10 \times 9 \times 8 \times 7 = 5040$.

>>> Python Solution**Example 3.3.4****CODE**

```
1 import math
2 print("(a) repetition allowed :", 10 ** 4)
3 print("(b) no repetition      :", math.perm(10, 4))
```

OUTPUT

```
(a) repetition allowed : 10000
(b) no repetition      : 5040
```

Example 3.3.5: Permutation versus combination

From 5 students: (a) choose a president and a secretary; (b) choose a 2-member committee.

Solution.

Step 1: (a) Ali-president/Sara-secretary differs from Sara-president/Ali-secretary: **order matters**. ${}^5P_2 = \frac{5!}{3!} = 5 \times 4 = 20$.

Step 2: (b) A committee {Ali, Sara} is the same either way: **order does not matter**. ${}^5C_2 = \frac{5!}{2!3!} = \frac{20}{2} = 10$.

Step 3: Link: each committee can be given posts in $2! = 2$ ways, so $20 = 10 \times 2$. ✓

>>> Python Solution

Example 3.3.5

CODE

```
1 import math
2 from itertools import permutations, combinations
3 students = ["Ali", "Sara", "Omar", "Hina", "Bilal"]
4 posts = list(permutations(students, 2)) # (president, secretary)
5 committees = list(combinations(students, 2))
6 print("President & secretary:", len(posts), "=", math.perm(5, 2))
7 print("2-member committees :", len(committees), "=", math.comb(5, 2))
```

OUTPUT

```
President & secretary: 20 = 20
2-member committees : 10 = 10
```

Example 3.3.6: Arrangements with conditions

- (a) In how many ways can 5 books be arranged on a shelf if a particular book must be first?
 (b) How many 3-digit even numbers can be formed from 1, 2, 3, 4, 5 without repetition?

Solution.

Step 1: (a) Fix the special book in place 1. The other 4 books fill 4 places: $4! = 24$ ways.

Step 2: (b) Fill the *restricted* place first. The last digit must be even: 2 or 4 (2 ways).

▷ *Why? Handling the condition first prevents running out of allowed choices.*

Step 3: Then the first digit: 4 remaining digits; the middle digit: 3 remaining. Total = $4 \times 3 \times 2 = 24$.

>>> Python Solution

Example 3.3.6

CODE

```
1 from itertools import permutations
2 books = ["A", "B", "C", "D", "E"]
3 first_A = [p for p in permutations(books) if p[0] == "A"]
4 print("(a) arrangements with book A first:", len(first_A))
5 nums = [p for p in permutations([1, 2, 3, 4, 5], 3) if p[-1] % 2 == 0]
6 print("(b) 3-digit even numbers:", len(nums), " e.g.", nums[:3])
```

OUTPUT

```
(a) arrangements with book A first: 24
(b) 3-digit even numbers: 24 e.g. [(1, 2, 4), (1, 3, 2), (1, 3, 4)]
```

Example 3.3.7: Arranging letters with repetition

How many arrangements are there of the letters of (a) BOOK (b) PAKISTAN?

Solution.

Step 1: (a) 4 letters, so $4! = 24$ if all were different. But swapping the two O's gives the same word, so divide by $2!$: $\frac{24}{2} = 12$.

Step 2: (b) PAKISTAN has 8 letters with A appearing twice: $\frac{8!}{2!} = \frac{40320}{2} = 20160$.

>>> Python Solution**Example 3.3.7****CODE**

```
1 from itertools import permutations
2 import math
3 print("BOOK      :", len(set(permutations("BOOK"))))
4 print("PAKISTAN:", math.factorial(8) // math.factorial(2))
```

OUTPUT

```
BOOK      : 12
PAKISTAN: 20160
```

Example 3.3.8: Combined selection

A committee of 2 men and 1 woman is chosen from 5 men and 4 women. How many committees are possible?

Solution.

Step 1: Choose the men: ${}^5C_2 = 10$ ways.

Step 2: Choose the woman: ${}^4C_1 = 4$ ways.

Step 3: Both choices are needed ("and"), so multiply: $10 \times 4 = 40$ committees.

>>> Python Solution**Example 3.3.8****CODE**

```
1 import math
2 print("Committees:", math.comb(5, 2), "x", math.comb(4, 1), "=", math.comb(5, 2) *
    ↪ math.comb(4, 1))
```

OUTPUT

```
Committees: 10 x 4 = 40
```

Example 3.3.9: "At least" selections

A committee of 3 is chosen from 4 men and 3 women. In how many ways can it include at least 2 women?

Solution.

Step 1: "At least 2 women" means 2 women *or* 3 women. Count each case and add.

Step 2: 2 women and 1 man: ${}^3C_2 \times {}^4C_1 = 3 \times 4 = 12$.

Step 3: 3 women: ${}^3C_3 = 1$.

Step 4: Total = $12 + 1 = 13$.

>>> Python Solution

Example 3.3.9

CODE

```
1 import math
2 from itertools import combinations
3 two_w = math.comb(3, 2) * math.comb(4, 1)
4 three_w = math.comb(3, 3)
5 print("2 women:", two_w, " 3 women:", three_w, " Total:", two_w + three_w)
6 people = ["M1", "M2", "M3", "M4", "W1", "W2", "W3"]
7 count = sum(1 for c in combinations(people, 3) if sum(p[0] == "W" for p in c) >= 2)
8 print("Check by listing:", count)
```

OUTPUT

```
2 women: 12 3 women: 1 Total: 13
Check by listing: 13
```

Example 3.3.10: Vehicle number plates

A plate has 3 letters (A–Z) followed by 3 digits (0–9), repetition allowed. How many plates are possible?

Solution.

Step 1: Each letter place: 26 choices; each digit place: 10 choices.

Step 2: Total = $26 \times 26 \times 26 \times 10 \times 10 \times 10 = 26^3 \times 10^3$.

Step 3: = $17\,576 \times 1000 = 17\,576\,000$.

>>> Python Solution

Example 3.3.10

CODE

```
1 plates = 26 ** 3 * 10 ** 3
2 print(f"Number plates: {plates:,}")
```

OUTPUT

```
Number plates: 17,576,000
```

Exercise 3.3 (solve by hand, then check with Python)

1. There are 4 roads from A to B and 3 roads from B to C. In how many ways can one travel from A to C via B?
2. Evaluate $6!$, 6P_3 and 7C_3 .
3. How many arrangements of the letters of LEVEL are there?
4. In a meeting of 8 people, everyone shakes hands with everyone else once. How many handshakes?
5. How many 3-digit numbers can be formed from 1, 2, 3, 4, 5 without repetition?
6. In how many ways can 1st, 2nd and 3rd prizes be given to 10 participants?

Multiple Choice Questions – 3.3

1. $0!$ equals:

(a) 0	(b) 1
(c) undefined	(d) ∞
2. ${}^4C_4 =$

(a) 0	(b) 1
(c) 4	(d) 24
3. nC_r is equal to:

(a) nP_r	(b) ${}^nC_{n-r}$
(c) $n!$	(d) $r!$
4. Choosing a captain and vice-captain is a problem of:

(a) combination	(b) permutation
(c) probability	(d) addition
5. The number of ways to arrange 5 books on a shelf is:

(a) 5	(b) 25
(c) 120	(d) 60

Summary of 3.3

- Multiply the number of choices at each stage (multiplicative rule).
- Order matters $\Rightarrow {}^nP_r$; order does not matter $\Rightarrow {}^nC_r$; and ${}^nP_r = r! \cdot {}^nC_r$.
- Divide by $p!$ for each group of identical objects.
- $0! = 1$, ${}^nC_0 = {}^nC_n = 1$.

3.4 Basic Probability Theory

Key Concepts

- **Experiment** – an action with uncertain result (tossing a coin). **Sample space** S – set of all outcomes. **Event** E – a subset of S .
- **Classical probability** (equally likely outcomes): $P(E) = \frac{n(E)}{n(S)}$.
- $0 \leq P(E) \leq 1$; $P(\text{impossible}) = 0$; $P(\text{certain}) = 1$.
- **Complement**: $P(E') = 1 - P(E)$.
- **Addition rule**: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$; if A, B are *mutually exclusive* (cannot happen together), $P(A \cup B) = P(A) + P(B)$.
- **Multiplication rule** for *independent* events: $P(A \cap B) = P(A) \times P(B)$.
- **Conditional probability**: $P(B | A) = \frac{P(A \cap B)}{P(A)}$ (probability of B given A has happened).

A standard deck has 52 cards: 4 suits (hearts, diamonds – red; clubs, spades – black) of 13 cards each; 4 aces, 4 kings, etc.

Example 3.4.1: Rolling a die

A fair die is rolled. Find (a) $P(\text{even})$ (b) $P(\text{number} > 4)$ (c) $P(7)$.

Solution.

Step 1: Sample space $S = \{1, 2, 3, 4, 5, 6\}$; all outcomes equally likely, $n(S) = 6$.

Step 2: (a) Even = $\{2, 4, 6\}$: $P = \frac{3}{6} = \frac{1}{2}$.

Step 3: (b) Greater than 4 = $\{5, 6\}$: $P = \frac{2}{6} = \frac{1}{3}$.

Step 4: (c) 7 is not in S : impossible event, $P = 0$.

>>> Python Solution**Example 3.4.1****CODE**

```
1 from fractions import Fraction
2 S = {1, 2, 3, 4, 5, 6}
3 P = lambda E: Fraction(len(E), len(S))
4 print("P(even) =", P({x for x in S if x % 2 == 0}))
5 print("P(>4)    =", P({x for x in S if x > 4}))
6 print("P(7)     =", P({x for x in S if x == 7}))
```

OUTPUT

```
P(even) = 1/2
P(>4)   = 1/3
P(7)    = 0
```

Example 3.4.2: Mutually exclusive events

A die is rolled. Find $P(1 \text{ or } 6)$.

Solution.

Step 1: Getting 1 and getting 6 cannot happen together: the events are **mutually exclusive**.

Step 2: So simply add: $P(1 \text{ or } 6) = \frac{1}{6} + \frac{1}{6} = \frac{2}{6} = \frac{1}{3}$.

>>> Python Solution**Example 3.4.2****CODE**

```
1 from fractions import Fraction
2 print("P(1 or 6) =", Fraction(1, 6) + Fraction(1, 6))
```

OUTPUT

```
P(1 or 6) = 1/3
```

Example 3.4.3: Cards and the addition rule

A card is drawn from a well-shuffled deck. Find $P(\text{king})$, $P(\text{red face card})$ and $P(\text{heart or king})$.

Solution.

Step 1: There are 4 kings: $P(K) = \frac{4}{52} = \frac{1}{13}$.

Step 2: Face cards are J, Q, K; red suits have $2 \times 3 = 6$ face cards: $P = \frac{6}{52} = \frac{3}{26}$.

Step 3: $P(H) = \frac{13}{52}$ and $P(K) = \frac{4}{52}$, but the king of hearts is in both: $P(H \cap K) = \frac{1}{52}$.

Step 4: $P(H \cup K) = \frac{13}{52} + \frac{4}{52} - \frac{1}{52} = \frac{16}{52} = \frac{4}{13}$.

▷ Why? We subtract the king of hearts so it is not counted twice.

>>> Python Solution

Example 3.4.3

CODE

```
1 from fractions import Fraction
2 ranks = ["A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"]
3 suits = ["hearts", "diamonds", "clubs", "spades"]
4 deck = [(r, s) for r in ranks for s in suits]
5 P = lambda E: Fraction(len(E), len(deck))
6 kings = [c for c in deck if c[0] == "K"]
7 red_face = [c for c in deck if c[0] in "JQK" and c[1] in ("hearts", "diamonds")]
8 heart_or_king = [c for c in deck if c[1] == "hearts" or c[0] == "K"]
9 print("P(king)          =", P(kings))
10 print("P(red face card)=", P(red_face))
11 print("P(heart or king)=", P(heart_or_king))
```

OUTPUT

```
P(king)          = 1/13
P(red face card)= 3/26
P(heart or king)= 4/13
```

Example 3.4.4: Two coins

Two coins are tossed. Find the probability of at least one head.

Solution.

Step 1: List the sample space: $S = \{HH, HT, TH, TT\}$ (4 equally likely outcomes).

Step 2: At least one head = $\{HH, HT, TH\}$: $P = \frac{3}{4}$.

Step 3: Using the complement: the only outcome with no head is TT , so $P = 1 - \frac{1}{4} = \frac{3}{4}$. ✓

>>> Python Solution

Example 3.4.4

CODE

```
1 from itertools import product
2 from fractions import Fraction
3 S = list(product("HT", repeat=2))
4 E = [o for o in S if "H" in o]
5 print("Sample space:", ["".join(o) for o in S])
6 print("P(at least one head) =", Fraction(len(E), len(S)))
```

OUTPUT

```
Sample space: ['HH', 'HT', 'TH', 'TT']
P(at least one head) = 3/4
```

Example 3.4.5: Three coins – “at least one”

Three coins are tossed. Find the probability of at least one head.

Solution.

Step 1: Number of outcomes = $2 \times 2 \times 2 = 8$.

Step 2: The opposite of “at least one head” is “no heads” = $\{TTT\}$: probability $\frac{1}{8}$.

Step 3: $P(\text{at least one head}) = 1 - \frac{1}{8} = \frac{7}{8}$.

▷ Why? Listing the 7 favourable outcomes is slower; the complement is quicker.

>>> Python Solution

Example 3.4.5

CODE

```
1 from itertools import product
2 from fractions import Fraction
3 S = list(product("HT", repeat=3))
4 no_head = [o for o in S if "H" not in o]
5 print("Outcomes:", len(S))
6 print("P(at least one head) = 1 -", Fraction(len(no_head), len(S)), "=",
7       1 - Fraction(len(no_head), len(S)))
```

OUTPUT

```
Outcomes: 8
P(at least one head) = 1 - 1/8 = 7/8
```

Example 3.4.6: Two dice

Two dice are rolled. Find the probability that (a) the sum is 7, (b) a double is thrown, (c) the sum is at least 10.

Solution.

Step 1: $n(S) = 6 \times 6 = 36$ ordered pairs.

Step 2: (a) Sum 7: (1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1): $P = \frac{6}{36} = \frac{1}{6}$.

Step 3: (b) Doubles: (1, 1), ..., (6, 6): $P = \frac{6}{36} = \frac{1}{6}$.

Step 4: (c) Sum 10: 3 ways; sum 11: 2 ways; sum 12: 1 way. Total 6: $P = \frac{6}{36} = \frac{1}{6}$.

>>> Python Solution

Example 3.4.6

CODE

```
1 from itertools import product
2 from fractions import Fraction
3 S = list(product(range(1, 7), repeat=2))
4 P = lambda cond: Fraction(sum(1 for a, b in S if cond(a, b)), len(S))
5 print("P(sum = 7)    =", P(lambda a, b: a + b == 7))
6 print("P(double)     =", P(lambda a, b: a == b))
7 print("P(sum >= 10)  =", P(lambda a, b: a + b >= 10))
```

OUTPUT

```
P(sum = 7)    = 1/6
P(double)     = 1/6
P(sum >= 10)  = 1/6
```

Example 3.4.7: Independent events

A coin is tossed and a die is rolled. Find $P(\text{head and } 6)$.

Solution.

Step 1: The coin does not affect the die: the events are **independent**.

Step 2: Multiply: $P = \frac{1}{2} \times \frac{1}{6} = \frac{1}{12}$.

Step 3: Check by counting: $2 \times 6 = 12$ outcomes, only $(H, 6)$ works: $\frac{1}{12}$. ✓

>>> Python Solution

Example 3.4.7

CODE

```
1 from itertools import product
2 from fractions import Fraction
3 S = list(product("HT", range(1, 7)))
4 E = [o for o in S if o == ("H", 6)]
5 print("By counting :", Fraction(len(E), len(S)))
6 print("By multiplying:", Fraction(1, 2) * Fraction(1, 6))
```

OUTPUT

```
By counting : 1/12
By multiplying: 1/12
```

Example 3.4.8: Drawing without replacement

A bag has 5 red and 3 blue balls. Find (a) $P(\text{red})$, (b) $P(\text{not red})$, (c) the probability that two balls drawn one after another *without replacement* are both red.

Solution.

Step 1: (a) $\frac{5}{8}$. (b) $1 - \frac{5}{8} = \frac{3}{8}$.

Step 2: (c) First ball red: $\frac{5}{8}$. After removing it, 4 red remain among 7 balls: $\frac{4}{7}$.

▷ Why? Without replacement the bag changes, so the second probability changes.

Step 3: $P(\text{both red}) = \frac{5}{8} \times \frac{4}{7} = \frac{20}{56} = \frac{5}{14}$.

>>> Python Solution

Example 3.4.8

CODE

```
1 from fractions import Fraction
2 red, blue = 5, 3
3 total = red + blue
4 print("P(red)      =", Fraction(red, total))
5 print("P(not red)   =", 1 - Fraction(red, total))
6 both = Fraction(red, total) * Fraction(red - 1, total - 1)
7 print("P(both red) =", both)
```

OUTPUT

```
P(red)      = 5/8
P(not red)   = 3/8
P(both red)  = 5/14
```

Example 3.4.9: Conditional probability from a table

A class has 25 boys and 15 girls; 10 boys and 9 girls wear glasses. A student chosen at random wears glasses. Find the probability that the student is a girl.

Solution.

Step 1: Organise the data in a two-way table:

	Glasses	No glasses	Total
Boys	10	15	25
Girls	9	6	15
Total	19	21	40

Step 2: We are told the student wears glasses, so only the 19 students in that column matter.

Step 3: $P(\text{girl} \mid \text{glasses}) = \frac{9}{19} \approx 0.47$.

Step 4: With the formula: $\frac{P(G \cap \text{glasses})}{P(\text{glasses})} = \frac{9/40}{19/40} = \frac{9}{19} \cdot \checkmark$

>>> Python Solution

Example 3.4.9

CODE

```
1 from fractions import Fraction
2 table = {"Boys": {"glasses": 10, "none": 15}, "Girls": {"glasses": 9, "none": 6}}
3 glasses = sum(row["glasses"] for row in table.values())
4 print("Students with glasses:", glasses)
5 print("P(girl | glasses) =", Fraction(table["Girls"]["glasses"], glasses),
6       f"= {table['Girls']['glasses'] / glasses:.2f}")
```

OUTPUT

```
Students with glasses: 19
P(girl | glasses) = 9/19 = 0.47
```

Example 3.4.10: Experimental probability

A coin is tossed 200 times and shows 108 heads. Find the experimental probability of a head and compare it with theory.

Solution.

Step 1: Experimental (relative frequency) probability = $\frac{108}{200} = 0.54$.

Step 2: Theoretical probability = $\frac{1}{2} = 0.5$.

Step 3: The small difference is due to chance; with more tosses the relative frequency tends to get closer to 0.5.

>>> Python Solution

Example 3.4.10

CODE

```
1 import random
2 random.seed(42)
3 for n in [20, 200, 2000, 20000]:
4     heads = sum(random.choice("HT") == "H" for _ in range(n))
5     print(f"{n}>6} tosses: P(head) approx {heads / n:.3f}")
```

OUTPUT

```
20 tosses: P(head) approx 0.750
200 tosses: P(head) approx 0.535
2000 tosses: P(head) approx 0.478
20000 tosses: P(head) approx 0.505
```

Exercise 3.4 (solve by hand, then check with Python)

1. A die is rolled. Find the probability of getting a prime number.
2. A card is drawn from a deck. Find the probability that it is an ace.
3. Two coins are tossed. Find the probability of exactly one head.
4. Two dice are rolled. Find the probability that the sum is 10.
5. The probability of rain tomorrow is 0.3. What is the probability of no rain?
6. If $P(A) = 0.5$, $P(B) = 0.4$ and $P(A \cap B) = 0.2$, find $P(A \cup B)$.
7. A bag has 4 white and 6 black balls. Two balls are drawn *with* replacement. Find $P(\text{both black})$.

Multiple Choice Questions – 3.4

1. The probability of an impossible event is:
 (a) 0 (b) 1
 (c) 0.5 (d) -1
2. $P(E) + P(E') =$
 (a) 0 (b) 0.5
 (c) 1 (d) 2
3. The probability that the sum of two dice is 13 is:
 (a) $\frac{1}{36}$ (b) $\frac{1}{13}$
 (c) 0 (d) 1
4. Number of outcomes when 3 coins are tossed:
 (a) 3 (b) 6
 (c) 8 (d) 9
5. If A and B are independent with $P(A) = 0.5$, $P(B) = 0.2$, then $P(A \cap B) =$
 (a) 0.7 (b) 0.1
 (c) 0.3 (d) 0.25

Summary of 3.4

- $P(E) = \frac{\text{favourable outcomes}}{\text{total outcomes}}$, always between 0 and 1.
- “Not” $\Rightarrow 1 - P$; “or” $\Rightarrow$ add (subtract overlap); “and” (independent) $\Rightarrow$ multiply.
- “At least one” is often easiest as $1 - P(\text{none})$.
- Without replacement, the second probability changes.

3.5 Introduction to Random Variables and their Probability Distributions

Key Concepts

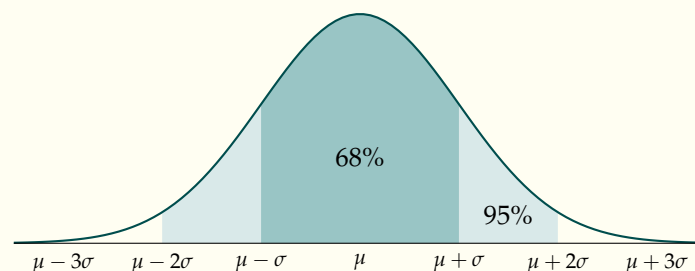
- A **random variable** X assigns a number to each outcome of an experiment (e.g. X = number of heads in 2 tosses). **Discrete** RV: countable values; **continuous** RV: any value in an interval (height, time).
- A **probability distribution** lists every value of X with its probability. It is valid if every $P(x) \geq 0$ and $\sum P(x) = 1$.
- **Expected value (mean):** $\mu = E(X) = \sum x P(x)$ – the long-run average.
- **Variance:** $\sigma^2 = E(X^2) - \mu^2 = \sum x^2 P(x) - \mu^2$; SD $\sigma = \sqrt{\sigma^2}$.

Binomial and Normal Distributions

Binomial distribution – n independent trials, each with two outcomes (success with probability p , failure $q = 1 - p$):

$$P(X = r) = {}^nC_r p^r q^{n-r}, \quad r = 0, 1, \dots, n; \quad \text{mean} = np, \quad \text{variance} = npq.$$

Normal distribution – a bell-shaped, symmetric continuous distribution with mean μ and SD σ . **Empirical rule (68–95–99.7):** about 68% of values lie within $\mu \pm \sigma$, 95% within $\mu \pm 2\sigma$ and 99.7% within $\mu \pm 3\sigma$. **z-score:** $z = \frac{x - \mu}{\sigma}$.



Example 3.5.1: Distribution of the number of heads

Two coins are tossed and X = number of heads. Find the distribution, $E(X)$, variance and SD.

Solution.

Step 1: List outcomes and the value of X for each: $TT \rightarrow 0$, $HT \rightarrow 1$, $TH \rightarrow 1$, $HH \rightarrow 2$.

Step 2: Distribution:

x	0	1	2
$P(x)$	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{4}$

 (sum = 1 ✓)

Step 3: $E(X) = \sum xP(x) = 0 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} = 1$.

Step 4: $E(X^2) = \sum x^2P(x) = 0 + 1 \cdot \frac{1}{2} + 4 \cdot \frac{1}{4} = 1.5$.

Step 5: Variance = $E(X^2) - [E(X)]^2 = 1.5 - 1 = 0.5$; SD = $\sqrt{0.5} \approx 0.71$.

>>> Python Solution

Example 3.5.1

CODE

```
1 from fractions import Fraction
```

```

2 from itertools import product
3 outcomes = list(product("HT", repeat=2))
4 dist = {}
5 for o in outcomes:
6     x = o.count("H")
7     dist[x] = dist.get(x, 0) + Fraction(1, len(outcomes))
8 print("Distribution:", {k: str(v) for k, v in sorted(dist.items())})
9 E = sum(x * p for x, p in dist.items())
10 E2 = sum(x**2 * p for x, p in dist.items())
11 print("E(X) =", E, " E(X^2) =", E2, " Var =", E2 - E**2, f" SD = {float(E2 - E**2)
    ↪ ** 0.5:.2f}")

```

OUTPUT

```

Distribution: {0: '1/4', 1: '1/2', 2: '1/4'}
E(X) = 1 E(X^2) = 3/2 Var = 1/2 SD = 0.71

```

Example 3.5.2: Is it a valid distribution?

Can $P(x)$: 0.2, 0.5, 0.4 be a probability distribution? What about 0.3, 0.8, -0.1 ?

Solution.

Step 1: Check two conditions: every $P(x) \geq 0$, and $\sum P(x) = 1$.

Step 2: First: $0.2 + 0.5 + 0.4 = 1.1 \neq 1 \Rightarrow$ **not valid**.

Step 3: Second: sum is 1, but $-0.1 < 0 \Rightarrow$ **not valid** (a probability cannot be negative).

>>> Python Solution**Example 3.5.2****CODE**

```

1 def valid(probs):
2     return all(p >= 0 for p in probs) and abs(sum(probs) - 1) < 1e-9
3
4 print([0.2, 0.5, 0.4], "valid?", valid([0.2, 0.5, 0.4]))
5 print([0.3, 0.8, -0.1], "valid?", valid([0.3, 0.8, -0.1]))

```

OUTPUT

```

[0.2, 0.5, 0.4] valid? False
[0.3, 0.8, -0.1] valid? False

```

Example 3.5.3: Finding an unknown constant

$P(X = x) = kx^2$ for $x = 1, 2, 3$. Find k and $P(X \geq 2)$.

Solution.

Step 1: Probabilities: $k(1)^2 = k$, $k(2)^2 = 4k$, $k(3)^2 = 9k$.

Step 2: They must add to 1: $k + 4k + 9k = 14k = 1 \Rightarrow k = \frac{1}{14}$.

Step 3: $P(X \geq 2) = P(2) + P(3) = \frac{4}{14} + \frac{9}{14} = \frac{13}{14}$.

>>> Python Solution**Example 3.5.3****CODE**

```

1 from sympy import symbols, Eq, solve, Rational
2 k = symbols("k")

```

```

3 k_val = solve(Eq(sum(k * x**2 for x in [1, 2, 3]), 1), k)[0]
4 print("k =", k_val)
5 print("P(X >= 2) =", k_val * 2**2 + k_val * 3**2)

```

OUTPUT

```

k = 1/14
P(X >= 2) = 13/14

```

Example 3.5.4: Expected value of a die

Find the mean and variance of the score on a fair die.

Solution.

Step 1: Each score $1, \dots, 6$ has probability $\frac{1}{6}$.

Step 2: $E(X) = \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3.5$.

▷ Why? The expected value need not be a possible outcome – it is a long-run average.

Step 3: $E(X^2) = \frac{1+4+9+16+25+36}{6} = \frac{91}{6} \approx 15.17$.

Step 4: Variance = $\frac{91}{6} - 3.5^2 = \frac{91}{6} - \frac{49}{4} = \frac{35}{12} \approx 2.92$.

>>> Python Solution**Example 3.5.4****CODE**

```

1 from fractions import Fraction
2 p = Fraction(1, 6)
3 E = sum(x * p for x in range(1, 7))
4 E2 = sum(x**2 * p for x in range(1, 7))
5 print("E(X) =", E, "=", float(E))
6 print("Var  =", E2 - E**2, f"= {float(E2 - E**2):.2f}")

```

OUTPUT

```

E(X) = 7/2 = 3.5
Var  = 35/12 = 2.92

```

Example 3.5.5: Expected value of a game

A game costs Rs.20 to play. With probability 0.1 you win Rs.100; otherwise you win nothing. Is it a fair game?

Solution.

Step 1: Define net gain X : win $\Rightarrow 100 - 20 = +80$ with $P = 0.1$; lose $\Rightarrow -20$ with $P = 0.9$.

Step 2: $E(X) = 80(0.1) + (-20)(0.9) = 8 - 18 = -10$.

Step 3: On average the player **loses Rs.10 per game**; a fair game would have $E(X) = 0$.

>>> Python Solution**Example 3.5.5****CODE**

```

1 gains = [80, -20]
2 probs = [0.1, 0.9]
3 E = sum(g * p for g, p in zip(gains, probs))
4 print("Expected gain per game: Rs.", round(E, 2))
5 print("Fair game?", round(E, 2) == 0)

```

OUTPUT

```
Expected gain per game: Rs. -10.0
Fair game? False
```

Example 3.5.6: Binomial – coins

A fair coin is tossed 4 times. Find the probability of exactly 2 heads, and the mean and variance of the number of heads.

Solution.

Step 1: Check binomial conditions: fixed $n = 4$ trials, two outcomes, constant $p = \frac{1}{2}$, independent tosses. ✓

Step 2: ${}^4C_2 = 6$ counts the arrangements of 2 heads in 4 tosses (HHTT, HTHT, ...).

Step 3: $P(X = 2) = {}^4C_2 \left(\frac{1}{2}\right)^2 \left(\frac{1}{2}\right)^2 = 6 \times \frac{1}{16} = \frac{3}{8} = 0.375$.

Step 4: Mean = $np = 2$; variance = $npq = 4 \times \frac{1}{2} \times \frac{1}{2} = 1$.

>>> Python Solution**Example 3.5.6****CODE**

```
1 from math import comb
2 n, p, r = 4, 0.5, 2
3 prob = comb(n, r) * p**r * (1 - p)**(n - r)
4 print("4C2 =", comb(4, 2))
5 print("P(X = 2) =", prob)
6 print("Mean =", n * p, " Variance =", n * p * (1 - p))
```

OUTPUT

```
4C2 = 6
P(X = 2) = 0.375
Mean = 2.0 Variance = 1.0
```

Example 3.5.7: Full binomial distribution

Write the distribution of the number of heads in 3 tosses of a fair coin and check its mean.

Solution.

Step 1: $n = 3$, $p = q = \frac{1}{2}$, so $P(X = r) = {}^3C_r \left(\frac{1}{2}\right)^3 = \frac{{}^3C_r}{8}$.

Step 2: ${}^3C_0, {}^3C_1, {}^3C_2, {}^3C_3 = 1, 3, 3, 1$, giving $P = \frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}$.

Step 3: $E(X) = 0 \cdot \frac{1}{8} + 1 \cdot \frac{3}{8} + 2 \cdot \frac{3}{8} + 3 \cdot \frac{1}{8} = \frac{12}{8} = 1.5$.

Step 4: Formula: $np = 3 \times \frac{1}{2} = 1.5$. ✓

>>> Python Solution**Example 3.5.7****CODE**

```
1 from math import comb
2 from fractions import Fraction
3 n = 3
4 dist = {r: Fraction(comb(n, r), 2**n) for r in range(n + 1)}
5 print({r: str(p) for r, p in dist.items()})
6 print("Mean by table :", float(sum(r * p for r, p in dist.items())))
```

```
7 print("Mean by formula:", n * 0.5)
```

OUTPUT

```
{0: '1/8', 1: '3/8', 2: '3/8', 3: '1/8'}
Mean by table : 1.5
Mean by formula: 1.5
```

Example 3.5.8: Binomial – defective items

10% of items from a machine are defective. In a sample of 5 items, find $P(\text{none defective})$ and $P(\text{exactly 1 defective})$.

Solution.

Step 1: “Success” = defective: $n = 5$, $p = 0.1$, $q = 0.9$.

Step 2: $P(X = 0) = {}^5C_0(0.1)^0(0.9)^5 = 0.9^5 \approx 0.5905$.

Step 3: $P(X = 1) = {}^5C_1(0.1)^1(0.9)^4 = 5 \times 0.1 \times 0.6561 \approx 0.3281$.

Step 4: So there is about a 59% chance of no defective item.

>>> Python Solution**Example 3.5.8****CODE**

```
1 from scipy import stats
2 n, p = 5, 0.1
3 print(f"P(no defective) = {stats.binom.pmf(0, n, p):.5f}")
4 print(f"P(exactly 1) = {stats.binom.pmf(1, n, p):.5f}")
```

OUTPUT

```
P(no defective) = 0.59049
P(exactly 1) = 0.32805
```

Example 3.5.9: Binomial – “at least one”

A student guesses 3 MCQs, each with 5 options (only one correct). Find the probability of at least one correct answer.

Solution.

Step 1: $n = 3$, $p = \frac{1}{5} = 0.2$, $q = 0.8$.

Step 2: $P(\text{none correct}) = 0.8^3 = 0.512$.

Step 3: $P(\text{at least one}) = 1 - 0.512 = 0.488$.

>>> Python Solution**Example 3.5.9****CODE**

```
1 n, p = 3, 0.2
2 none = (1 - p) ** n
3 print(f"P(none correct) = {none:.3f}")
4 print(f"P(at least one) = {1 - none:.3f}")
```

OUTPUT

```
P(none correct) = 0.512
P(at least one) = 0.488
```

Example 3.5.10: Normal distribution – empirical rule

Heights of students are normally distributed with mean 165 cm and SD 5 cm. Interpret.

Solution.

Step 1: $\mu \pm \sigma = 160$ to 170 cm: about 68% of students.

Step 2: $\mu \pm 2\sigma = 155$ to 175 cm: about 95%.

Step 3: $\mu \pm 3\sigma = 150$ to 180 cm: about 99.7% – almost everyone.

Step 4: A student 175 cm tall has $z = \frac{175-165}{5} = 2$: taller than about 97.5% of students.

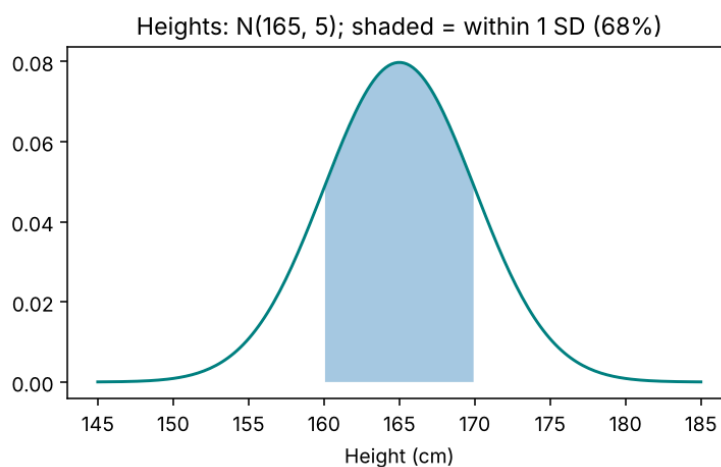
▷ Why? 95% lie within $\pm 2\sigma$, leaving 2.5% in each tail.

>>> Python Solution**Example 3.5.10****CODE**

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import stats
4 mu, sigma = 165, 5
5 for k in [1, 2, 3]:
6     lo, hi = mu - k*sigma, mu + k*sigma
7     share = stats.norm.cdf(hi, mu, sigma) - stats.norm.cdf(lo, mu, sigma)
8     print(f"{lo}-{hi} cm: {share:.1%}")
9 x = np.linspace(145, 185, 200)
10 plt.figure(figsize=(6, 3.2))
11 plt.plot(x, stats.norm.pdf(x, mu, sigma), color="teal")
12 plt.fill_between(x, stats.norm.pdf(x, mu, sigma), where=(x >= 160) & (x <= 170),
13     ↪ alpha=0.4)
14 plt.title("Heights: N(165, 5); shaded = within 1 SD (68%)"); plt.xlabel("Height
15     ↪ (cm)")
16 plt.show()
```

OUTPUT

```
160-170 cm: 68.3%
155-175 cm: 95.4%
150-180 cm: 99.7%
```



Example 3.5.11: Using z-scores

Exam marks are normal with mean 60 and SD 8. About what percentage of students score above 76? Below 52?

Solution.

Step 1: $z = \frac{76-60}{8} = 2$: two SDs above the mean. Above $z = 2$ is about $\frac{100-95}{2} = 2.5\%$.

Step 2: $z = \frac{52-60}{8} = -1$: one SD below. Within ± 1 SD is 68%, so each tail has $\frac{100-68}{2} = 16\%$.

Step 3: About 16% score below 52.

>>> Python Solution**Example 3.5.11****CODE**

```
1 from scipy import stats
2 mu, sigma = 60, 8
3 for score in [76, 52]:
4     z = (score - mu) / sigma
5     print(f"Score {score}: z = {z:+.0f}")
6 print(f"P(above 76) = {1 - stats.norm.cdf(76, mu, sigma):.1%} (rule says about
   ↪ 2.5%)")
7 print(f"P(below 52) = {stats.norm.cdf(52, mu, sigma):.1%} (rule says about 16%)")
```

OUTPUT

```
Score 76: z = +2
Score 52: z = -1
P(above 76) = 2.3% (rule says about 2.5%)
P(below 52) = 15.9% (rule says about 16%)
```

Exercise 3.5 (solve by hand, then check with Python)

1. X takes values 1, 2, 3 with probabilities 0.2, 0.5, 0.3. Find $E(X)$ and $\text{Var}(X)$.
2. X takes values 1, 2, 3, 4 with probabilities $k, 2k, 3k, 4k$. Find k .
3. A fair coin is tossed 3 times. Find the probability of 3 heads.
4. For a binomial distribution with $n = 6$ and $p = \frac{1}{3}$, find the mean and variance.
5. Marks are normal with mean 70 and SD 10. Between which two marks do about 95% of students lie?

Multiple Choice Questions – 3.5

1. The sum of all probabilities in a probability distribution is:

(a) 0	(b) 1
(c) 100	(d) any number
2. The mean of a binomial distribution is:

(a) npq	(b) np
(c) nq	(d) $\sqrt{npq}$
3. In a normal distribution, about what percentage lies within $\mu \pm \sigma$?

(a) 50%	(b) 68%
(c) 95%	(d) 99.7%

4. The number of heads in 10 tosses is a:
- | | |
|-------------------|-----------------|
| (a) continuous RV | (b) discrete RV |
| (c) constant | (d) parameter |
5. A fair game has expected gain:
- | | |
|--------------|--------------|
| (a) positive | (b) negative |
| (c) zero | (d) one |

Summary of 3.5

- A random variable turns outcomes into numbers; its distribution gives each value's probability ($\sum P = 1$).
- $E(X) = \sum xP(x)$ is the long-run average; $\text{Var}(X) = \sum x^2P(x) - \mu^2$.
- Binomial: fixed n independent trials, $P(X = r) = {}^nC_r p^r q^{n-r}$, mean np , variance npq .
- Normal: bell curve; 68–95–99.7 rule; z-score measures how many SDs a value is from the mean.

3.6 Quantitative Reasoning Exercises Using Statistical Concepts

Thinking clearly with data

- Which average represents the data best? (median when there are outliers)
- How spread out are the data? (an average without a spread can mislead)
- Who was sampled, and is the sample biased?
- What is the expected (long-run) result of a decision involving chance?

Example 3.6.1: A misleading average

Monthly salaries (thousand rupees) in a small firm: 30, 30, 35, 40, 300 (the owner). The firm advertises "average salary Rs. 87 000". Is this fair?

Solution.

Step 1: Mean = $\frac{30 + 30 + 35 + 40 + 300}{5} = \frac{435}{5} = 87$ thousand – the claim is arithmetically true.

Step 2: Median = 35 thousand (middle of ordered values).

Step 3: Four of five employees earn far less than 87 thousand; the single outlier pulls the mean up.

Step 4: The **median** (Rs. 35 000) describes a typical salary much better.

>>> Python Solution

Example 3.6.1

CODE

```
1 import statistics as st
2 salaries = [30, 30, 35, 40, 300]           # thousand rupees
3 print("Mean :", st.mean(salaries), "thousand")
4 print("Median:", st.median(salaries), "thousand <- more typical")
```

OUTPUT

```
Mean : 87 thousand
Median: 35 thousand <- more typical
```

Example 3.6.2: Comparing two sections

Section A: mean 62, SD 8. Section B: mean 62, SD 15. Interpret.

Solution.

Step 1: Same mean: on average the sections did equally well.

Step 2: B's SD is almost twice A's: B's marks are much more spread out.

Step 3: So B has both very strong and very weak students; A is more uniform. B may need extra help for weak students.

>>> Python Solution

Example 3.6.2

CODE

```
1 import statistics as st
2 # Small illustrative sections with the same mean but different spread
3 A = [54, 54, 70, 70]
4 B = [47, 47, 77, 77]
5 for name, d in [("A", A), ("B", B)]:
6     print(f"Section {name}: mean {st.mean(d)}, SD {st.pstdev(d)}, "
7           f"range {min(d)}-{max(d)}")
8 print("Same mean, but B is far more spread out.")
```

OUTPUT

```
Section A: mean 62, SD 8.0, range 54-70
Section B: mean 62, SD 15.0, range 47-77
Same mean, but B is far more spread out.
```

Example 3.6.3: Is the raffle worth it?

1000 raffle tickets are sold at Rs.50 each; there is one prize of Rs.20 000. Find the expected gain of buying one ticket.

Solution.

Step 1: $P(\text{win}) = \frac{1}{1000}$; expected winnings = $20\,000 \times \frac{1}{1000} = \text{Rs. } 20$.

Step 2: Expected gain = $20 - 50 = -\text{Rs. } 30$ per ticket.

Step 3: On average a buyer loses Rs. 30; the organiser keeps $50\,000 - 20\,000 = \text{Rs. } 30\,000$.

>>> Python Solution

Example 3.6.3

CODE

```
1 tickets, price, prize = 1000, 50, 20000
2 expected_win = prize / tickets
3 print("Expected winnings: Rs.", expected_win)
4 print("Expected gain : Rs.", expected_win - price)
5 print("Organiser keeps : Rs.", tickets * price - prize)
```

OUTPUT

```
Expected winnings: Rs. 20.0
Expected gain : Rs. -30.0
```

Organiser keeps : Rs. 30000

Example 3.6.4: Password strength (counting)

Compare the number of 6-character passwords made of (a) digits only, (b) lowercase letters only.

Solution.

Step 1: (a) Each place has 10 choices: $10^6 = 1\,000\,000$.

Step 2: (b) Each place has 26 choices: $26^6 = 308\,915\,776$.

Step 3: Ratio $\approx \frac{308\,915\,776}{1\,000\,000} \approx 309$: letters give about 309 times more possibilities.

>>> Python Solution

Example 3.6.4

CODE

```
1 digits = 10 ** 6
2 letters = 26 ** 6
3 print(f"Digits only : {digits:,}")
4 print(f"Letters only: {letters:,}")
5 print(f"Letters give about {letters / digits:.0f} times more")
```

OUTPUT

```
Digits only : 1,000,000
Letters only: 308,915,776
Letters give about 309 times more
```

Example 3.6.5: Comparing performance with z-scores

Ali scored 75 in Maths (class mean 60, SD 10) and 80 in English (class mean 75, SD 10). In which subject did he do relatively better?

Solution.

Step 1: Raw marks cannot be compared directly because the classes performed differently.

Step 2: $z_{\text{Maths}} = \frac{75-60}{10} = 1.5$; $z_{\text{English}} = \frac{80-75}{10} = 0.5$.

Step 3: He is 1.5 SDs above average in Maths but only 0.5 in English: **relatively better in Maths.**

>>> Python Solution

Example 3.6.5

CODE

```
1 z = lambda x, mu, sd: (x - mu) / sd
2 print("Maths    z =", z(75, 60, 10))
3 print("English z =", z(80, 75, 10))
```

OUTPUT

```
Maths    z = 1.5
English z = 0.5
```

Example 3.6.6: Percent versus percentage points

A college's pass rate rose from 40% to 50%. A report says "pass rate up 10%"; another says "up 25%". Which is right?

Solution.

Step 1: The difference $50\% - 40\%$ is **10 percentage points**.

Step 2: The relative change is $\frac{50-40}{40} \times 100 = 25\%$.

Step 3: Both are correct if stated precisely: "up 10 percentage points" or "up 25%". Saying "up 10%" is ambiguous.

>>> Python Solution**Example 3.6.6****CODE**

```
1 old, new = 40, 50
2 print("Change in percentage points:", new - old)
3 print(f"Relative change: {(new - old) / old:.0%}")
```

OUTPUT

```
Change in percentage points: 10
Relative change: 25%
```

Example 3.6.7: Correlation is not causation

In a city, months with high ice-cream sales also have more drowning accidents. Does ice cream cause drowning?

Solution.

Step 1: The two quantities rise and fall together: they are *correlated*.

Step 2: A third factor – hot weather – increases both ice-cream sales and swimming.

Step 3: So there is no evidence that one causes the other.

▷ *Why? Before claiming cause and effect, always look for a hidden (confounding) factor.*

>>> Python Solution**Example 3.6.7****CODE**

```
1 import statistics as st
2 temperature = [10, 14, 20, 27, 33, 36, 35, 33, 29, 22, 15, 11] # monthly, deg C
3 ice_cream = [20, 28, 45, 70, 95, 110, 108, 98, 80, 52, 30, 22] # sales (hundreds)
4 drownings = [1, 1, 2, 4, 6, 8, 7, 6, 5, 3, 1, 1]
5 print(f"ice cream vs drownings : r = {st.correlation(ice_cream, drownings):.2f}")
6 print(f"temperature vs ice cream: r = {st.correlation(temperature,
    ↪ ice_cream):.2f}")
7 print(f"temperature vs drownings: r = {st.correlation(temperature,
    ↪ drownings):.2f}")
8 print("Hot weather drives both -> correlation, not causation.")
```

OUTPUT

```
ice cream vs drownings : r = 0.99
temperature vs ice cream: r = 0.99
temperature vs drownings: r = 0.98
Hot weather drives both -> correlation, not causation.
```

Example 3.6.8: A misleading graph

A bar chart shows Brand A at 96% and Brand B at 98% customer satisfaction, with the y -axis starting at 95%. B's bar looks three times as tall. Is B three times better?

Solution.

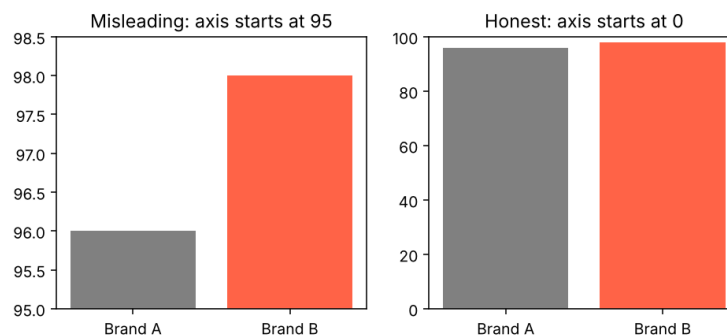
Step 1: Above the cut-off at 95, A's bar is 1 unit tall and B's is 3 units: hence "three times".

Step 2: The real difference is only $98 - 96 = 2$ percentage points.

Step 3: The truncated axis exaggerates the difference; bars should start at zero.

>>> Python Solution**Example 3.6.8****CODE**

```
1 import matplotlib.pyplot as plt
2 brands, values = ["Brand A", "Brand B"], [96, 98]
3 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3.2))
4 ax1.bar(brands, values, color=["gray", "tomato"]); ax1.set_ylim(95, 98.5)
5 ax1.set_title("Misleading: axis starts at 95")
6 ax2.bar(brands, values, color=["gray", "tomato"]); ax2.set_ylim(0, 100)
7 ax2.set_title("Honest: axis starts at 0")
8 plt.show()
```

OUTPUT (FIGURE)**Example 3.6.9: Reading a medical test result**

A disease affects 1% of 2000 people. A test detects 90% of sick people but also gives a positive result for 5% of healthy people. If a person tests positive, how likely is he to be sick?

Solution.

Step 1: Sick people: 1% of 2000 = 20; they test positive: 90% of 20 = 18.

Step 2: Healthy people: 1980; false positives: 5% of 1980 = 99.

Step 3: Total positives = $18 + 99 = 117$.

Step 4: $P(\text{sick} \mid \text{positive}) = \frac{18}{117} \approx 0.15$, only about 15%.

▷ Why? When a condition is rare, most positive results come from the large healthy group.

>>> Python Solution**Example 3.6.9****CODE**

```
1 people = 2000
2 sick = people * 0.01
```

```

3 healthy = people - sick
4 true_pos = 0.90 * sick
5 false_pos = 0.05 * healthy
6 p = true_pos / (true_pos + false_pos)
7 print("True positives :", true_pos)
8 print("False positives:", false_pos)
9 print(f"P(sick | positive) = {p:.2f}")

```

OUTPUT

```

True positives : 18.0
False positives: 99.0
P(sick | positive) = 0.15

```

Example 3.6.10: Margin of error in a poll

A random poll of 400 people finds 52% support a proposal. Can we say a majority supports it?

Solution.

Step 1: A rough margin of error for a sample proportion is $\frac{1}{\sqrt{n}} = \frac{1}{\sqrt{400}} = \frac{1}{20} = 5\%$.

Step 2: So the true support is likely between $52 - 5 = 47\%$ and $52 + 5 = 57\%$.

Step 3: This range includes values below 50%, so we **cannot** confidently claim a majority.

>>> Python Solution**Example 3.6.10****CODE**

```

1 import math
2 n, p_hat = 400, 0.52
3 margin = 1 / math.sqrt(n)
4 print(f"Margin of error about {margin:.0%}")
5 print(f"Likely range: {p_hat - margin:.0%} to {p_hat + margin:.0%}")
6 print("Majority certain?", p_hat - margin > 0.5)

```

OUTPUT

```

Margin of error about 5%
Likely range: 47% to 57%
Majority certain? False

```

Exercise 3.6 (solve by hand, then check with Python)

1. House prices (lakh rupees) in a street: 40, 45, 50, 55, 400. Find the mean and median. Which better represents a typical house?
2. 500 lottery tickets are sold at Rs. 100 each with one prize of Rs. 25 000. Find the expected gain per ticket.
3. Sara scored 68 in Physics (mean 60, SD 4) and 75 in Chemistry (mean 70, SD 10). In which subject was she relatively better?
4. A survey of 200 people at a gym finds that 80% exercise daily. Can we say 80% of the city exercises daily? Explain.
5. How many 4-digit PINs are possible if the first digit cannot be 0 and digits may repeat?

Multiple Choice Questions – 3.6

1. For income data with a few very rich people, the best average is usually the:
(a) mean (b) median
(c) range (d) SD
2. A z-score of 0 means the value is:
(a) zero (b) equal to the mean
(c) one SD above the mean (d) an outlier
3. Expected value of a game with gains +10 ($P = 0.5$) and -10 ($P = 0.5$) is:
(a) 10 (b) -10
(c) 0 (d) 5
4. A poll taken only among university students to predict a national election is:
(a) a census (b) unbiased
(c) biased (d) stratified

Summary of 3.6

- Use the median when outliers distort the mean; report spread along with averages.
- Expected value shows the long-run gain or loss of decisions involving chance.
- z-scores compare values from different distributions.
- Always question how the data were collected before accepting a conclusion.

Answer Key

Unit 0: Python Fundamentals and Google Colab

0.1 Getting Started

Exercise:

1. e.g. `print("Ali Khan")` and `print("GBPG College, Quetta")` 2. 42 3. Shift + Enter
 4. Print must be `print` (Python is case-sensitive): `print("Quetta")` 5. Colab Notebooks
- MCQs: 1 (b) 2 (b) 3 (b) 4 (a) 5 (b)

0.2 Variables, Data Types and Arithmetic

Exercise:

1. 1024, 4, 1 2. `float(2.0)`: / always gives a float
 3. `print(round(3.14159 * 5**2, 2))` gives 78.54
 4. Valid: `total_marks`, `Price`; invalid: `2nd_test` (starts with a digit), `my marks` (space) 5. 9
- MCQs: 1 (b) 2 (a) 3 (c) 4 (c) 5 (b)

0.3 Strings and Formatting

Exercise:

1. `QUANTITATIVE REASONING`, length 22 2. `print(f"Total = Rs. {bill}")`
 3. `f"{2/3:.3f}"` gives 0.667 4. `f"{0.35:.0%}"` gives 35%
- MCQs: 1 (b) 2 (a) 3 (c) 4 (b)

0.4 Lists, Tuples, Dictionaries and Sets

Exercise:

1. 9, 7, [2, 5, 7, 9] 2. 25.0
 3. e.g. `t = {"Quetta": 12, "Karachi": 30, "Lahore": 24}; print(t["Quetta"])`
 4. `P | Q = {1,2,3,4}`, `P & Q = {3}`
- MCQs: 1 (a) 2 (c) 3 (b) 4 (b)

0.5 Conditions

Exercise:

1. True, False, False
 2. `if n > 0: print("Positive")` `elif n < 0: print("Negative")` `else: print("Zero")`
 3. False, True, False 4. `if year % 4 == 0: print("divisible")` – 2028 is divisible by 4
- MCQs: 1 (b) 2 (b) 3 (b) 4 (b)

0.6 Loops

Exercise:

1. 2, 4, 6, 8 2. `for i in range(1, 6): print(i**3)` gives 1, 8, 27, 64, 125
 3. 100 4. `[x for x in range(1, 31) if x % 3 == 0]` gives 3, 6, ..., 30
- MCQs: 1 (b) 2 (a) 3 (b) 4 (b)

0.7 Functions

Exercise:

1. `def rect_perimeter(l, w): return 2*(l + w); result 40` 2. `g = lambda x: x**2 + 1; g(3) is 10`
 3. None 4. `def is_even(n): return n % 2 == 0`
MCQs: 1 (b) 2 (b) 3 (b) 4 (a)

0.8 Python Libraries for Mathematics

Exercise:

1. 15.0, 12, 5040 2. 5/12 3. 8.5 4. 6
MCQs: 1 (a) 2 (b) 3 (b) 4 (a)

0.9 Algebra with SymPy

Exercise:

1. $(x + 3)(x + 4)$ 2. [5] 3. $[2 - \sqrt{3}, \sqrt{3} + 2]$ 4. $4x^2 - 4x + 1$
MCQs: 1 (b) 2 (b) 3 (b) 4 (a)

0.10 Matplotlib, NumPy and SciPy

Exercise:

1. `np.array([1, 2, 3, 4])**2` gives [1 4 9 16]
 2. `x = np.linspace(-3, 3, 100); plt.plot(x, x**2); plt.show()`
 3. `plt.bar(days, temps); plt.show()` (with your own lists) 4. 0.125
MCQs: 1 (b) 2 (b) 3 (b) 4 (a)

Unit 1: Numerical Literacy

1.1 Number System and Arithmetic

Exercise:

1. -5: integer, rational; $\frac{3}{7}$: rational; $\sqrt{9} = 3$: natural (also whole, integer, rational); $\sqrt{5}$: irrational; 0: whole, integer, rational.
 2. $20 - 12 + 4 = 12$ 3. $\frac{8}{12} - \frac{3}{12} = \frac{5}{12}$ 4. HCF = 12, LCM = 72
 5. $\frac{2}{3}$ 6. 3.2×10^6
MCQs: 1 (c) 2 (b) 3 (b) 4 (a) 5 (a)

1.2 Units, Area, Perimeter and Volume

Exercise:

1. 350 cm 2. 25 m/s 3. $P = 36$ cm, $A = 81$ cm² 4. 30 cm²
 5. $C = 44$ cm, $A = 154$ cm² 6. 64 cm³ 7. 25.4 cm
MCQs: 1 (c) 2 (c) 3 (a) 4 (b) 5 (b)

1.3 Rates, Ratios, Proportions and Percentages

Exercise:

1. 3 : 4 2. Rs. 200, Rs. 400, Rs. 600 3. $x = 3$ 4. 15 km/L
 5. 6 days 6. 25% 7. Rs. 44 800 8. 20%
MCQs: 1 (b) 2 (b) 3 (b) 4 (b) 5 (c)

1.4 Types and Sources of Data

Exercise:

1. (a) qualitative (b) discrete (c) continuous (d) discrete (e) qualitative (f) continuous
 2. (a) secondary (b) primary (c) primary
 3. Cross-sectional: marks of all students in one exam; time-series: Pakistan's yearly inflation 2015–2025 (any suitable examples).

MCQs: 1 (c) 2 (b) 3 (b) 4 (c) 5 (a)

1.5 Measurement Scales

Exercise:

1. (a) nominal (b) ratio (c) ordinal (d) interval (e) ratio (f) ordinal
2. Only the ratio scale (it has a true zero), e.g. 80 kg is twice 40 kg.
3. Shirt numbers are nominal labels; an “average shirt number” has no meaning.

MCQs: 1 (a) 2 (b) 3 (d) 4 (c)

1.6 Tabular and Graphical Presentation

Exercise:

1. Children: $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 5$, $3 \rightarrow 4$, $4 \rightarrow 2$, $5 \rightarrow 1$ (total 15); most common = 2 children.
2. Cricket 150° , Football 90° , Hockey 60° , Others 60° .
3. Boundaries 29.5–39.5, width 10, mid-point 34.5.
4. Bar chart: separate bars for categories; histogram: touching bars for continuous classes.

MCQs: 1 (c) 2 (b) 3 (b) 4 (c) 5 (c)

1.7 QR Exercises Using Number Knowledge

Exercise:

1. $200 \times 5 = 1000$ (exact 995.94) 2. $180 \times 1.5 = \text{Rs. } 270$ 3. 60 km/h; 4 hours
4. Rs. 4000 5. 15% 6. $20 \text{ m}^2 \times 1200 = \text{Rs. } 24\,000$

MCQs: 1 (b) 2 (b) 3 (c) 4 (c)

Unit 2: Fundamental Mathematical Concepts

2.1 Basics of Geometry

Exercise:

1. 108° 2. 50° 3. 12 cm 4. 135° 5. 12 sides 6. 132 cm

MCQs: 1 (c) 2 (b) 3 (b) 4 (c) 5 (a)

2.2 Sets and their Operations

Exercise:

1. $P \cup Q = \{1, 2, 3, 4, 6, 8, 10, 12\}$, $P \cap Q = \{2, 4, 6, 12\}$, $P - Q = \{1, 3\}$, $Q' = \{1, 3, 5, 7, 9, 11\}$
2. $\{2, 4, 6, 8, 10\}$ 3. $2^4 = 16$ 4. $100 - (60 + 45 - 20) = 15$
5. $(A \cap B)' = \{2, 3\}' = \{1, 4, 5\}$; $A' \cup B' = \{4, 5\} \cup \{1, 5\} = \{1, 4, 5\}$. Verified.

MCQs: 1 (b) 2 (c) 3 (b) 4 (b) 5 (b)

2.3 Relations, Functions and Graphs

Exercise:

1. $f(3) = 10$, $f(-2) = 5$ 2. Domain $\{1, 2, 3\}$, range $\{4, 5, 6\}$; yes, a function
3. Not a function: input 2 has two outputs 4. slope -3 , y -intercept 7
5. $m = \frac{8-2}{3-1} = 3$ 6. $500 + 300 = \text{Rs. } 800$

MCQs: 1 (b) 2 (b) 3 (c) 4 (c) 5 (b)

2.4 Exponents, Factoring and Simplifying

Exercise:

1. (a) 243 (b) $8a^6$ (c) 4 (d) $\frac{1}{16}$ 2. $11x - 7$
3. (a) $(x - 7)(x + 7)$ (b) $4x(x - 3)$ (c) $(x + 3)(x + 4)$ (d) $(x - 5)(x + 3)$ (e) $(3x + 4)(x + 2)$
4. $x + 2$ ($x \neq 2$)

MCQs: 1 (c) 2 (b) 3 (b) 4 (a) 5 (a)

2.5 Equations and Inequalities

Exercise:

1. (a) $x = 5$ (b) $x = 4$ 2. $x = 3, y = 1$ 3. (a) 3, 4 (b) $-4, 2$ 4. $x = 2 \pm \sqrt{3}$
 5. (a) $x > 3$ (b) $x \leq 2$ 6. $-3 < x < 3$ 7. $D = 1 - 4 = -3 < 0$: no real roots

MCQs: 1 (b) 2 (b) 3 (b) 4 (c) 5 (c)

2.6 QR Exercises Using Mathematical Concepts

Exercise:

1. $1600 + 800n = 1000n \Rightarrow n = 8$ classes 2. $6 \times 12 = 72 \text{ cm}^2$ 3. 7 and 8
 4. $t = 3 \text{ s}, h = 45 \text{ m}$ 5. $40 - 37 = 3$ students 6. $d \leq 10 \text{ km}$

MCQs: 1 (b) 2 (b) 3 (c) 4 (b)

Unit 3: Fundamental Statistical Concepts

3.1 Population and Sample

Exercise:

1. $k = 1200/60 = 20$ 2. 48 men, 32 women 3. (a) parameter (b) statistic
 4. Friends tend to share similar views, so the sample is not representative (biased).
 5. Saves time and money; populations may be very large or testing may destroy items.

MCQs: 1 (b) 2 (c) 3 (b) 4 (c) 5 (d)

3.2 Central Tendency and Dispersion

Exercise:

1. Mean 5, median 4, mode 4, range 7
 2. $\sum (x - \bar{x})^2 = 28$; $\sigma^2 = 5.6, \sigma \approx 2.37$; $s^2 = 7, s \approx 2.65$
 3. 8.5 4. $300/20 = 15$ 5. 76 6. Class X (smaller SD)

MCQs: 1 (c) 2 (a) 3 (b) 4 (b) 5 (c)

3.3 Rules of Counting

Exercise:

1. 12 2. 720; 120; 35 3. $\frac{5!}{2!2!} = 30$ 4. ${}^8C_2 = 28$ 5. ${}^5P_3 = 60$ 6. ${}^{10}P_3 = 720$

MCQs: 1 (b) 2 (b) 3 (b) 4 (b) 5 (c)

3.4 Basic Probability

Exercise:

1. $\frac{3}{6} = \frac{1}{2}$ 2. $\frac{1}{13}$ 3. $\frac{1}{2}$ 4. $\frac{3}{36} = \frac{1}{12}$ 5. 0.7 6. 0.7 7. $0.6 \times 0.6 = 0.36$

MCQs: 1 (a) 2 (c) 3 (c) 4 (c) 5 (b)

3.5 Random Variables and Distributions

Exercise:

1. $E(X) = 2.1, \text{Var}(X) = 4.9 - 4.41 = 0.49$ 2. $10k = 1 \Rightarrow k = 0.1$ 3. $\frac{1}{8}$
 4. Mean = 2, variance = $\frac{4}{3} \approx 1.33$ 5. Between 50 and 90

MCQs: 1 (b) 2 (b) 3 (b) 4 (b) 5 (c)

3.6 QR Exercises Using Statistical Concepts

Exercise:

1. Mean = 118 lakh, median = 50 lakh; the median is more typical (400 is an outlier).
 2. $25\,000/500 - 100 = 50 - 100 = -\text{Rs. } 50$ (expected loss of Rs. 50 per ticket)
 3. Physics $z = 2$, Chemistry $z = 0.5$: relatively better in Physics.

4. No – gym-goers are a biased sample; they exercise more than the general population.

5. $9 \times 10 \times 10 \times 10 = 9000$

MCQs: 1 (b) 2 (b) 3 (c) 4 (c)

“Numbers are not just to be calculated – they are to be understood, questioned and used to make good decisions.”